

Frankenstein Transformer: Biblioteca Unificada de Codificador-Decodificador, CLI y Notas de Diseño Basadas en Investigación

Erick F. Merino M.
Este trabajo no está afiliado
`erickfmm@gmail.com`

Febrero 2026

Resumen

Frankenstein Transformer presenta un conjunto de herramientas unificado y basado en configuración para la experimentación sistemática con arquitecturas modernas de transformers, abarcando diecisiete variantes de mezcladores de secuencia y veintitrés familias de optimizadores. El sistema admite tanto el modelado de lenguaje enmascarado (MLM) tipo codificador como la predicción autorregresiva (AR) del siguiente token tipo decodificador mediante una configuración flexible de clase de modelo y modo, con flujos de trabajo de ajuste fino especializados para ambas arquitecturas. Las contribuciones de investigación son tres: (i) un contrato de configuración estricto basado en esquemas que permite la experimentación reproducible a través de diversos mecanismos de atención, incluyendo atención softmax estándar, atención sigmoide, redes retentivas, modelos selectivos de espacio de estados, transformers de profundidad continua, enrutamiento adaptativo de profundidad (Mixture-of-Depths) [69], aumento de memoria condicional (Engram) [10], patrones de atención dispersa y mecanismos con compuerta; (ii) un marco integral de enrutamiento de optimizadores que admite métodos de reducción de varianza (MARS, Adan, AdEMAMix), variantes eficientes en memoria (Adafactor, GaLore, Lion), enfoques sin programación de tasa de aprendizaje, preconditionadores de segundo orden (Shampoo, SOAP, Sophia) y optimizadores de bajo rango de la familia APOLLO (Apollo, Apollo-Mini, Q-Apollo) [67]; y (iii) flujos de trabajo de extremo a extremo que abarcan el despliegue cuantizado mediante empaquetado ternario de pesos y el entrenamiento de incrustaciones de oraciones inspirado en SBERT, respaldados por una pila de aseguramiento de calidad ampliada con amplia cobertura de pruebas unitarias, validación de ejemplos YAML y ejecución de integración continua. El conjunto de herramientas implementa una interfaz de configuración basada en web que proporciona renderizado de formularios impulsado por esquemas con documentación en línea y validación en tiempo real. Este documento de referencia técnica incluye diagramas arquitectónicos, visualizaciones de flujo de ejecución, tablas de decisión y apéndices completos que sintetizan la literatura sobre arquitecturas de transformers, mecanismos de atención dispersa, arquitecturas con compuerta y familias de optimizadores.

Índice

1. Introducción	7
1.1. Motivación y Planteamiento del Problema	7
1.2. Contribuciones	8
1.3. Guía de Lectura	9
1.4. Constructor de Configuración Basado en Web	9

2. Trabajo Relacionado	10
2.1. Arquitecturas de Mezcladores de Secuencia	10
2.1.1. Líneas Base de Atención Densa	10
2.1.2. Arquitecturas Recurrentes y Retentivas	11
2.1.3. Mecanismos de Atención Dispersa	11
2.1.4. Mecanismos de Atención con Compuerta	11
2.2. Algoritmos de Optimización	11
3. Diseño y Arquitectura del Sistema	12
3.1. Arquitectura Centrada en Configuración	12
3.2. Alcance del Esquema y Reglas de Validación	14
3.3. Inventario Completo de Características del Modelo	14
3.4. Tipos de Tarea de Entrenamiento	15
3.5. Inventario Completo de Características de Entrenamiento	16
3.6. Contrato de Prefijo del Optimizador (Completo)	17
3.7. Seguridad de Entrenamiento y Semántica de Ejecución	18
3.8. Variantes de Normalización	18
4. Taxonomía de Arquitectura e Implementación	18
4.1. Familias de Atención y Mezcladores de Secuencias	18
4.2. Familias Densas y Recurrentes	19
4.3. Familia de Atención Latente y de Rango Bajo	19
4.4. Extensiones de Atención Dispersa	19
4.5. Extensiones de Atención con Compuerta	20
5. Familias de Optimizadores y Dinámicas de Entrenamiento	20
6. Cuantización y Despliegue	21
6.1. Cuantización Ternaria y Escalado de Activaciones	21
7. Tareas Posteriores de SBERT	21
8. Tablas Resumen	22
8.1. Resumen de Atención y Mezcladores de Secuencia	22
8.2. Resumen de Optimizadores	23
9. Discusión	24
9.1. Compromisos de Diseño Basado en Esquemas	24
9.2. Cobertura Arquitectónica y Vacíos	24
9.3. Fragmentación del Panorama de Optimizadores	25
9.4. Consideraciones de Despliegue y Producción	25
9.5. Desafíos de Integración y Extensibilidad	25
10. Conclusión	25
10.1. Limitaciones y Direcciones Futuras	26
Bibliografía	27
Anexos	

1. Familias de Optimizadores	32
1.1. Comparación de Memoria y Complejidad	32
1.2. La Evolución de la Optimización en Redes Neuronales	33
1.3. Línea Base Estándar y Optimizadores Adaptativos	33
1.4. Momentum Avanzado y Reducción de Varianza (2024–2025)	35
1.5. Optimizadores de Lote Grande, Eficientes en Memoria y Sin Tasa de Aprendizaje	39
1.6. Optimizadores de Segundo Orden, Geométricos y de Ortogonalidad	45
2. Transformadores Densos, Recurrentes y Aumentados con Memoria	50
2.1. Atención Densa de Referencia: Estándar y Sigmoides	50
2.1.1. Atención Softmax Estándar	50
2.1.2. Atención Sigmoides	51
2.1.3. Atención por Consultas Agrupadas (GQA)	51
2.2. Arquitecturas Recurrentes y Retentivas	52
2.2.1. Redes Retentivas (RetNet)	52
2.2.2. Mamba: Modelos de Espacio de Estado Selectivos	52
2.3. Transformers de Profundidad Continua: Integración EDO	53
2.3.1. Transformer EDO	53
2.4. Memoria en Tiempo de Prueba: Titans	54
2.5. Despacho de Mezclador Guiado por Patrón	54
2.6. Comparación y Síntesis Arquitectónica	54
3. Familias de Atención Latente y de Rango Bajo	56
3.1. Atención Latente Multi-Cabeza (MLA)	56
3.1.1. Formulación Matemática	56
3.1.2. Pseudocódigo Algorítmico	56
3.1.3. Características Clave	56
3.2. Atención Latente por Consultas Agrupadas (GQLA)	57
3.2.1. Formulación Matemática	57
3.2.2. Pseudocódigo Algorítmico	57
3.2.3. Características Clave	57
3.3. Atención Multi-Cabeza de Rango Bajo (MLRA)	58
3.3.1. Formulación Matemática	58
3.3.2. Pseudocódigo Algorítmico	58
3.3.3. Características Clave	58
3.4. Atención Tucker	59
3.4.1. Formulación Matemática	59
3.4.2. Pseudocódigo Algorítmico	59
3.4.3. Características Clave	59
3.5. Atención de Cabezas Entrelazadas (IHA)	59
3.5.1. Formulación Matemática	60
3.5.2. Pseudocódigo Algorítmico	60
3.5.3. Características Clave	60
3.6. Atención laTenT por Cabezas Agrupadas (GTA)	60
3.6.1. Formulación Matemática	60
3.6.2. Pseudocódigo Algorítmico	61
3.6.3. Características Clave	61
3.7. Atención Latente Temporal Multi-Cabeza (MTLA)	61
3.7.1. Formulación Matemática	61
3.7.2. Pseudocódigo Algorítmico	62
3.7.3. Características Clave	62
3.8. Comparación y Síntesis Arquitectónica	62

3.9.	Atención Convolutiva Comprimida (CCA)	62
3.9.1.	Formulación Matemática	62
3.9.2.	Características Clave	63
3.10.	Atención Convolutiva Comprimida por Consultas Agrupadas (CCGQA)	63
3.10.1.	Características Clave	63
3.11.	Comparación y Síntesis Arquitectónica	63
4.	Mecanismos de Atención Dispersa Exhaustivos	64
4.1.	Resumen Ejecutivo	64
4.2.	Sparse Transformer: Patrones Estrideados y Fijos Factorizados	65
4.2.1.	Formulación Matemática	65
4.2.2.	Pseudocódigo Algorítmico	66
4.2.3.	Características Clave	66
4.3.	Longformer: Ventana Deslizante, Dilatación y Tokens Globales	66
4.3.1.	Formulación Matemática	66
4.3.2.	Pseudocódigo Algorítmico	67
4.3.3.	Características Clave	67
4.4.	BigBird: Grafo Disperso Aleatorio, Local y Global	67
4.4.1.	Formulación Matemática	67
4.4.2.	Garantía Teórica	68
4.4.3.	Pseudocódigo Algorítmico	68
4.4.4.	Características Clave	68
4.5.	Atención SparseK: Selección Diferencial Top- k	69
4.5.1.	Formulación Matemática	69
4.5.2.	Pseudocódigo Algorítmico	69
4.5.3.	Características Clave	69
4.6.	NSA (Native Sparse Attention): Ramas Jerárquicas Alineadas con el Hardware	70
4.6.1.	Formulación Matemática	70
4.6.2.	Pseudocódigo Algorítmico	71
4.6.3.	Características Clave	71
4.7.	FASA: Atención Dispersa Consciente de la Frecuencia	71
4.7.1.	Formulación Matemática	71
4.7.2.	Pseudocódigo Algorítmico	72
4.7.3.	Características Clave	72
4.8.	SpargeAttn: Filtrado de Dos Etapas a Nivel de Bloques	73
4.8.1.	Formulación Matemática	73
4.8.2.	Pseudocódigo Algorítmico	74
4.8.3.	Características Clave	74
4.9.	MSA (MiniMax Sparse Attention): Atención Dispersa por Bloques con Rama de Índice Ligera	74
4.9.1.	Formulación Matemática	74
4.9.2.	Pseudocódigo Algorítmico	75
4.9.3.	Características Clave	76
4.10.	SparDA (Atención Dispersa Desacoplada): Selección de Bloques Anticipada Guiada por Pronóstico	76
4.10.1.	Formulación Matemática	76
4.10.2.	Pseudocódigo Algorítmico	77
4.10.3.	Características Clave	77
4.11.	Resumen Comparativo	77
4.12.	Espacio de Diseño y Criterios de Selección	78

5. Familias de Atención con Compuerta—Análisis Exhaustivo de Literatura	78
5.1. Resumen Ejecutivo: Compuertas para el Control de Memoria	78
5.2. 1. Atención Lineal con Compuerta (GLA)	79
5.2.1. Fundamento Matemático	79
5.2.2. Entrenamiento Eficiente en Hardware	80
5.2.3. Fortalezas y Limitaciones	80
5.3. 2. DeltaNet: Atención Lineal con Corrección de Errores	80
5.3.1. Fundamento Matemático	80
5.3.2. Entrenamiento Paralelo Eficiente	81
5.3.3. Fortalezas y Limitaciones	82
5.4. 3. Gated DeltaNet: Síntesis de Compuerta y Corrección de Errores	82
5.4.1. Fundamento Matemático	82
5.4.2. Rendimiento Empírico y Compensaciones	83
5.5. 4. HGRN2: Compuerta Jerárquica con Expansión de Producto Externo	83
5.5.1. Fundamento Matemático	83
5.5.2. Escalado y Resultados Empíricos	84
5.6. 5. Forgetting Transformer (FoX): Compuerta en el Espacio de Logits Softmax	84
5.6.1. Fundamento Matemático	84
5.6.2. Integración con FlashAttention	85
5.6.3. Fortalezas y Limitaciones	85
5.7. 6. Atención con Compuerta (Post-SDPA Sigmoide)	85
5.7.1. Fundamento Matemático	85
5.7.2. Hallazgos Clave	86
5.8. 7. Gated DeltaNet-2: Borrado y Escritura Canalizados Decouplados	86
5.8.1. Fundamento Matemático	86
5.8.2. Rendimiento Empírico y Compromisos	88
5.9. 8. Kimi Delta Attention (KDA): Compuerta de Decaimiento Canalizada para la Regla Delta	88
5.9.1. Fundamento Matemático	88
5.9.2. Rendimiento Empírico y Compromisos	89
5.10. Análisis Comparativo: Taxonomía de Mecanismos con Compuerta	89
5.11. Principio Unificado de Compuerta	91
5.12. Implementación y Orientación Práctica	92
5.12.1. Cuándo Usar Cada Arquitectura	92
5.12.2. Consideraciones de Hardware	92
6. Introducción Conceptual—Transformers y Atención para Principiantes	93
6.1. ¿Qué es un Transformer?	93
6.2. El Mecanismo de Atención: Una Analogía	93
6.3. Ejemplo Práctico Paso a Paso	93
6.3.1. Contexto 1: “El gato come pescado”	93
6.3.2. Contexto 2: “El restaurante come los márgenes de beneficio”	94
6.4. Cómo Funciona la Atención Matemáticamente (Versión Simple)	94
6.4.1. Paso 1: Consultas, Claves y Valores	94
6.4.2. Paso 2: Compatibilidad	94
6.4.3. Paso 3: Enfoque	94
6.4.4. Paso 4: Combinar	94
6.5. Múltiples Cabezas de Atención: Múltiples Perspectivas	95
6.6. Apilar Capas	95
6.7. ¿Por Qué es Importante?	95
6.8. Desafíos Prácticos	95

6.8.1.	Complejidad Computacional	95
6.8.2.	Requisitos de Memoria	96
6.8.3.	Eficiencia del Dispositivo	96
6.9.	Soluciones Modernas	96
6.10.	Conclusiones Clave	96
7.	Normalización, Cuantización, Enrutamiento de Profundidad y Algoritmos de Ejecución	97
7.1.	Variantes de Normalización: LayerNorm, RMSNorm, Tanh Dinámico y Erf Dinámico	97
7.1.1.	LayerNorm (Línea Base)	97
7.1.2.	RMSNorm	97
7.1.3.	RMSNorm Parcial (pRMSNorm)	97
7.1.4.	Tanh Dinámico (DyT)	98
7.1.5.	Erf Dinámico (Derf)	98
7.1.6.	Comparación	98
7.2.	Ruta de Cuantización Ternaria BitNet	98
7.3.	Enrutamiento de Tokens Mixture-of-Depths (MoD)	99
7.3.1.	Configuración	99
7.4.	Memoria Condicional Engram	99
7.4.1.	Configuración	99
7.5.	Embeddings Factorizados y Convolución de Embedding	100
7.5.1.	Embeddings Factorizados	100
7.5.2.	Convolución de Embedding	100
7.6.	Controles de Estabilidad del Paso de Entrenamiento	100
7.7.	Enrutador de Modo de Inferencia SBERT	100

1. Introducción

1.1. Motivación y Planteamiento del Problema

Las arquitecturas de transformers han transformado fundamentalmente el panorama del modelado de secuencias en el procesamiento del lenguaje natural [51], la visión por computadora y la biología computacional. El éxito de modelos como BERT [14], GPT y sus variantes ha establecido al transformer como el estándar de facto para el aprendizaje de representaciones en el aprendizaje profundo. Sin embargo, la rápida proliferación de innovaciones arquitectónicas presenta desafíos prácticos significativos para investigadores y profesionales.

Los últimos años han sido testigos de una explosión de mecanismos alternativos de atención y mezcla de secuencias, cada uno abordando limitaciones específicas de la atención softmax estándar: complejidad computacional cuadrática [11], requisitos de inferencia eficiente en memoria [49, 18], gestión selectiva de estados basada en contenido [4] y optimizaciones conscientes del hardware [42]. Simultáneamente, la literatura sobre optimización se ha diversificado más allá del AdamW clásico, introduciendo técnicas de reducción de varianza [55, 38, 61], variantes eficientes en memoria [45, 66], enfoques sin programación de tasa de aprendizaje [12] y métodos de preconditionamiento de segundo orden [20, 52].

La consecuencia práctica es un ecosistema de investigación fragmentado donde la comparación experimental entre arquitecturas y optimizadores requiere un esfuerzo de ingeniería significativo. Los investigadores deben implementar y depurar múltiples variantes desde cero, garantizar tuberías de entrenamiento consistentes y gestionar espacios de hiperparámetros complejos. Esta fragmentación dificulta la reproducibilidad, ralentiza el progreso científico y aumenta la barrera de entrada para nuevos investigadores.

Este trabajo aborda estos desafíos mediante un conjunto de herramientas de experimentación unificado y basado en configuración, disponible en <https://github.com/erickfmm/frankestein-transformer>, que proporciona:

1. **Diseño Basado en Esquemas:** Un contrato de configuración estricto y validado que garantiza la reproducibilidad mientras admite diecisiete arquitecturas de mezcladores de secuencia distintas y veintitrés familias de optimizadores.
2. **Entrenamiento Agnóstico a la Arquitectura:** Infraestructura de entrenamiento común que soporta líneas base de atención densa (estándar, sigmoide), alternativas recurrentes (RetNet, Mamba, bloques tipo EDO), enrutamiento adaptativo de profundidad (Mixture-of-Depths) [69], bloques de memoria condicional (Engram) [10], patrones de atención dispersa (Sparse Transformer, Longformer, BigBird, SparseK, NSA, SpargeAttn, FASA) y mecanismos con compuerta (GLA, DeltaNet, Gated DeltaNet, Gated DeltaNet-2, HGRN2, FoX, Gated Softmax).
3. **Marco de Enrutamiento de Optimizadores:** Grupos de hiperparámetros con prefijos que permiten un control detallado sobre incrustaciones, capas de normalización, bloques recurrentes, bloques de atención y otros subconjuntos de parámetros a través de diversos optimizadores, incluyendo la familia APOLLO (Apollo, Apollo-Mini, Q-Apollo) [67].
4. **Flujos de Trabajo de Extremo a Extremo:** Despliegue integrado mediante cuantización (empaquetado ternario de pesos, activaciones INT8) y capacidades de incrustación de oraciones inspiradas en SBERT [43].
5. **Configuración Interactiva:** Interfaz basada en web que proporciona renderizado de formularios impulsado por esquemas, validación en tiempo real y generación de comandos CLI.
6. **Herramientas de Confiabilidad:** Pruebas automatizadas integrales con suites de pruebas unitarias, validación de presets YAML y automatización de CI en versiones de Python compatibles.

La superficie de comandos del proyecto es:

`frankestein-transformer`

con subcomandos para flujos de trabajo de codificador y decodificador: `train`, `finetune`, `deploy`, `quantize`, `infer`, `sbert-train`, `sbert-infer`, `web-server`.

El comando `web-server` lanza un constructor de configuración basado en Streamlit que proporciona:

- Campos de formulario impulsados por esquemas con títulos de parámetros y descripciones detalladas
- Información sobre herramientas en tiempo real y texto de ayuda para cada opción de configuración
- Vista previa YAML en vivo y funcionalidad de descarga
- Comandos CLI generados para entrenamiento, despliegue, inferencia y flujos de trabajo SBERT

Esta interfaz interactiva sirve como alternativa a la edición manual de YAML, mejorando la usabilidad para usuarios que exploran las opciones de configuración disponibles y comprenden su impacto en el comportamiento del modelo y la dinámica de entrenamiento.

1.2. Contribuciones

Este trabajo realiza las siguientes contribuciones principales:

1. **Esquema de Configuración Unificado:** Un contrato de esquema basado en YAML con validación estricta que soporta diecisiete arquitecturas de mezcladores de secuencia distintas en cuatro categorías (líneas base densas, bloques recurrentes/retentivos, patrones de atención dispersa y mecanismos con compuerta), junto con controles de profundidad adaptativa y memoria condicional, mientras garantiza la reproducibilidad mediante restricciones `additionalProperties: false`.
2. **Soporte Integral de Arquitecturas:** Implementación de variantes modernas de transformers incluyendo atención softmax estándar [51], atención sigmoide [42], RetNet [49], Mamba [18], transformers continuos tipo EDO [64], atención con aumento de memoria Titans [4], capas de memoria condicional Engram [10], enrutamiento de tokens Mixture-of-Depths [69], mecanismos de atención dispersa [11, 5, 62, 33, 60, 65, 54] y arquitecturas de atención con compuerta [56, 57, 58, 21, 40, 28, 41]. El sistema soporta tanto entrenamiento en modo codificador con atención bidireccional para modelado de lenguaje enmascarado (MLM) como entrenamiento en modo decodificador con enmascaramiento causal para predicción autorregresiva del siguiente token.
3. **Marco de Enrutamiento de Optimizadores:** Sistema de hiperparámetros con prefijos que permite control por grupo de parámetros a través de veintitrés optimizadores, incluyendo Anon con adaptabilidad ajustable [2], métodos de reducción de varianza (MARS, Adan, AdEMAMix, Cautious AdamW), variantes eficientes en memoria (Adafactor, GaLore, Lion), enfoques sin programación de tasa de aprendizaje (Schedule-Free AdamW), métodos conscientes de curvatura (Sophia), preconditionadores de segundo orden (Shampoo, SOAP), optimizadores orientados a ortogonalidad (Muon, Turbo-Muon), escalado de lotes grandes (LAMB) y la familia APOLLO (Apollo, Apollo-Mini, Q-Apollo) [67].
4. **Cuantización y Despliegue:** Tubería de despliegue integrada que soporta empaquetado ternario de pesos y cuantización de activaciones INT8 con estimaciones de tamaño siguiendo la aproximación de almacenamiento de 1,58 bits.

5. **Flujos de Trabajo de Incrustación de Oraciones:** Tuberías de entrenamiento e inferencia inspiradas en SBERT que soportan puntuación de similitud, recuperación, agrupamiento y exportación persistente de incrustaciones.
6. **Interfaz de Configuración Interactiva:** Servidor web basado en Streamlit que proporciona generación de formularios impulsada por esquemas, validación en tiempo real, documentación en línea y generación de comandos CLI.
7. **Tubería de Validación Automatizada:** Integración continua y pruebas unitarias ampliadas que verifican las rutas de entrenamiento del modelo, la integración optimizador/esquema y la compatibilidad de ejemplos YAML.

1.3. Guía de Lectura

Este documento está organizado como una referencia técnica que aborda cuatro preocupaciones operativas:

1. **Requisitos Previos:** El Apéndice 6 proporciona una introducción accesible a los transformers y mecanismos de atención para lectores nuevos en el campo.
2. **Contrato de Configuración:** La Sección 3.1 describe el esquema YAML que garantiza experimentos válidos y la Sección 3.2 explica las reglas de validación.
3. **Selección de Arquitectura:** La Sección 4.1 proporciona una comparación exhaustiva de las familias de mezcladores de secuencia; los Apéndices 2, 3, 4 y 5 sintetizan la literatura de respaldo; el Apéndice 7 detalla las variantes de normalización, la cuantización BitNet, el enrutamiento Mixture-of-Depths y la memoria condicional Engram.
4. **Dinámica de Optimización:** La Sección 5 detalla el enrutamiento de optimizadores y la dinámica de entrenamiento; el Apéndice 1 proporciona un análisis exhaustivo de las familias de optimizadores.
5. **Despliegue e Inferencia:** Las Secciones 6 y 7 describen el despliegue cuantizado y los flujos de trabajo de incrustación de oraciones.

1.4. Constructor de Configuración Basado en Web

Además de la edición directa de YAML, este proyecto proporciona una interfaz web basada en Streamlit (accesible mediante el comando `web-server`) que mejora la accesibilidad y descubribilidad de la configuración. La interfaz presenta campos de esquema con:

- **Renderizado de formularios impulsado por esquemas** — Todos los campos se generan dinámicamente a partir del esquema autoritativo, garantizando consistencia y validación.
- **Documentación de parámetros en línea** — Cada campo del formulario muestra un *título* del esquema como su etiqueta, con la *descripción* mostrada como información sobre herramientas al pasar el cursor.
- **Vista previa de configuración en tiempo real** — Los usuarios ven la salida YAML en vivo mientras modifican los campos del formulario, permitiendo retroalimentación de validación inmediata.
- **Generación de comandos CLI** — La interfaz genera comandos CLI completos para entrenamiento, despliegue, inferencia y flujos de trabajo SBERT basados en la configuración actual.

- **Mejoras de accesibilidad** — La información sobre herramientas y los formularios estructurados facilitan la comprensión de las opciones de configuración, especialmente para usuarios nuevos en el proyecto o que exploran arquitecturas novedosas.

Este enfoque basado en web aborda barreras de usabilidad comunes en la experimentación impulsada por configuración:

- Reduce la necesidad de memorizar la estructura YAML y los nombres de los campos
- Previene errores tipográficos mediante la validación del esquema
- Proporciona contexto educativo a través de documentación en línea
- Permite la experimentación rápida con ajuste guiado de parámetros
- Sirve tanto como herramienta de configuración como recurso de aprendizaje

La implementación de la interfaz web utiliza widgets de formulario de Streamlit con:

- `st.checkbox()` para alternativas binarias con texto de ayuda
- `st.number_input()` para campos numéricos con tamaño de paso y formato
- `st.selectbox()` para opciones de enumeración con visualización de opciones
- `st.multiselect()` para selecciones de arreglo a partir de opciones definidas
- `st.text_input()` para campos de cadena
- `st.info()`, `st.caption()` para información complementaria

Los metadatos del esquema (campos de título y descripción) se extraen y renderizan sistemáticamente en todas las secciones del formulario, incluyendo:

- Parámetros de arquitectura del modelo (tamaño oculto, capas, cabezas de atención, etc.)
- Configuraciones de ejecución de entrenamiento (tamaño de lote, acumulación, programador)
- Configuración del optimizador con hiperparámetros por grupo de parámetros
- Opciones de despliegue y cuantización
- Parámetros específicos de entrenamiento e inferencia SBERT

2. Trabajo Relacionado

Este trabajo se sitúa en la intersección de tres áreas activas de investigación: arquitecturas de atención alternativas, mecanismos de atención dispersa y algoritmos de optimización avanzados. Esta sección proporciona un estudio conciso; las formulaciones matemáticas detalladas y las descripciones algorítmicas se difieren a los apéndices.

2.1. Arquitecturas de Mezcladores de Secuencia

2.1.1. Líneas Base de Atención Densa

La atención softmax estándar [51] logra un contexto global completo mediante interacciones por pares de n^2 , pero incurre en costos prohibitivos de memoria y cómputo para secuencias largas. La atención sigmoide [42] reemplaza la softmax por filas con una sigmoide elemento a elemento, logrando una convergencia más rápida y hasta un 17% de aceleración del núcleo, aunque requiere estabilización con normalización híbrida a escala. Ambas sirven como líneas base densas en este conjunto de herramientas.

2.1.2. Arquitecturas Recurrentes y Retentivas

RetNet [49] resuelve el “triángulo imposible” de entrenamiento paralelo, inferencia $\mathcal{O}(1)$ y rendimiento sólido mediante una formulación dual de atención-retención. Mamba [18] introduce selectividad dependiente de la entrada en modelos de espacio de estados, logrando complejidad lineal con algoritmos de exploración conscientes del hardware. Los transformers tipo EDO [64] tratan la profundidad como integración numérica sobre un sistema dinámico continuo. Titans [4] aumenta la inferencia con memorización neuronal en tiempo de prueba para contextos extremadamente largos. En el Apéndice 2 se proporcionan formulaciones detalladas.

2.1.3. Mecanismos de Atención Dispersa

Los métodos de atención dispersa reducen el cuello de botella cuadrático restringiendo las interacciones entre tokens: Sparse Transformer [11] utiliza patrones factorizados con paso y fijos ($\mathcal{O}(n\sqrt{n})$); Longformer [5] emplea ventanas deslizantes con tokens globales ($\mathcal{O}(n \cdot w)$); BigBird [62] combina caminos locales, aleatorios y globales; SparseK [33] aplica selección diferenciable top- k ; NSA [60] utiliza un diseño jerárquico de tres ramas; SpargeAttn [65] realiza poda a nivel de bloques en dos etapas; y FASA [54] aprovecha características de frecuencia de RoPE para la selección de tokens. Las descripciones completas se encuentran en el Apéndice 4.

2.1.4. Mecanismos de Atención con Compuerta

Las arquitecturas con compuerta controlan el flujo de información mediante compuertas aprendibles: GLA [56] agrega compuertas diagonales dependientes de datos a la atención lineal; DeltaNet [57] aplica la regla de aprendizaje delta a las actualizaciones de estado recurrente; Gated DeltaNet [58] sintetiza ambos mecanismos; Gated DeltaNet-2 [21] desacopla la compuerta delta escalar en compuertas de borrado canalizadas (eje de claves) y escritura canalizada (eje de valores); HGRN2 [40] introduce compuertas de olvido jerárquicas con expansión de estado mediante producto exterior; FoX [28] incrusta compuertas de olvido directamente en la atención softmax; y Gated Softmax [41] aplica compuertas de canal posteriores a SDPA. Las derivaciones matemáticas completas aparecen en el Apéndice 5.

2.2. Algoritmos de Optimización

La optimización de arquitecturas de transformers altamente parametrizadas requiere navegar paisajes de pérdida no convexos con espectros de Hessiana heterogéneos. La literatura se ha diversificado en varias familias más allá de la línea base clásica de AdamW [32]:

- **Reducción de varianza y momento:** RAdam [30], Adan [55], ADOPT [50], AdEMAMix [38], MARS [61] y optimizadores Cautious [27] abordan la inestabilidad en pasos iniciales, las garantías de convergencia y la mezcla de historial con EMA dual.
- **Eficientes en memoria:** Adafactor [45], GaLore [66], Lion [9] y APOLLO [67] reducen la memoria del estado del optimizador mediante factorización, proyección de bajo rango o actualizaciones basadas en signo.
- **Sin programación de tasa y libres de parámetros:** Schedule-Free AdamW [12] y Prodigy [36] absorben el programador o el ajuste de la tasa de aprendizaje en la dinámica del optimizador.
- **De segundo orden y conscientes de curvatura:** Shampoo [20], SOAP [52] y Sophia [29] incorporan información aproximada de segundo orden.
- **Orientados a geometría:** Muon [46] y Turbo-Muon [6] remodelan la geometría de actualización mediante ortogonalización.

En el Apéndice 1 se proporcionan descripciones algorítmicas detalladas, pseudocódigo y una comparación exhaustiva de complejidad.

3. Diseño y Arquitectura del Sistema

3.1. Arquitectura Centrada en Configuración

La decisión de diseño principal en este repositorio es que la experimentación es *esquema primero*. En lugar de exponer una gran cantidad de indicadores laxamente verificados, el proyecto obliga a la topología del modelo, la familia de optimizadores, los límites de entrenamiento y las opciones de telemetría a través de un único documento de configuración validado. Esto reduce la ambigüedad al reproducir resultados y permite comparar muchas arquitecturas bajo una interfaz operativa consistente.

El contrato autoritativo es `configs/schema.yaml`. Este impone tres objetos de primer nivel:

- `model_class`
- `model`
- `training`

Selección de Clase de Modelo. El campo `model_class` determina la variante arquitectónica instanciada por el pipeline de entrenamiento. Se soportan tres opciones:

- **frankenstein:** Modelos encoder de arquitectura mixta que soportan diversos mecanismos de atención (estándar, sigmoide, retentiva, espacio de estados, dispersa y mezcladores con compuerta) con MoE (Mezcla de Expertos) y características avanzadas. Optimizado para entrenamiento bidireccional tipo encoder con objetivos de modelado de lenguaje enmascarado (MLM).
- **mini:** Variante de encoder simplificada diseñada para escenarios de entrenamiento a menor escala. Proporciona una sobrecarga de parámetros reducida e iteración más rápida para experimentación y creación de prototipos.
- **frankesteindecoder:** Decoder causal autorregresivo para generación de siguiente token estilo LLM. Permite el enmascaramiento de atención causal para tareas de generación secuencial de texto. Cuando se selecciona esta clase, en tiempo de ejecución se fuerza `mode='decoder'`.

Selección de Modo de Entrenamiento. El campo `model.mode` controla el comportamiento de enmascaramiento de atención en todo el modelo:

- **encoder:** Utiliza atención bidireccional donde todos los tokens atienden a todos los demás tokens en la secuencia. Adecuado para tareas de preentrenamiento de modelado de lenguaje enmascarado (MLM) donde el modelo aprende a predecir tokens enmascarados aleatoriamente basándose en el contexto completo.
- **decoder:** Utiliza enmascaramiento causal donde cada token solo puede atender a tokens anteriores en la secuencia. Requerido para tareas autorregresivas (AR) de predicción del siguiente token, como modelado de lenguaje y generación de texto. Cuando `model_class='frankesteindecoder'`, el sistema fuerza automáticamente `mode='decoder'` en tiempo de ejecución.

Este soporte de arquitectura dual permite al sistema manejar tanto preentrenamiento tipo encoder (MLM en contextos bidireccionales) como generación tipo decoder (predicción causal autorregresiva) a través de una interfaz de configuración unificada.

El `model.layer_pattern` soporta bloques heredados, dispersos y con compuerta:

- **Retentive Network (RetNet)** — referencia interna: `sun_retentive_2023` — nombre de código: `retnet`, `retnet_attn`
- **Mamba (Selective State Space Model)** — referencia interna: `gu_mamba_2023` — nombre de código: `mamba`
- **Bloque de Profundidad Continua estilo ODE** — referencia interna: `zhang_continuous_2021` — nombre de código: `ode`
- **Atención con Memoria Aumentada Titans** — referencia interna: `behrouz_titans_2025` — nombre de código: `titan_attn`
- **Atención Softmax Estándar** — referencia interna: `vaswani_attention_2017` — nombre de código: `standard_attn`
- **Auto-Atención Sigmoide** — referencia interna: `ramapuram_theory_2024` — nombre de código: `sigmoid_attn`
- **Transformer Disperso** — referencia interna: `child_sparse_transformer_2019` — nombre de código: `sparse_transformer_attn`
- **Longformer** — referencia interna: `beltagy_longformer_2020` — nombre de código: `longformer_attn`
- **BigBird** — referencia interna: `zaheer_bigbird_2020` — nombre de código: `bigbird_attn`
- **Atención SparseK** — referencia interna: `lou_sparsek_2024` — nombre de código: `sparsek_attn`
- **Atención Dispersa Nativa (NSA)** — referencia interna: `yuan_nsa_2025` — nombre de código: `nsa_attn`
- **SpargeAttn** — referencia interna: `zhang_spargeattn_2025` — nombre de código: `sparge_attn`
- **FASA (Frequency-aware Sparse Attention)** — referencia interna: `wang_fasa_2026` — nombre de código: `fasa_attn`
- **Atención Lineal con Compuerta (GLA)** — referencia interna: `yang_gla_2023` — nombre de código: `gla_attn`
- **DeltaNet** — referencia interna: `yang_deltanet_2024` — nombre de código: `deltanet_attn`
- **DeltaNet con Compuerta** — referencia interna: `yang_gated_deltanet_2024` — nombre de código: `gated_deltanet_attn`
- **Gated DeltaNet-2** — referencia interna: `hatamizadeh_gated_deltanet2_2026` — nombre de código: `gated_deltanet2_attn`
- **HGRN2** — referencia interna: `qin_hgrn2_2024` — nombre de código: `hgrn2_attn`
- **Forgetting Transformer (FoX)** — referencia interna: `lin_forgetting_transformer_2025` — nombre de código: `fox_attn`
- **Atención Softmax con Compuerta** — referencia interna: `qiu_gated_attention_2025` — nombre de código: `gated_softmax_attn`

que corresponde a la literatura actual de atención y mezcladores de secuencias [49, 18, 64, 4, 42, 51, 11, 5, 62, 33, 60, 65, 54, 56, 57, 58, 21, 40, 28, 41]

El `training.optimizer.optimizer_class` soporta una amplia familia de optimizadores: `sgd_momentum`, `adamw`, `adafactor`, `galore_adamw`, `prodigy`, `lion`, `sophia`, `muon`, `turbo_muon`, `radam`, `adan`, `adopt`, `ademamix`, `mars_adamw`, `cautious_adamw`, `lamb`, `schedulefree_adamw`, `shampoo`, `soap`, `anon`, `apollo`, `apollo_mini`, y `q_apollo`.

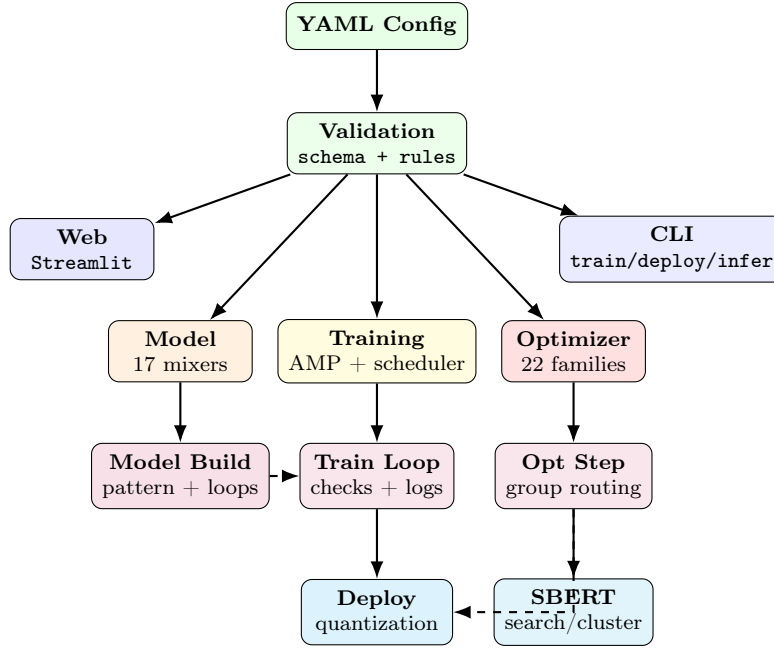


Figura 1: Arquitectura del sistema: la configuración fluye a través de la validación, se divide en modelo/entrenamiento/optimizador, ejecuta en tiempo real y produce artefactos de despliegue/SBERT.

3.2. Alcance del Esquema y Reglas de Validación

El esquema es estricto: los objetos de primer nivel y anidados establecen `additionalProperties: false`. Esto garantiza que las claves desconocidas fallen rápidamente en lugar de ser ignoradas silenciosamente. El objeto `training.optimizer.parameters` está adicionalmente restringido por reglas de prefijo específicas del optimizador mediante verificaciones de patrón `allOf+if/then`.

Los valores de normalización actualmente aceptados por el esquema son:

$$\text{norm_type} \in \{\text{layer_norm}, \text{dynamic_tanh}, \text{derf}\}$$

Por lo tanto, `rms_norm` **no** es un valor de esquema válido en el contrato actual.

3.3. Inventario Completo de Características del Modelo

Campo	Tipo/Rango	Significado
<code>model_class</code>	enum	Variante arquitectónica: <code>frankenstein</code> (encoder de arquitectura mixta), <code>mini</code> (encoder simplificado), <code>frankesteindecoder</code> (decoder autorregresivo).
<code>model.mode</code>	enum	Modo de atención: <code>encoder</code> (bidireccional, para MLM) o <code>decoder</code> (causal, para AR). Cuando <code>model_class='frankesteindecoder'</code> , fuerza <code>mode='decoder'</code> .
<code>vocab_size</code>	$\text{int} \geq 1$	Tamaño del vocabulario.
<code>hidden_size</code>	$\text{int} \geq 1$	Dimensión oculta.
<code>num_layers</code>	$\text{int} \geq 1$	Número de capas físicas.
<code>num_loops</code>	$\text{int} \geq 1$	Número de bucles lógicos (bloques en bucle).
<code>num_heads</code>	$\text{int} \geq 1$	Cabezas de atención.
<code>retention_heads</code>	$\text{int} \geq 1$	Cabezas de retención para mezcladores estilo RetNet.

Campo	Tipo/Rango	Significado
num_experts	int ≥ 1	Número de expertos en MoE.
top_k_experts	int ≥ 1	Enrutamiento top- k de expertos en MoE.
dropout	float [0, 1]	Dropout global.
layer_pattern	array enum	Lista ordenada de bloques: heredados (retnet, retnet_attn, mamba, ode, titan_attn, standard_attn, sigmoid_attn), dispersos (sparse_transformer_attn, longformer_attn, bigbird_attn, sparsek_attn, nsa_attn, sparge_attn, fasa_attn), y con compuerta (gla_attn, deltaret_attn, gated_deltaret_attn, gated_deltaret2_attn, hgrn2_attn, fox_attn, gated_softmax_attn).
ode_solver	enum	rk4 o euler.
ode_steps	int ≥ 1	Pasos de integración ODE.
use_bitnet	bool	Habilitar ruta BitLinear de bajo bit.
norm_type	enum	layer_norm, dynamic_tanh, derf.
use_factorized_embedding	bool	Habilitar embeddings factorizados.
factorized_embedding_dim	int ≥ 1	Dimensión reducida de embedding para factorización.
use_embedding_conv	bool	Habilitar Conv1d sobre el flujo de embeddings.
embedding_conv_kernel	int ≥ 1	Tamaño del kernel Conv1d.
hope_base	float ≥ 0	Valor base de HoPE (opcional en el esquema).
hope_damping	float ≥ 0	Amortiguamiento de HoPE (opcional en el esquema).
use_hope	bool	Aplicar HoPE en titan_attn.
use_moe	bool	Habilitar ruta FFN con MoE.
ffn_hidden_size	int ≥ 1	Ancho intermedio de la FFN.
ffn_activation	enum	silu o gelu.

La profundidad en bucle inducida por el esquema es:

$$L_{\text{logical}} = \text{num_layers} \times \text{num_loops}$$

que es la definición a nivel de configuración de los bloques en bucle.

3.4. Tipos de Tarea de Entrenamiento

El campo `training.task` determina el objetivo de entrenamiento, trabajando en conjunto con `model.mode` para definir cómo aprende el modelo:

Modelado de Lenguaje Enmascarado (MLM).

- **Modo:** Requiere `mode='encoder'` para atención bidireccional.
- **Objetivo:** Enmascarar aleatoriamente tokens en la secuencia de entrada (típicamente 15%) y entrenar al modelo para predecir los tokens enmascarados basándose en el contexto bidireccional completo.
- **Caso de Uso:** Preentrenamiento de encoders para aprendizaje de representaciones, siguiendo la metodología BERT [14]. El modelo aprende representaciones bidireccionales que capturan contexto tanto de izquierda como de derecha.
- **Configuración:** Utiliza el parámetro `mlm_probability` para controlar la fracción de enmascaramiento.

Predicción Autorregresiva (AR) del Siguiente Token.

- **Modo:** Requiere `mode='decoder'` para enmascaramiento causal.
- **Objetivo:** Entrenar al modelo para predecir el siguiente token en la secuencia dados solo los tokens anteriores.
- **Caso de Uso:** Generación de lenguaje y tareas estilo LLM siguiendo la metodología GPT. El modelo aprende dependencias secuenciales con atención causal donde cada token solo puede atender a tokens precedentes.
- **Clase de Modelo:** Típicamente usa `model_class='frankesteindecoder'` para arquitecturas de decoder autorregresivo.

Este soporte de tareas dual permite la experimentación unificada tanto en preentrenamiento tipo encoder (MLM para comprensión bidireccional) como en generación tipo decoder (AR para producción secuencial de texto) dentro del mismo código base.

3.5. Inventario Completo de Características de Entrenamiento

Campo	Tipo/Rango	Significado
<code>batch_size</code>	$\text{int} \geq 1$	Tamaño de lote del cargador.
<code>dataloader_workers</code>	$\text{int} \geq 0$	Trabajadores del dataloader de PyTorch.
<code>max_length</code>	$\text{int} \geq 1$	Límite de longitud de secuencia.
<code>task</code>	enum	Objetivo de entrenamiento: <code>mlm</code> (modelado de lenguaje enmascarado) o <code>sbert</code> (embedding de oraciones).
<code>mlm_probability</code>	float [0,1]	Probabilidad de enmascaramiento MLM (aplica solo cuando <code>task='mlm'</code>).
<code>max_samples</code>	$\text{int} \geq 1$	Máximo de muestras transmitidas.
<code>dataset_batch_size</code>	$\text{int} \geq 1$	Tamaño interno de fragmento del conjunto de datos en streaming.
<code>num_workers</code>	$\text{int} \geq 0$	Trabajadores del conjunto de datos en streaming.
<code>cache_dir</code>	string	Directorio de caché del conjunto de datos.
<code>local_parquet_dir</code>	string	Ruta opcional local a parquet.
<code>prefer_local_cache</code>	bool	Preferir caché local cuando esté disponible.
<code>stream_local_parquet</code>	bool	Modo de transmisión desde parquet local.
<code>use_amp</code>	bool	Alternar precisión mixta.
<code>gradient_accumulation_steps</code>	$\text{int} \geq 1$	Lote efectivo mediante acumulación.
<code>optimizer</code>	object	Contiene <code>optimizer_class</code> y <code>parameters</code> con prefijo.
<code>scheduler_total_steps</code>	$\text{int} \geq 1$	Horizonte del scheduler.
<code>scheduler_warmup_ratio</code>	float [0,1]	Proporción de calentamiento.
<code>scheduler_type</code>	enum	<code>cosine</code> , <code>constant</code> , <code>linear_warmup_then_constant</code> .
<code>grad_clip_max_norm</code>	float ≥ 0	Umbral de recorte de norma global.
<code>inf_post_clip_threshold</code>	float ≥ 0	Guarda contra gradientes explosivos después del recorte.
<code>max_nan_retries</code>	$\text{int} \geq 0$	Presupuesto de reintentos para inestabilidad NaN/Inf.
<code>checkpoint_every_n_steps</code>	$\text{int} \geq 1$	Frecuencia de checkpoint rodante.
<code>max_rolling_checkpoints</code>	$\text{int} \geq 1$	Número de checkpoints rodantes a mantener.
<code>num_best_checkpoints</code>	$\text{int} \geq 1$	Número de mejores checkpoints rastreados.
<code>nan_check_interval</code>	$\text{int} \geq 1$	Cadencia de verificación NaN/Inf.

Campo	Tipo/Rango	Significado
log_gradient_stats	bool	Habilitar registro de estadísticas de gradientes.
gradient_log_interval	int ≥ 1	Cadencia de registro de gradientes.
csv_log_path	string	Ruta de salida CSV a nivel de paso.
csv_rotate_on_schema_change	bool	Rotar CSV si el esquema de registro cambia.
gpu_metrics_backend	enum	nvml o none.
nvml_device_index	int ≥ 0	Índice de dispositivo para telemetría NVML.
enable_block_grad_norms	bool	Incluir telemetría de norma de gradiente por bloque.
telemetry_log_interval	int ≥ 1	Intervalo de telemetría pesada (pasos de optimizador).
use_galore	bool	Habilitar estrategia GaLore.
galore_rank	int ≥ 1	Dimensión de proyección de bajo rango de GaLore.
galore_update_interval	int ≥ 1	Intervalo de actualización de proyección.
galore_scale	float ≥ 0	Escalado de gradientes en el espacio proyectado.
galore_max_dim	int ≥ 1	Dimensión máxima del tensor para proyección GaLore.

3.6. Contrato de Prefijo del Optimizador (Completo)

Los valores soportados de `optimizer_class` son: `sgd_momentum`, `adamw`, `adafactor`, `galore_adamw`, `prodigy`, `lion`, `sophia`, `muon`, `turbo_muon`, `radam`, `adan`, `adopt`, `ademamix`, `mars_adamw`, `cautious_adamw`, `lamb`, `schedulefree_adamw`, `shampoo`, `soap`, `apollo`, `apollo_mini`, `q_apollo`.

Las familias de sufijos compartidos por grupo (todos prefijados por el nombre del optimizador) son:

- Grupos de LR: `lr_embeddings`, `lr_norms`, `lr_ode`, `lr_retnet`, `lr_mamba`, `lr_attention`, `lr_other`
- Grupos de decaimiento de peso: `wd_embeddings`, `wd_norms`, `wd_ode`, `wd_retnet`, `wd_mamba`, `wd_attention`, `wd_other`
- Grupos de betas: `betas_embeddings`, `betas_norms`, `betas_ode`, `betas_retnet`, `betas_mamba`, `betas_attention`, `betas_other`
- Grupos de épsilon: `eps_embeddings`, `eps_norms`, `eps_ode`, `eps_retnet`, `eps_mamba`, `eps_attention`, `eps_other`

Sufijos globales específicos del optimizador:

- `sgd_momentum`: `momentum`, `nesterov`
- `adafactor`: `beta2_decay`, `clip_threshold`, `eps1`, `eps2`
- `galore_adamw`: `rank`, `update_proj_gap`
- `prodigy`: `d_coef`
- `sophia`: `rho`, `update_k`
- `muon` / `turbo_muon`: `momentum`, `nesterov`, `ns_steps`, `ns_eps`
- `cautious_adamw`: `cautious_clip`
- `apollo`: `rank`, `update_proj_gap`, `scale`, `scale_type`, `proj_type`, `scale_front`, `disable_nl`

- `apollo_mini`: `update_proj_gap`, `scale`, `proj_type`, `scale_front`, `disable_nl`
- `q_apollo`: `rank`, `update_proj_gap`, `scale`, `scale_type`, `proj_type`, `scale_front`, `disable_nl`, `quant_bits`

Todas las demás clases en la lista anterior aceptan solo grupos compartidos con prefijo.

3.7. Seguridad de Entrenamiento y Semántica de Ejecución

Las características de seguridad a nivel de esquema incluyen acumulación, recorte, verificación de explosión post-recorte y reintentos por NaN:

$$g_{\text{acc}} = \frac{1}{K} \sum_{i=1}^K g_i, \quad K = \text{gradient_accumulation_steps}$$

$$g_{\text{clip}} = g_{\text{acc}} \cdot \min \left(1, \frac{\tau}{\|g_{\text{acc}}\|_2 + \epsilon} \right), \quad \tau = \text{grad_clip_max_norm}$$

luego los guardas de desbordamiento usan `inf_post_clip_threshold` y lógica de reintentos limitada por `max_nan_retries`.

El pseudocódigo completo del paso de entrenamiento guiado por esquema—incluyendo acumulación de gradientes, recorte de norma global, guardas de explosión post-recorte, reintentos acotados de NaN/Inf, despacho del scheduler y rotación de checkpoints rodantes/mejores—se difiere al Apéndice 7.

3.8. Variantes de Normalización

La normalización controla la escala de activación a través de la profundidad y es también una cuestión de compatibilidad con el esquema, ya que los valores aceptados de `norm_type` son `layer_norm`, `dynamic_tanh`, `derf`, `rms_norm` y `prms_norm` (RMSNorm parcial). Las formulaciones matemáticas de LayerNorm, RMSNorm, pRMSNorm, Tanh Dinámico (DyT) [68] y Erf Dinámico (Derf) [8], junto con una tabla comparativa, se detallan en el Apéndice 7.

4. Taxonomía de Arquitectura e Implementación

4.1. Familias de Atención y Mezcladores de Secuencias

Este sistema implementa diecisiete arquitecturas de mezcladores de secuencias distintas organizadas en cinco categorías funcionales que reflejan las tendencias de investigación en el diseño de modelado de secuencias. La organización taxonómica refleja la comprensión evolutiva de cómo equilibrar expresividad, eficiencia computacional y restricciones de memoria.

1. **Líneas Base de Atención Densas:** La atención softmax estándar y la atención sigmoide proporcionan contextualización global completa a costo computacional cuadrático, sirviendo como líneas base de referencia para comparación con alternativas más eficientes.
2. **Arquitecturas Recurrentes y Retentivas:** RetNet, Mamba, bloques estilo ODE y Titans mantienen representaciones de estado que permiten un costo de inferencia $\mathcal{O}(1)$ mientras preservan la expresividad mediante dinámicas recurrentes, parámetros selectivos o adaptación de memoria en tiempo de prueba.
3. **Patrones de Atención Dispersa:** Siete variantes dispersas (Sparse Transformer, Longformer, BigBird, SparseK, NSA, SpargeAttn, FASA) reducen la complejidad cuadrática mediante dispersión estructurada, selección de tokens o estrategias de poda sin entrenamiento.

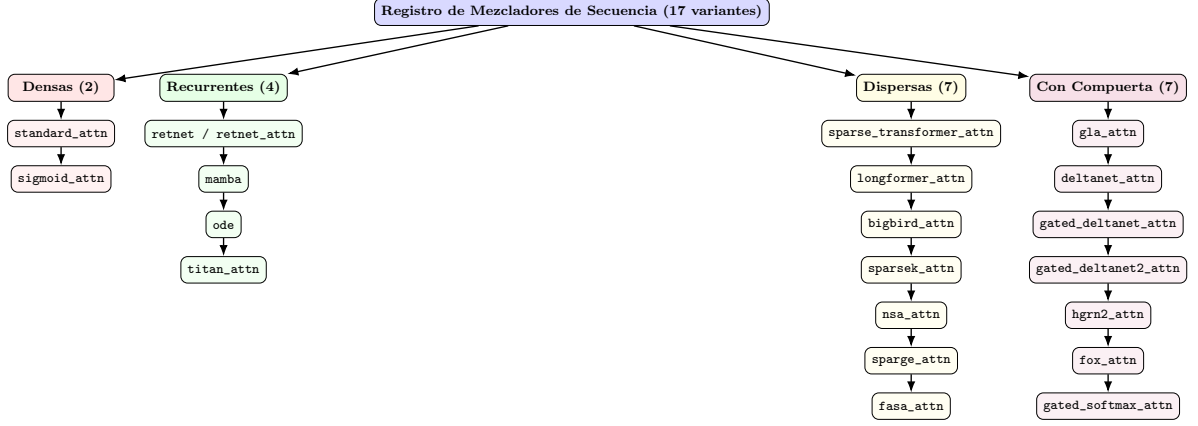


Figura 2: Taxonomía completa de las diecisiete arquitecturas de mezcladores de secuencias soportadas. Las líneas base densas proporcionan enrutamiento global completo a costo cuadrático. Las arquitecturas recurrentes permiten inferencia en tiempo constante mediante compresión de estado. Las variantes dispersas reducen la complejidad mediante patrones estructurados o selección de tokens. Los mecanismos con compuerta introducen control dependiente de datos sobre la retención y el olvido de memoria.

4. **Mecanismos de Memoria con Compuerta:** Siete arquitecturas con compuerta (GLA, DeltaNet, Gated DeltaNet, Gated DeltaNet-2, HGRN2, FoX, Gated Softmax) introducen control dependiente de datos sobre la retención de memoria, el olvido y la fuerza de actualización.

4.2. Familias Densas y Recurrentes

Las líneas base densas (softmax estándar [51] y sigmoide [42]) y la familia recurrente/retentiva (RetNet [49], Mamba [18], bloques continuos estilo ODE [64] y memoria en tiempo de prueba Titans [4]) proporcionan la compensación fundamental entre expresividad y eficiencia. Las formulaciones matemáticas, las formas paralela y recurrente, el pseudocódigo algorítmico y una tabla comparativa arquitectónica se recopilan en el Apéndice 2. El algoritmo de despacho de mezclador guiado por patrón (incluyendo la política de cumplimiento sin entrenamiento para `fasa_attn` y `sparge_attn`) se detalla igualmente allí.

4.3. Familia de Atención Latente y de Rango Bajo

Una familia complementaria comprime el estado clave-valor por token en un latente de rango bajo: Atención Latente Multi-Cabeza (MLA), Atención Latente por Consultas Agrupadas (GQLA), Atención Multi-Cabeza de Rango Bajo (MLRA), Atención Tucker, Atención de Cabezas Entrelazadas (IHA), Atención laTenT por Cabezas Agrupadas (GTA) y Atención Latente Temporal Multi-Cabeza (MTLA). Estos métodos unifican diseños de consulta agrupada, latente, factorizado y latente temporal bajo un cuello de botella de rango bajo común. Las formulaciones completas, el pseudocódigo y una tabla comparativa se recopilan en el Apéndice 3.

4.4. Extensiones de Atención Dispersa

El registro de mezcladores incluye siete bloques dispersos: `sparse_transformer_attn`, `longformer_attn`, `bigbird_attn`, `sparsek_attn`, `nsa_attn`, `sparge_attn` y `fasa_attn` [11, 5, 62, 33, 60, 65, 54]. La implementación impone una política de ejecución explícita para métodos dispersos sin entrenamiento: `fasa_attn` y `sparge_attn` son solo para evaluación/inferencia y generan errores

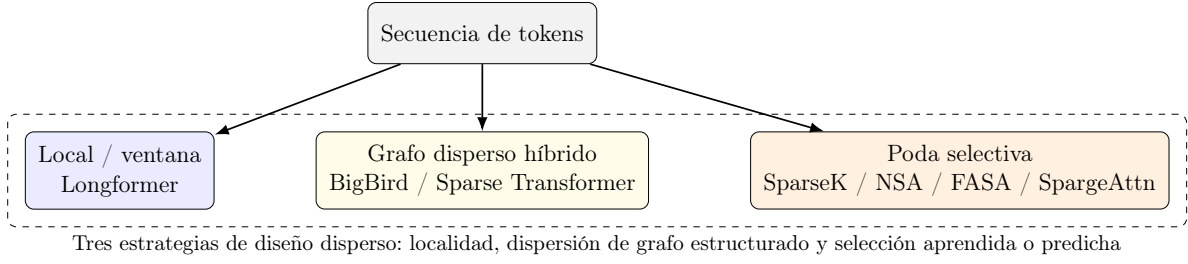


Figura 3: Mapa conceptual de las estrategias de diseño de atención dispersa utilizadas en el código base. Diferentes métodos reducen el costo restringiendo vecindarios, construyendo grafos dispersos o seleccionando solo tokens/bloques de alto valor.

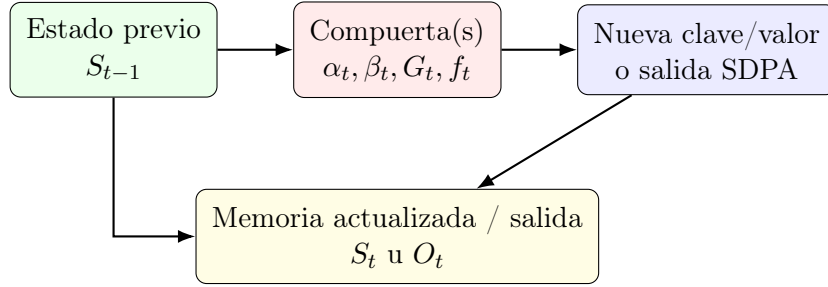


Figura 4: Plantilla de compuerta genérica. Una compuerta puede atenuar la memoria existente, regular la fuerza de escritura o modular las salidas de atención densa, dependiendo de la familia del bloque.

en tiempo de ejecución si se usan mientras el modelo está en modo de entrenamiento. Las formulaciones matemáticas, el pseudocódigo algorítmico, las características por bloque y una tabla comparativa se recopilan en el Apéndice 4.

4.5. Extensiones de Atención con Compuerta

El registro de mezcladores incluye ocho bloques con compuerta: `gla_attn`, `deltanet_attn`, `gated_deltanet_attn`, `gated_deltanet2_attn`, `hgrn2_attn`, `fox_attn` y `gated_softmax_attn` [56, 57, 58, 21, 40, 28, 41]. La idea unificadora es que las compuertas controlan *qué información sobrevive*: algunas compuertas actúan sobre actualizaciones de estado recurrente (GLA, variantes de DeltaNet, HGRN2), mientras que otras modifican la ruta de atención completa (FoX y Gated Softmax). Esto hace que las compuertas sean especialmente útiles cuando el modelo debe equilibrar recuerdo, actualidad y memoria limitada. Las formulaciones matemáticas, el pseudocódigo algorítmico, una plantilla de compuerta genérica y una tabla comparativa se recopilan en el Apéndice 5.

5. Familias de Optimizadores y Dinámicas de Entrenamiento

La optimización de arquitecturas transformer altamente parametrizadas presenta desafíos significativos debido a paisajes de pérdida no convexos, puntos de silla y heterogeneidad de bloques entre grupos de parámetros. Este sistema aborda estos desafíos mediante un marco unificado que soporta veintitrés familias de optimizadores que abarcan seis categorías algorítmicas: (1) líneas base clásicas (SGD, AdamW), (2) momento avanzado y reducción de varianza (Adan, ADOPT, AdEMAMix, MARS, Cautious), (3) variantes eficientes en memoria (Adafactor, GaLore, Lion, APOLLO, APOLLO-Mini, Q-APOLLO), (4) métodos sin programación y sin parámetros (Schedule-Free AdamW, Prodigy) más escalado de lotes grandes mediante LAMB, (5)

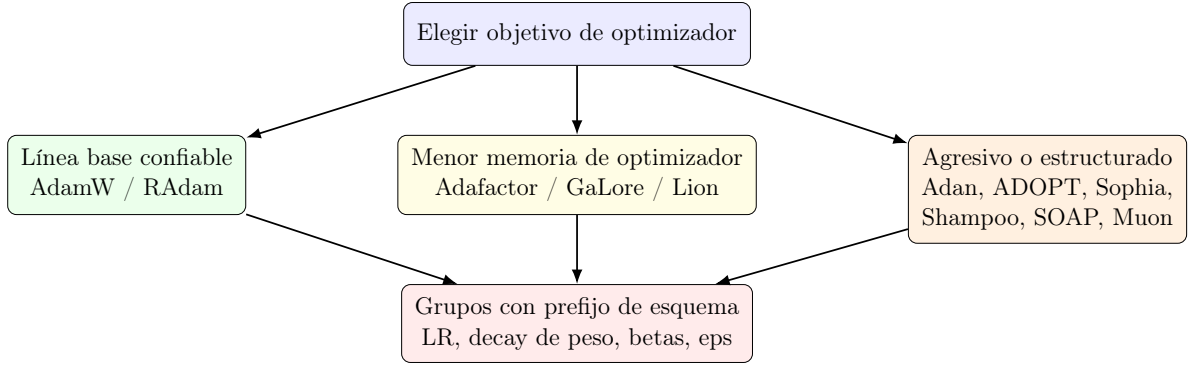


Figura 5: Selección de optimizador en la práctica: la clase determina la regla de actualización, mientras que el esquema controla cómo se aplican los hiperparámetros entre embeddings, normas, bloques recurrentes, bloques de atención y otros parámetros.

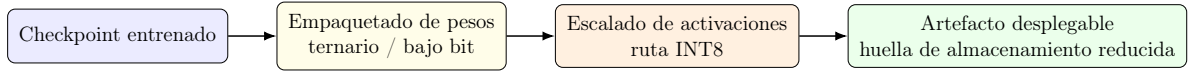


Figura 6: Ruta de despliegue desde un checkpoint entrenado hasta un artefacto compacto. El código base trata la cuantización como una transformación de la etapa de despliegue en lugar de una familia de modelos separada.

conscientes de curvatura y segundo orden (Sophia, Shampoo, SOAP), (6) orientados a geometría (Muon, Turbo-Muon), y (7) optimizadores con adaptabilidad ajustable (Anon). Las descripciones algorítmicas detalladas, pseudocódigo y una tabla comparativa de memoria/complejidad para todos los optimizadores se proporcionan en el Apéndice 1.

6. Cuantización y Despliegue

El stack de despliegue utiliza empaquetado ternario de pesos más cuantización de activaciones INT8 para producir artefactos eficientes.

6.1. Cuantización Ternaria y Escalado de Activaciones

La etapa de despliegue transforma un checkpoint entrenado en un artefacto compacto mediante empaquetado ternario de pesos estilo BitNet [53], escalado de activaciones INT8 y aproximación de almacenamiento de 1.58 bits alineada con objetivos de despliegue ligero [44, 7]. Las formulaciones matemáticas del escalado ternario de pesos, la cuantización/des-cuantización de activaciones INT8 y las estimaciones de tamaño FP32/FP16/1.58 bits se diferencian al Apéndice 7.

7. Tareas Posteriores de SBERT

El embedding de oraciones se construye sobre entrenamiento estilo Siamese [43]. Para el par de oraciones (s_1, s_2) con embeddings (e_1, e_2) :

$$\cos(e_1, e_2) = \frac{e_1^\top e_2}{\|e_1\| \|e_2\|}$$

y pérdida coseno estilo regresión:

$$\mathcal{L}_{\cos} = (\cos(e_1, e_2) - y)^2$$

con $y \in [-1, 1]$ en este pipeline.

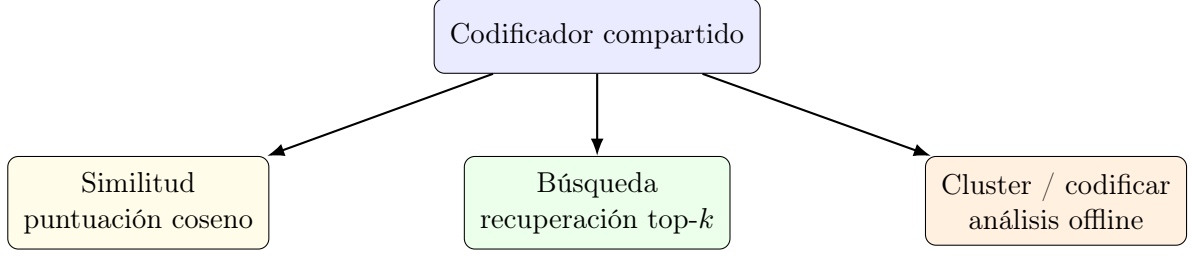


Figura 7: Reutilización del flujo de trabajo SBERT. Un único codificador soporta puntuación de pares en línea, recuperación sobre corpus, clustering y exportación persistente de embeddings.

Modos posteriores soportados:

- **Similitud:** puntuación por pares entre dos oraciones.
- **Búsqueda:** k vecinos más cercanos sobre un corpus.
- **Clustering:** agrupación de embeddings (ej., k-means).
- **Codificación:** exportación persistente de embeddings para recuperación posterior.

El pseudocódigo del enrutador de modo de inferencia SBERT—despachando entre los modos de similitud, búsqueda, cluster y codificación sobre un codificador compartido—se difiere al Apéndice 7.

8. Tablas Resumen

8.1. Resumen de Atención y Mezcladores de Secuencia

Tipo		Ecuación Princi- pal	Entrenar	Inferir	Notas
Atención	Es-	$\text{softmax}(QK^\top/\sqrt{d_k})V$	$\mathcal{O}(n^2d)$	$\mathcal{O}(n)/\text{paso}$	Enrutamiento global expresivo de línea base [51].
Atención	Sig-	$\sigma(QK^\top/\sqrt{d_k} + b)V$	$\mathcal{O}(n^2d)$	$\mathcal{O}(n)/\text{paso}$	Compuerta elemento a elemen- to; a menudo necesita norma de estabilización [42].
RetNet		$(QK^\top \odot D)V$	$\mathcal{O}(n^2d)$ o por fragmentos	$\mathcal{O}(1)/\text{paso}$	Forma dual paralela/recurrente con retención de decaimiento [49].
Mamba		$h_t = \bar{A}_t h_{t-1} + \bar{B}_t x_t$	$\mathcal{O}(nd)$	$\mathcal{O}(1)/\text{paso}$	Espacio de estados selectivo con barrido consciente de hardware [18].
Bloque EDO	estilo	$\frac{dh}{dt} = f_\theta(h, t)$	depende del solver	depende del solver	Interpretación de profundidad continua; integración RK [64].
Memoria Tians	Ti-	$M_t = (1 - \alpha_t)M_{t-1} + S_t$	aprox. $\mathcal{O}(nd)$	centrado en recuperación	Actualizaciones de memoria en tiempo de prueba con dinámica impulsada por sorpresa [4].
CCA		$\text{softmax}(\hat{Q}\hat{K}^\top/\sqrt{d_h})\tilde{V}$ en latente	$\mathcal{O}(n^2d/C)$	$\mathcal{O}(n)/\text{paso}$	Atención completa en latente comprimido; convs + qk-mean + value-shift; $C \times$ menos FLOPs+params+caché [16].

Tipo	Ecuación Principal	Principio	Entrenar	Inferir	Notas
CCGQA	Igual que CCA + GQA desacoplada		$\mathcal{O}(n^2 d / C_1)$	$\mathcal{O}(n)$ /paso	CCA con compresión KV $C_2 \geq C_1$ y cabezas latentes compartidas por grupo; mejor pérdida a $8\times$ reducción de caché [16].

8.2. Resumen de Optimizadores

Optimizador	Familia	Costo de Estado	Idea Clave	Ref
SGD+Momentum	Primer orden clásico	Bajo	Línea base acelerada por momento con estado mínimo	[39]
AdamW	Adaptativo primer/segundo momento	Alto	Línea base con decaimiento de peso desacoplado	[32]
RAdam	Adaptativo con corrección de varianza	Alto	Rectifica la varianza adaptativa temprana	[30]
Adan	Momento + reducción de varianza	Alto	Actualización adaptativa estilo Nesterov	[55]
ADOPT	Variante de Adam	Alto	Actualizaciones reordenadas con mejores garantías de convergencia	[50]
AdEMAMix	Adaptativo multi-EMA	Alto	Mezcla EMAs de horizonte corto y largo	[38]
MARS	Precondicionado con reducción de varianza	Alto	Corrección de momento recursiva	[61]
Cautious AdamW	Momento enmascarado	Alto	Aplica actualizaciones solo en direcciones consistentes con signo	[27]
LAMB	Momentos adaptativos por capa	Alto	Escalado de razón de confianza para entrenamiento con lotes muy grandes	[59]
Schedule-free AdamW	Adaptativo sin programador	Alto	Elimina la dependencia de programación LR explícita	[12]
Adafactor	Adaptativo eficiente en memoria	Medio	Segundos momentos factorizados para tensores matriciales	[45]
GaLore AdamW	Proyección de gradiente de bajo rango	Medio	Optimiza en espacio de gradiente de bajo rango proyectado	[66]
APOLLO	Proyección adaptativa de bajo rango	Medio	Escalado estructurado a partir de momentos estilo Adam proyectados	[67]
APOLLO-Mini	Proyección adaptativa rango-1	Bajo	Variante APOLLO escalada por tensor para ahorro extremo de memoria	[67]
Q-APOLLO	Proyección de bajo rango cuantizada	Bajo	Estado APOLLO cuantizado para entrenamiento de memoria ultra-baja	[67]
Anon	Similar a Adam con adaptabilidad ajustable	Alto	Adaptabilidad $\gamma \in \mathbb{R}$ ajustable continuamente con actualización de retardo incremental (IDU) para convergencia	[2]

Optimizador	Familia		Costo de Estado	Idea Clave	Ref
Prodigy	Adaptación sin parámetros		Medio	Calibración de paso adaptativa a distancia	[36]
Lion	Momento de signo		Bajo	Actualización de signo de momento, estado reducido	[9]
Sophia	Segundo orden aproximado		Medio	Precondicionamiento diagonal de Hessiana con recorte	[29]
Shampoo	Precondicionador matricial		Alto	Estadísticas de segundo orden estructuradas Kronecker	[20]
SOAP	Shampoo + base Adam		Alto	Seguimiento similar a Adam en base propia del preconditionador	[52]
Muon	Basado en ortogonalidad		Medio	Actualizaciones matriciales ortogonalizadas	[46]
Turbo-Muon	Ortogonalización acelerada		Medio	Aceleración Newton-Schulz preconditionada	[6]

9. Discusión

Las decisiones de diseño en Transformer Encoder Frankenstein reflejan varias tensiones de ingeniería e investigación en las herramientas modernas de aprendizaje profundo.

9.1. Compromisos de Diseño Basado en Esquemas

El enfoque basado en esquemas proporciona beneficios significativos de reproducibilidad al imponer contratos explícitos y fallar rápidamente ante configuraciones inválidas. Sin embargo, este enfoque también introduce rigidez: agregar nuevas arquitecturas u optimizadores requiere extensiones del esquema en lugar de argumentos de línea de comandos flexibles. El sistema de hiperparámetros prefijados permite un control detallado pero aumenta la complejidad de configuración para usuarios acostumbrados a interfaces más simples.

La decisión de imponer `additionalProperties: false` en todos los niveles del esquema elimina la absorción silenciosa de parámetros que ha afectado a sistemas de configuración anteriores, pero esta rigurosidad requiere un mantenimiento cuidadoso del esquema al extender las capacidades del sistema. Cada nuevo mecanismo de atención o variante de optimizador debe integrarse adecuadamente en el marco de validación, incluyendo definiciones de campos del esquema con tipos y restricciones apropiados, mapeo de hiperparámetros prefijados para grupos específicos de optimizadores, valores predeterminados alineados con las mejores prácticas de investigación y cadenas de documentación para la representación en la interfaz web.

9.2. Cobertura Arquitectónica y Vacíos

Las diecisiete arquitecturas de mezclador implementadas abarcan las principales direcciones de investigación en modelado de secuencias, pero ciertos vacíos permanecen. El sistema carece de arquitecturas híbridas recientes como Griffin [47] y Jamba [19], que combinan compuertas con modelos de espacio de estados. El enrutamiento MoE (Mezcla de Expertos) está implementado para capas FFN pero no para el cómputo de atención, donde trabajos recientes han mostrado beneficios [26].

La cobertura de atención dispersa es completa, pero la implementación de métodos sin entrenamiento (FASA, SpargeAttn) genera errores en tiempo de ejecución durante el entrenamiento, reflejando restricciones arquitectónicas: estos métodos requieren checkpoints preentrenados de modelos de atención completa o procedimientos específicos de ajuste fino que actualmente no están automatizados.

La cobertura de mecanismos de compuerta es sólida en todas las categorías principales (GLA, DeltaNet, Gated DeltaNet, Gated DeltaNet-2, HGRN2, FoX, Gated Softmax).

9.3. Fragmentación del Panorama de Optimizadores

El soporte para veintidós familias de optimizadores en seis categorías algorítmicas demuestra exhaustividad pero también resalta el estado fragmentado de la investigación en optimización. Los usuarios enfrentan una complejidad significativa de decisión al elegir entre métodos de reducción de varianza (Adan, MARS), variantes eficientes en memoria (GaLore, Adafactor, APOLLO) y enfoques conscientes de curvatura (Sophia, Shampoo). El sistema de hiperparámetros prefijados, aunque poderoso, requiere comprender qué parámetros son relevantes para cada clase de optimizador.

La calidad de implementación varía entre optimizadores: los métodos clásicos (AdamW, SGD con momento) están altamente optimizados en PyTorch, mientras que los métodos más nuevos (Muon, Turbo-Muon, SOAP) pueden requerir implementaciones personalizadas que afectan la estabilidad numérica y las características de rendimiento.

9.4. Consideraciones de Despliegue y Producción

El pipeline de cuantización demuestra preocupaciones prácticas de despliegue pero realiza compromisos de ingeniería específicos. El empaquetado ternario de pesos reduce el almacenamiento a aproximadamente 1,58 bits por parámetro, pero esta compresión agresiva puede degradar el rendimiento, especialmente para modelos más pequeños donde el error de cuantización es más significativo. La implementación actual aplica cuantización uniforme entre todos los tipos de parámetros.

Los flujos de trabajo SBERT proporcionan utilidad práctica para tareas de similitud semántica y recuperación, pero la implementación asume estrategias de pooling estándar (token CLS, pooling promedio). Avances recientes como los embeddings Matryoshka [37] y los refinamientos de aprendizaje contrastivo [22] aún no están incorporados.

9.5. Desafíos de Integración y Extensibilidad

La estructura actual del código base, aunque funcional, presenta desafíos de mantenimiento a medida que se expanden las familias de arquitecturas y optimizadores. El patrón de despachador para la selección de mezcladores y el enrutamiento de optimizadores maneja la extensibilidad pero corre el riesgo de convertirse en un “cajón de sastre” de lógica condicional. Las versiones futuras se beneficiarían de arquitecturas basadas en plugins donde nuevos mezcladores y optimizadores puedan registrarse de forma declarativa en lugar de modificar la lógica central de despacho.

La interfaz de configuración web proporciona mejoras significativas de usabilidad pero introduce complejidad de despliegue: ejecutar Streamlit junto con trabajos de entrenamiento requiere recursos adicionales y consideraciones de infraestructura que pueden no ser apropiadas para todos los entornos, particularmente clústeres HPC sin acceso web.

10. Conclusión

Transformer Encoder Frankenstein presenta una plataforma de experimentación unificada, impulsada por configuración, que aborda desafíos críticos en la investigación moderna de aprendizaje profundo: fragmentación arquitectónica entre atención densa, modelos recurrentes, patrones dispersos y mecanismos de compuerta; complejidad del panorama de optimizadores que abarca líneas base clásicas, métodos de reducción de varianza, variantes eficientes en memoria, enfoques

sin programación, algoritmos conscientes de curvatura y métodos orientados a geometría; y flujos de trabajo de despliegue de extremo a extremo que abarcan cuantización y aplicaciones de embeddings de oraciones.

Las contribuciones principales del sistema son:

1. **Diseño Basado en Esquemas:** Un contrato de configuración estricto basado en YAML con validación y enrutamiento de hiperparámetros prefijados que permite experimentos reproducibles en diecisiete arquitecturas de mezclador y veintitrés familias de optimizadores.
2. **Soporte Arquitectónico Integral:** Implementación que abarca las principales categorías de investigación incluyendo líneas base densas (atención estándar, sigmoide), alternativas recurrentes (RetNet, Mamba, estilo EDO, Titans), atención dispersa (Sparse Transformer, Longformer, BigBird, SparseK, NSA, SpargeAttn, FASA) y mecanismos de compuerta (GLA, DeltaNet, Gated DeltaNet, Gated DeltaNet-2, HGRN2, FoX, Gated Softmax).
3. **Marco Unificado de Optimizadores:** Grupos de hiperparámetros prefijados que permiten control detallado sobre embeddings, capas de normalización, bloques recurrentes, pesos de atención y parámetros FFN en líneas base clásicas (SGD+Momentum, AdamW), reducción de varianza (Adan, ADOPT, AdEMAMix, MARS, Cautious), eficientes en memoria (Adafactor, GaLore, Lion, APOLLO, APOLLO-Mini, Q-APOLLO), lotes grandes y simplificación de programación (LAMB, Schedule-Free AdamW, Prodigy), conscientes de curvatura (Sophia), segundo orden (Shampoo, SOAP) y orientados a geometría (Muon, Turbo-Muon).
4. **Flujos de Trabajo de Extremo a Extremo:** Pipeline de despliegue integrado que soporta empaquetado ternario de pesos y cuantización de activaciones INT8; entrenamiento e inferencia inspirados en SBERT para similitud semántica, recuperación y tareas de clustering.
5. **Configuración Interactiva:** Interfaz web basada en Streamlit que proporciona generación de formularios impulsada por esquemas, validación en tiempo real, documentación en línea y síntesis de comandos CLI.

Este sistema permite iteración experimental rápida mientras mantiene reproducibilidad mediante contratos de configuración estrictos. Al consolidar diversas contribuciones de investigación en un conjunto de herramientas unificado, reduce las barreras para explorar arquitecturas y estrategias de optimización novedosas, particularmente para investigadores que pueden carecer de recursos para implementar y validar cada variante de forma independiente.

10.1. Limitaciones y Direcciones Futuras

Varias limitaciones y direcciones prometedoras para trabajo futuro surgen del diseño e implementación de este sistema:

1. **Integración Arquitectónica:** Arquitecturas híbridas recientes (Griffin, Jamba, Mamba-X) demuestran beneficios de combinar múltiples mecanismos en bloques unificados. Versiones futuras deberían integrar estas arquitecturas y explorar patrones de composición sistemática.
2. **Cuantización Avanzada:** La implementación actual utiliza empaquetado ternario uniforme en todos los parámetros. La investigación sobre cuantización por capas, por canales y consciente de importancia sugiere que estrategias más sofisticadas podrían mejorar los compromisos calidad-eficiencia.
3. **Extensibilidad Basada en Plugins:** El patrón de despacho actual se vuelve cada vez más complejo con cada nueva adición. Una arquitectura de plugins que permita el registro declarativo de nuevos mezcladores, optimizadores y métodos de normalización mejoraría el mantenimiento y reduciría el riesgo de errores en la lógica central de despacho.

4. **Optimización Automatizada de Hiperparámetros:** El esquema soporta espacios extensos de hiperparámetros, pero los usuarios deben explorar estos espacios manualmente. La integración con optimización bayesiana, estrategias de bandidos multi-brazo o ajuste de hiperparámetros basado en gradientes podría automatizar el descubrimiento de configuraciones efectivas.
5. **Despliegue en Producción:** La interfaz web mejora la usabilidad pero puede no ser apropiada para todos los entornos de despliegue. Modos de configuración sin interfaz gráfica, gestión de configuración basada en API o ergonomía CLI mejorada podrían servir a flujos de trabajo HPC y de producción.
6. **Evaluación Comparativa:** Si bien el sistema permite entrenamiento con diversas arquitecturas, una evaluación comparativa exhaustiva que compare el rendimiento entre mezcladores y optimizadores en tareas estandarizadas proporcionaría orientación valiosa para la selección de configuraciones.
7. **Garantías de Estabilidad de Entrenamiento:** La implementación actual incluye guardas contra NaN/Inf y recorte de gradiente, pero el análisis formal de condiciones de estabilidad para diferentes combinaciones mezclador-optimizador, particularmente con bloques en bucle y cuantización agresiva, sigue abierto.
8. **Extensiones Multimodales y Específicas de Tarea:** El diseño actual se centra en modelado de secuencias. Extensiones para modelos de visión-lenguaje, arquitecturas multimodales y flujos de trabajo de ajuste fino específicos de tarea (ej., ajuste por instrucciones, RLHF) ampliarían la aplicabilidad.

La trayectoria de investigación del modelado de secuencias continúa hacia enfoques híbridos que combinan fortalezas de múltiples paradigmas—compresión de la recurrencia, selectividad de la atención, compuertas para gestión de memoria y dispersión para eficiencia. Una plataforma de experimentación unificada como Transformer Encoder Frankenstein es cada vez más valiosa a medida que esta convergencia se acelera, permitiendo a los investigadores explorar sistemáticamente este creciente espacio de diseño con infraestructura reproducible y bien diseñada.

Bibliografía

- [1] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyansky, Federico Lebrón, and Sumit Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints, 2023. URL <https://arxiv.org/abs/2305.13245>.
- [2] Anonymous authors. Anon: Extrapolating adaptivity beyond sgdr and adam, 2026. URL <https://arxiv.org/abs/2605.02317>.
- [3] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization, 2016. URL <https://arxiv.org/abs/1607.06450>.
- [4] Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. Titans: Learning to memorize at test time. URL <https://arxiv.org/abs/2501.00663>. Version Number: 1.
- [5] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer, 2020. URL <https://arxiv.org/abs/2004.05150>.
- [6] Thibaut Boissin, Thomas Massena, Franck Mamalet, and Mathieu Serrurier. Turbo-muon: Accelerating orthogonality-based optimization with pre-conditioning. URL <https://arxiv.org/abs/2512.04632>. Version Number: 1.

- [7] Riccardo Bravin, Massimo Pavan, Hazem Hesham Yousef Shalby, Fabrizio Pittorino, and Manuel Roveri. EmbBERT: Attention under 2 MB memory. URL <http://arxiv.org/abs/2502.10001>.
- [8] Mingzhi Chen, Taiming Lu, Jiachen Zhu, Mingjie Sun, and Zhuang Liu. Stronger normalization-free transformers, . URL <http://arxiv.org/abs/2512.10938>.
- [9] Xiangning Chen, Chen Liang, Da Huang, Esteban Real, Kaiyuan Wang, Yao Liu, Hieu Pham, Xuanyi Dong, Thang Luong, Cho-Jui Hsieh, Yifeng Lu, and Quoc V. Le. Symbolic discovery of optimization algorithms, . URL <https://arxiv.org/abs/2302.06675>. Version Number: 4.
- [10] Xin Cheng, Wangding Zeng, Damai Dai, Qinyu Chen, Bingxuan Wang, Zhenda Xie, Kezhao Huang, Xingkai Yu, Zhewen Hao, Yukun Li, Han Zhang, Huishuai Zhang, Dongyan Zhao, and Wenfeng Liang. Conditional memory via scalable lookup: A new axis of sparsity for large language models, 2026. URL <https://arxiv.org/abs/2601.07372>.
- [11] Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers, 2019. URL <https://arxiv.org/abs/1904.10509>.
- [12] Aaron Defazio, Xingyu Alice Yang, Harsh Mehta, Konstantin Mishchenko, Ahmed Khaled, and Ashok Cutkosky. The road less scheduled. URL <https://arxiv.org/abs/2405.15682>. Version Number: 4.
- [13] Keqi Deng and Philip C. Woodland. Multi-head temporal latent attention, 2025. URL <https://arxiv.org/abs/2505.13544>.
- [14] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. URL <http://arxiv.org/abs/1810.04805>.
- [15] Sai Surya Duvvuri, Chanakya Ekbote, Rachit Bansal, Rishabh Tiwari, Devvrit Khatri, David Brandfonbrener, Paul Liang, Inderjit Dhillon, and Manzil Zaheer. Interleaved head attention, 2026. URL <https://arxiv.org/abs/2602.21371>.
- [16] Tomas Figliola, Nicholas Alonso, Rishi Iyer, Quentin Anthony, and Beren Millidge. Compressed convolutional attention: Efficient attention in a compressed latent space, 2025. URL <https://arxiv.org/abs/2510.04476>.
- [17] Yaosheng Fu, Guangxuan Xiao, Xin Dong, Song Han, and Oreste Villa. Sparda: Sparse decoupled attention for efficient long-context llm inference, 2026. URL <https://arxiv.org/abs/2606.04511>.
- [18] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. URL <https://arxiv.org/abs/2312.00752>. Version Number: 2.
- [19] Albert Gu, AI21 Labs, et al. Jamba: A hybrid transformer-mamba language model, 2024. URL <https://arxiv.org/abs/2403.19887>.
- [20] Vineet Gupta, Tomer Koren, and Yoram Singer. Shampoo: Preconditioned stochastic tensor optimization. URL <https://arxiv.org/abs/1802.09568>. Version Number: 2.
- [21] Ali Hatamizadeh, Yejin Choi, and Jan Kautz. Gated deltanet-2: Decoupling erase and write in linear attention, 2026. URL <https://arxiv.org/abs/2605.22791>.
- [22] Prannay Khosla, Piotr Teterwak, Chen Wang, Aaron Sarna, Yonglong Tian, Phillip Isola, Aaron Maschinot, Ce Liu, and Dilip Krishnan. Supervised contrastive learning, 2020. URL <https://arxiv.org/abs/2004.11362>.

- [23] Kimi Team, Yu Zhang, Zongyu Lin, Xingcheng Yao, Jiaxi Hu, Fanqing Meng, Chengyin Liu, Xin Men, Songlin Yang, Zhiyuan Li, Wentao Li, Enzhe Lu, Weizhou Liu, Yanru Chen, Weixin Xu, Longhui Yu, Yejie Wang, Yu Fan, Longguang Zhong, Enming Yuan, Dehao Zhang, Yizhi Zhang, T. Y. Liu, Haiming Wang, Shengjun Fang, Weiran He, Shaowei Liu, Yiwei Li, Jianlin Su, Jiezhong Qiu, Bo Pang, Junjie Yan, Zhejun Jiang, Weixiao Huang, Bohong Yin, Jiacheng You, Chu Wei, Zhengtao Wang, Chao Hong, Yutian Chen, Guanduo Chen, Yucheng Wang, Huabin Zheng, Feng Wang, Yibo Liu, Mengnan Dong, Zheng Zhang, Siyuan Pan, Wenhao Wu, Yuhao Wu, Longyu Guan, Jiawen Tao, Guohong Fu, Xinran Xu, Yuzhi Wang, Guokun Lai, Yuxin Wu, Xinyu Zhou, Zhilin Yang, and Yulun Du. Kimi linear: An expressive, efficient attention architecture, 2025. URL <https://arxiv.org/abs/2510.26692>.
- [24] Timon Klein, Jonas Kusch, Sebastian Sager, Stefan Schnake, and Steffen Schotthöfer. Tucker attention: A generalization of approximate attention mechanisms, 2026. URL <https://arxiv.org/abs/2603.30033>.
- [25] Xunhao Lai, Weiqi Xu, Yufeng Yang, Qiaorui Chen, Yang Xu, Lunbin Zeng, Xiaolong Li, Haohai Sun, Haichao Zhu, Vito Zhang, Jinkai Hu, Jiayao Li, Rui Gao, Zekun Li, Songquan Zhu, Jingkai Zhou, and Pengyu Zhao. Minimax sparse attention, 2026. URL <https://arxiv.org/abs/2606.13392>.
- [26] Mike Lewis, Shruti Bhosale, Tim Dettmers, Douwe Kiela, and Luke Zettlemoyer. Base layers: Simplifying training of large, sparse models, 2021. URL <https://arxiv.org/abs/2103.16716>.
- [27] Kaizhao Liang, Lizhang Chen, Bo Liu, and Qiang Liu. Cautious optimizers: Improving training with one line of code. URL <https://arxiv.org/abs/2411.16085>. Version Number: 4.
- [28] Zhixuan Lin, Ke Wang, et al. Forgetting transformer: Softmax attention with a forget gate, 2025. URL <https://arxiv.org/abs/2503.02130>.
- [29] Hong Liu, Zhiyuan Li, David Hall, Percy Liang, and Tengyu Ma. Sophia: A scalable stochastic second-order optimizer for language model pre-training, . URL <https://arxiv.org/abs/2305.14342>. Version Number: 4.
- [30] Liyuan Liu, Haoming Jiang, Pengcheng He, Weizhu Chen, Xiaodong Liu, Jianfeng Gao, and Jiawei Han. On the variance of the adaptive learning rate and beyond, . URL <https://arxiv.org/abs/1908.03265>. Version Number: 4.
- [31] Songtao Liu, Hongwu Peng, Zhiwei Zhang, Zhengyu Chen, and Yue Guo. Multi-head low-rank attention, 2026. URL <https://arxiv.org/abs/2603.02188>.
- [32] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. URL <https://arxiv.org/abs/1711.05101>. Version Number: 3.
- [33] Tianyu Lou, Zheyu Chen, Tao Yu, et al. Efficient sparse attention for long-range transformers, 2024. URL <https://arxiv.org/abs/2406.16747>.
- [34] Sushant Mehta, Raj Dandekar, Rajat Dandekar, and Sreedath Panat. Latent multi-head attention for small language models, 2025. URL <https://arxiv.org/abs/2506.09342>.
- [35] Fanxu Meng. Gqla: Group-query latent attention for hardware-adaptive large language model decoding, 2026. URL <https://arxiv.org/abs/2605.15250>.
- [36] Konstantin Mishchenko and Aaron Defazio. Prodigy: An expeditiously adaptive parameter-free learner. URL <https://arxiv.org/abs/2306.06101>. Version Number: 4.

- [37] Niklas Muennighoff et al. Matryoshka representation learning, 2022. URL <https://arxiv.org/abs/2205.13147>.
- [38] Matteo Pagliardini, Pierre Ablin, and David Grangier. The AdEMAMix optimizer: Better, faster, older. URL <https://arxiv.org/abs/2409.03137>. Version Number: 2.
- [39] Boris T. Polyak. Some methods of speeding up the convergence of iteration methods. *USSR Computational Mathematics and Mathematical Physics*, 4(5):1–17, 1964. doi: 10.1016/0041-5553(64)90137-5.
- [40] Zhen Qin, Xu Han, et al. Hgrn2: Gated linear rnns with state expansion, 2024. URL <https://arxiv.org/abs/2404.07904>.
- [41] Yuxiang Qiu, Qwen Team, et al. Gated attention for large language models, 2025. URL <https://arxiv.org/abs/2505.06708>.
- [42] Jason Ramapuram, Federico Danieli, Eeshan Dhekane, Floris Weers, Dan Busbridge, Pierre Ablin, Tatiana Likhomanenko, Jagrit Digani, Zijin Gu, Amitis Shidani, and Russ Webb. Theory, analysis, and best practices for sigmoid self-attention. URL <https://arxiv.org/abs/2409.04431>. Version Number: 2.
- [43] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks. URL <http://arxiv.org/abs/1908.10084>.
- [44] Hema Hariharan Samson. Lightweight transformer architectures for edge devices in real-time applications. URL <http://arxiv.org/abs/2601.03290>.
- [45] Noam Shazeer and Mitchell Stern. Adafactor: Adaptive learning rates with sublinear memory cost. URL <https://arxiv.org/abs/1804.04235>. Version Number: 1.
- [46] Wei Shen, Ruichuan Huang, Minhui Huang, Cong Shen, and Jiawei Zhang. On the convergence analysis of muon. URL <https://arxiv.org/abs/2505.23737>. Version Number: 1.
- [47] Rafael Soares et al. Griffin: Mixing gated linear recurrences with local attention for efficient sequence modeling, 2024. URL <https://arxiv.org/abs/2402.19427>.
- [48] Luoyang Sun, Cheng Deng, Jiwen Jiang, Xinjian Wu, Haifeng Zhang, Lei Chen, Lionel Ni, and Jun Wang. Gta: Grouped-head latent attention, 2025. URL <https://arxiv.org/abs/2506.17286>.
- [49] Yutao Sun, Li Dong, Shaohan Huang, Shuming Ma, Yuqing Xia, Jilong Xue, Jianyong Wang, and Furu Wei. Retentive network: A successor to transformer for large language models. URL <https://arxiv.org/abs/2307.08621>. Version Number: 4.
- [50] Shohei Taniguchi, Keno Harada, Gouki Minegishi, Yuta Oshima, Seong Cheol Jeong, Go Nagahara, Tomoshi Iiyama, Masahiro Suzuki, Yusuke Iwasawa, and Yutaka Matsuo. ADOPT: Modified adam can converge with any β_2 with the optimal rate. URL <https://arxiv.org/abs/2411.02853>. Version Number: 3.
- [51] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. URL <https://arxiv.org/abs/1706.03762>. Version Number: 7.
- [52] Nikhil Vyas, Depen Morwani, Rosie Zhao, Mujin Kwun, Itai Shapira, David Brandfonbrener, Lucas Janson, and Sham Kakade. SOAP: Improving and stabilizing shampoo using adam. URL <https://arxiv.org/abs/2409.11321>. Version Number: 2.

- [53] Hongyu Wang, Shuming Ma, Li Dong, Shaohan Huang, Huaijie Wang, Lingxiao Ma, Fan Yang, Ruiping Wang, Yi Wu, and Furu Wei. BitNet: Scaling 1-bit transformers for large language models. URL <http://arxiv.org/abs/2310.11453>.
- [54] Zhe Wang, Ming Liu, et al. Fasa: Frequency-aware sparse attention, 2026. URL <https://arxiv.org/abs/2602.03152>.
- [55] Xingyu Xie, Pan Zhou, Huan Li, Zhouchen Lin, and Shuicheng Yan. Adan: Adaptive nesterov momentum algorithm for faster optimizing deep models. URL <https://arxiv.org/abs/2208.06677>. Version Number: 5.
- [56] Songlin Yang, Bailin Wang, et al. Gated linear attention transformers with hardware-efficient training, 2023. URL <https://arxiv.org/abs/2312.06635>.
- [57] Songlin Yang, Bailin Wang, et al. Parallelizing linear transformers with the delta rule over sequence length, 2024. URL <https://arxiv.org/abs/2406.06484>.
- [58] Songlin Yang, Bailin Wang, et al. Gated delta networks: Improving mamba2 with delta rule, 2024. URL <https://arxiv.org/abs/2412.06464>.
- [59] Yang You, Jing Li, Sashank Reddi, Jonathan Hseu, Sanjiv Kumar, Srinadh Bhojanapalli, Xiaodan Song, James Demmel, and Cho-Jui Hsieh. Large batch optimization for deep learning: Training BERT in 76 minutes. URL <https://arxiv.org/abs/1904.00962>. Version Number: 3.
- [60] Han Yuan, DeepSeek-AI, et al. Native sparse attention: Hardware-aligned and natively trainable sparse attention, 2025. URL <https://arxiv.org/abs/2502.11089>.
- [61] Huizhuo Yuan, Yifeng Liu, Shuang Wu, Xun Zhou, and Quanquan Gu. MARS: Unleashing the power of variance reduction for training large models. URL <https://arxiv.org/abs/2411.10438>. Version Number: 4.
- [62] Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Pike Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. Big bird: Transformers for longer sequences. In *Advances in Neural Information Processing Systems*, 2020. doi: 10.48550/ARXIV.2007.14062. URL <https://arxiv.org/abs/2007.14062>.
- [63] Biao Zhang and Rico Sennrich. Root Mean Square Layer Normalization. URL <http://arxiv.org/abs/1910.07467>.
- [64] Jing Zhang, Peng Zhang, Baiwen Kong, Junqiu Wei, and Xin Jiang. Continuous self-attention models with neural ODE networks. 35(16):14393–14401. ISSN 2374-3468, 2159-5399. doi: 10.1609/aaai.v35i16.17692. URL <https://ojs.aaai.org/index.php/AAAI/article/view/17692>.
- [65] Yichi Zhang, Yizhong Wang, et al. Accurate and training-free sparse attention accelerating any model inference, 2025. URL <https://arxiv.org/abs/2502.18137>.
- [66] Jiawei Zhao, Zhenyu Zhang, Beidi Chen, Zhangyang Wang, Anima Anandkumar, and Yandong Tian. GaLore: Memory-efficient LLM training by gradient low-rank projection. URL <https://arxiv.org/abs/2403.03507>. Version Number: 2.
- [67] Hanqing Zhu, Zhenyu Zhang, Wenyan Cong, Xi Liu, Sem Park, Vikas Chandra, Bo Long, David Z. Pan, Zhangyang Wang, and Jinwon Lee. Apollo: Sgd-like memory, adamw-level performance, 2025. URL <https://arxiv.org/abs/2412.05270>.

- [68] Jiachen Zhu, Xinlei Chen, Kaiming He, Yann LeCun, and Zhuang Liu. Transformers without normalization. URL <http://arxiv.org/abs/2503.10622>.
- [69] Lianghui Zhu, Yuxin Fang, Bencheng Liao, Shijie Wang, Tianheng Cheng, Zilong Huang, Chen Chen, Lai Wei, Yutao Zeng, Ya Wang, Yi Lin, Yu Li, and Xinggang Wang. Mixture-of-depths attention, 2026. URL <https://arxiv.org/abs/2603.15619>.

1. Familias de Optimizadores

1.1. Comparación de Memoria y Complejidad

La Tabla 5 resume la sobrecarga de memoria (número de búferes de estado por parámetro), la complejidad computacional por paso y los hiperparámetros clave para todos los optimizadores soportados. La sobrecarga de memoria se expresa en términos del número de tensores de estado mantenidos por parámetro del modelo, donde cada tensor tiene la misma forma que el parámetro.

Optimizador	Búferes de Estado	Costo por Paso	Hiperparámetros Clave
SGD+Momentum	1 (m)	$\mathcal{O}(n)$	lr, momentum, wd
AdamW	2 (m, v)	$\mathcal{O}(n)$	lr, β_1, β_2 , eps, wd
RAAdam	2 (m, v)	$\mathcal{O}(n)$	lr, β_1, β_2 , eps, wd
Adan	3 (m, v, s)	$\mathcal{O}(n)$	lr, $\beta_1, \beta_2, \beta_3$, eps, wd
ADOPT	2 (m, v)	$\mathcal{O}(n)$	lr, β_1, β_2 , eps, wd
AdEMAMix	3 (m_1, m_2, v)	$\mathcal{O}(n)$	lr, $\beta_1, \beta_2, \beta_3$, eps, wd
MARS	3 (m, v, z)	$\mathcal{O}(n)$	lr, β_1, β_2 , eps, wd, γ
Cautious AdamW	2 (m, v)	$\mathcal{O}(n)$	lr, β_1, β_2 , eps, wd
LAMB	2 (m, v)	$\mathcal{O}(n)$	lr, β_1, β_2 , eps, wd
Schedule-Free	3 (z, x, n)	$\mathcal{O}(n)$	lr, β_1, β_2 , wd
Adafactor	1–2 (row/col)	$\mathcal{O}(n)$	lr, β_1, β_2 , eps, wd, factored
GaLore	2 (m, v) + SVD/proj	$\mathcal{O}(nr)$	lr, rank r , β_1, β_2 , eps, wd
Prodigy	3 (m, v, d)	$\mathcal{O}(n)$	β_1, β_2 , eps, wd, d_0
Lion	1 (m)	$\mathcal{O}(n)$	lr, β_1, β_2 , wd
Shampoo	$2d (L_i, R_i)$	$\mathcal{O}(n^{1+2/d})$	lr, eps, wd, matrix eps
SOAP	$2d + 2 (m, v)$	$\mathcal{O}(n^{1+1/d})$	lr, β_1, β_2 , eps, wd, shampoo eps
Sophia	3 (m, v, h)	$\mathcal{O}(n)$	lr, β_1, β_2 , eps, wd, k
Muon	2 (m, v)	$\mathcal{O}(n \cdot k)$	lr, momentum, wd, NS steps k
Turbo-Muon	2 (m, v)	$\mathcal{O}(n \cdot k)$	lr, momentum, wd, NS steps k
APOLLO	2 bajo rango (m_R, v_R) + proy	$\mathcal{O}(nr)$	lr, rank r , update gap, scale, betas
APOLLO-Mini	2 rango-1 (m_R, v_R) + proy	$\mathcal{O}(n)$	lr, update gap, scale, betas, eps, wd
Q-APOLLO	2 bajo rango cuantizados + proy	$\mathcal{O}(nr)$	lr, rank r , update gap, scale, quantization
Anon	3 ($m, v, \text{fixed_lr}$)	$\mathcal{O}(n)$	lr, β_1, β_2 , eps, wd, γ

Cuadro 5: Comparación de memoria y complejidad de todos los optimizadores soportados. n = número de parámetros, d = dimensionalidad del tensor, r = dimensión de rango bajo, k = número de iteraciones de Newton–Schulz. Búferes de estado indica el número de tensores de estado del optimizador mantenidos por grupo de parámetros. El costo por paso se enfoca en el costo adicional dominante más allá del propio cálculo del gradiente.

1.2. La Evolución de la Optimización en Redes Neuronales

El estudio de optimizadores enmarca la optimización de transformadores como una respuesta a tres presiones estructurales: paisajes de pérdida no convexos, heterogeneidad severa de curvatura entre bloques de parámetros y el costo de memoria de almacenar el estado del optimizador para modelos muy grandes. El informe sostiene que el campo se ha diversificado en varias trayectorias: líneas base adaptativas de primer orden, métodos de reducción de varianza, métodos eficientes en memoria, preconditionadores estructurados de segundo orden, métodos sin programación de tasa de aprendizaje y actualizaciones orientadas a ortogonalidad.

1.3. Línea Base Estándar y Optimizadores Adaptativos

SGD con Momentum. La actualización clásica acumula un búfer de momentum y luego aplica una tasa de aprendizaje fija. En cada iteración t , el algoritmo mantiene un promedio móvil exponencial m_t de los gradientes pasados:

$$m_t = \beta m_{t-1} + g_t \quad (1)$$

$$\theta_{t+1} = \theta_t - \eta m_t \quad (2)$$

donde $g_t = \nabla f(\theta_t)$, $\beta \in [0, 0.99]$ controla la decadencia del momentum y η es la tasa de aprendizaje fija. El término de momentum actúa como velocidad: acelera el movimiento en direcciones consistentes mientras amortigua oscilaciones en direcciones variables. Sus fortalezas son la baja sobrecarga de memoria (un único búfer de momentum por parámetro) y una fuerte generalización cuando se ajusta cuidadosamente. Su principal debilidad en cargas de trabajo de transformadores es la escasa robustez frente a la heterogeneidad de curvatura (diferentes espectros del Hessiano entre grupos de parámetros) y una fuerte dependencia de las programaciones de tasa de aprendizaje.

Algorithm 1 SGD con Momentum

Require: Parámetros iniciales θ_0 , tasa de aprendizaje η , coeficiente de momentum β , decadencia de peso λ

Ensure: Parámetros actualizados θ_T

- 1: Inicializar: búfer de momentum $m \leftarrow 0$
 - 2: **for** $t = 0, 1, 2, \dots, T - 1$ **do**
 - 3: Calcular gradiente: $g_t \leftarrow \nabla f(\theta_t)$
 - 4: **if** $\text{decadencia_de_peso} > 0$ **then**
 - 5: $g_t \leftarrow g_t + \lambda \theta_t$ ▷ Regularización L2
 - 6: **end if**
 - 7: Actualizar momentum: $m \leftarrow \beta \cdot m + g_t$
 - 8: Actualizar parámetros: $\theta_{t+1} \leftarrow \theta_t - \eta \cdot m$
 - 9: **end for** **return** θ_T
-

Adam y AdamW. Adam rastrea promedios móviles exponenciales tanto del primer momento (media) como del segundo momento (varianza no centrada) para lograr tasas de aprendizaje adaptativas elemento a elemento. La regla de actualización es:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (3)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (4)$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (\text{corrección de sesgo}) \quad (5)$$

$$\theta_{t+1} = \theta_t - \eta \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \right) \quad (6)$$

AdamW introduce una modificación crucial: la decadencia de peso se aplica directamente a los parámetros (desacoplada de los gradientes): $\theta_t \leftarrow \theta_t(1 - \eta\lambda)$ en lugar de añadir $\lambda\theta_t$ al gradiente. Este desacoplamiento evita que el escalado adaptativo interfiera con la fuerza de la regularización. El informe trata a AdamW como la línea base práctica para el ajuste fino de transformadores porque converge rápidamente y es relativamente tolerante a la variación de hiperparámetros. La desventaja es el costo de memoria: ambos tensores de momento deben almacenarse para cada parámetro, duplicando la memoria del estado del optimizador en relación con SGD.

Algorithm 2 AdamW (Adam con Decadencia de Peso Desacoplada)

Require: Parámetros iniciales θ_0 , tasa de aprendizaje η , tasas de decadencia exponencial $\beta_1, \beta_2 \in [0, 1)$

Require: Constante de momentum ϵ , decadencia de peso λ

Ensure: Parámetros actualizados θ_T

- 1: Inicializar: primer momento $m \leftarrow 0$, segundo momento $v \leftarrow 0$, contador de pasos $t \leftarrow 0$
 - 2: **for** $t = 1, 2, \dots, T$ **do**
 - 3: Calcular gradiente: $g_t \leftarrow \nabla f(\theta_{t-1})$
 - 4: Decadencia de peso (desacoplada): $\theta_{t-1} \leftarrow \theta_{t-1}(1 - \eta\lambda)$
 - 5: Actualizar momentos: $m \leftarrow \beta_1 m + (1 - \beta_1)g_t$
 - 6: $v \leftarrow \beta_2 v + (1 - \beta_2)g_t^2$
 - 7: Corrección de sesgo: $\hat{m} \leftarrow m/(1 - \beta_1^t)$, $\hat{v} \leftarrow v/(1 - \beta_2^t)$
 - 8: Actualizar parámetros: $\theta_t \leftarrow \theta_{t-1} - \eta(\hat{m}/(\sqrt{\hat{v}} + \epsilon))$
 - 9: **end for** **return** θ_T
-

RAdam (Adam Rectificado). RAdam aborda la inestabilidad de Adam en el entrenamiento temprano rectificando dinámicamente la tasa de aprendizaje adaptativa. La observación clave es que el segundo momento v_t tiene una varianza muy alta en los primeros pasos, causando un escalado adaptativo poco fiable. RAdam calcula la longitud efectiva de la ventana de promedio móvil simple (SMA):

$$\rho_t = \rho_\infty - \frac{2t\beta_2^t}{1 - \beta_2^t}, \quad \text{donde} \quad \rho_\infty = \frac{2}{1 - \beta_2} - 1$$

Cuando $\rho_t > 4$ (suficientes muestras para la estimación de varianza), RAdam aplica el escalado adaptativo con un término de rectificación:

$$r_t = \sqrt{\frac{(\rho_t - 4)(\rho_t - 2)\rho_\infty}{(\rho_\infty - 4)(\rho_\infty - 2)\rho_t}}, \quad \theta_{t+1} = \theta_t - \eta r_t \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

Cuando $\rho_t \leq 4$, RAdam retrocede a SGD con momentum: $\theta_{t+1} = \theta_t - \eta \hat{m}_t$. Esta transición gradual elimina la necesidad de programaciones manuales de calentamiento de la tasa de aprendizaje y mejora la robustez.

Algorithm 3 RAdam (Adam Rectificado)

Require: Parámetros iniciales θ_0 , tasa de aprendizaje η , β_1, β_2 , decadencia de peso λ

Ensure: Parámetros actualizados θ_T

```
1: Inicializar:  $m \leftarrow 0, v \leftarrow 0, t \leftarrow 0$ 
2: Calcular:  $\rho_\infty \leftarrow 2/(1 - \beta_2) - 1$ 
3: for  $t = 1, 2, \dots, T$  do
4:    $g_t \leftarrow \nabla f(\theta_{t-1})$ 
5:   if decadencia_de_peso  $> 0$  then
6:      $\theta_{t-1} \leftarrow \theta_{t-1}(1 - \eta\lambda)$ 
7:   end if
8:    $m \leftarrow \beta_1 m + (1 - \beta_1)g_t$ 
9:    $v \leftarrow \beta_2 v + (1 - \beta_2)g_t^2$ 
10:   $\hat{m} \leftarrow m/(1 - \beta_1^t), \hat{v} \leftarrow v/(1 - \beta_2^t)$ 
11:   $\rho_t \leftarrow \rho_\infty - 2t\beta_2^t/(1 - \beta_2^t)$ 
12:  if  $\rho_t > 4$  then
13:     $r_t \leftarrow \sqrt{((\rho_t - 4)(\rho_t - 2)\rho_\infty)/((\rho_\infty - 4)(\rho_\infty - 2)\rho_t)}$ 
14:     $\theta_t \leftarrow \theta_{t-1} - \eta r_t \hat{m}/(\sqrt{\hat{v}} + \epsilon)$ 
15:  else
16:     $\theta_t \leftarrow \theta_{t-1} - \eta \hat{m}$  ▷ SGD con momentum
17:  end if
18: end for return  $\theta_T$ 
```

1.4. Momentum Avanzado y Reducción de Varianza (2024–2025)

Adan (Momentum Nesterov Adaptativo). Adan reformula la aceleración de Nesterov sin el cálculo de gradiente adicional requerido por el SGD Nesterov clásico. El algoritmo mantiene tres búferes de momentum:

$$m_t = (1 - \beta_1)m_{t-1} + \beta_1 g_t \quad (\text{primer momento}) \quad (7)$$

$$v_t = (1 - \beta_2)v_{t-1} + \beta_2(g_t - g_{t-1}) \quad (\text{velocidad/diferencia de gradientes}) \quad (8)$$

$$n_t = (1 - \beta_3)n_{t-1} + \beta_3[g_t + (1 - \beta_1)(g_t - g_{t-1})]^2 \quad (\text{segundo momento Nesterov}) \quad (9)$$

El término de **Estimación de Momentum Nesterov** (NME) $\bar{g}_t = g_t + (1 - \beta_1)(g_t - g_{t-1})$ estima el gradiente en una posición futura sin evaluarlo. La actualización combina el momentum con la velocidad:

$$\bar{m}_t = m_t + (1 - \beta_1)v_t, \quad \theta_{t+1} = \theta_t - \eta \frac{\bar{m}_t}{\sqrt{n_t} + \epsilon}$$

Adan logra una convergencia rápida en diversas arquitecturas (CNN, GAN, Transformadores) mediante esta aceleración. El costo es mantener tres búferes similares al momentum, aumentando la sobrecarga de memoria en relación con Adam.

Algorithm 4 Adan (Momentum Nesterov Adaptativo)

Require: Parámetros iniciales θ_0 , tasa de aprendizaje η , $\beta_1, \beta_2, \beta_3 \in (0, 1)$

Ensure: Parámetros actualizados θ_T

```
1: Inicializar:  $m \leftarrow 0$ ,  $v \leftarrow 0$ ,  $n \leftarrow 0$ ,  $g_{-1} \leftarrow 0$  (gradiente anterior)
2: for  $t = 1, 2, \dots, T$  do
3:    $g_t \leftarrow \nabla f(\theta_{t-1})$ 
4:    $m \leftarrow (1 - \beta_1)m + \beta_1 g_t$ 
5:    $\Delta g_t \leftarrow g_t - g_{t-1}$ 
6:    $v \leftarrow (1 - \beta_2)v + \beta_2 \Delta g_t$ 
7:   Estimación Nesterov:  $\bar{g}_t \leftarrow g_t + (1 - \beta_1)\Delta g_t$ 
8:    $n \leftarrow (1 - \beta_3)n + \beta_3 \bar{g}_t^2$ 
9:   Momentum combinado:  $\bar{m} \leftarrow m + (1 - \beta_1)v$ 
10:   $\theta_t \leftarrow \theta_{t-1} - \eta(\bar{m}/(\sqrt{n} + \epsilon))$ 
11:   $g_{t-1} \leftarrow g_t$ 
12: end for return  $\theta_T$ 
```

ADOPT (Adam con Ajuste Óptimo Podado). ADOPT corrige un problema teórico fundamental en Adam: el gradiente aparece tanto en la estimación del primer momento como en la del segundo momento, creando circularidad. ADOPT **desacopla** utilizando el segundo momento del paso anterior para el denominador:

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (10)$$

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \frac{g_t}{\sqrt{v_{t-1}} + \epsilon} \quad (\text{usa } v_{t-1}, \text{ no } v_t) \quad (11)$$

$$\theta_{t+1} = \theta_t - \eta m_t \quad (12)$$

Este simple reordenamiento logra la tasa de convergencia óptima $O(1/\sqrt{T})$ con **cualquier** elección de $\beta_2 \in (0, 1)$, sin suposiciones de ruido acotado. La consecuencia práctica es que ADOPT es un reemplazo directo de Adam con garantías teóricas más sólidas y un rendimiento empírico comparable o superior en los dominios de visión, PLN, RL y modelado generativo.

Algorithm 5 ADOPT (Adam con Ajuste Óptimo Podado)

Require: Parámetros iniciales θ_0 , tasa de aprendizaje η , β_1, β_2 , tolerancia ϵ

Ensure: Parámetros actualizados θ_T

```
1: Inicializar:  $m \leftarrow 0$ ,  $v \leftarrow 0$ 
2: for  $t = 1, 2, \dots, T$  do
3:    $g_t \leftarrow \nabla f(\theta_{t-1})$ 
4:   if  $\text{decadencia\_de\_peso} > 0$  then
5:      $\theta_{t-1} \leftarrow \theta_{t-1}(1 - \eta\lambda)$ 
6:   end if
7:   Usar segundo momento anterior:  $\text{denom} \leftarrow \sqrt{\max(v, \epsilon)} + \epsilon$ 
8:   Actualizar primer momento usando varianza anterior:  $m \leftarrow \beta_1 m + (1 - \beta_1)(g_t/\text{denom})$ 
9:   Actualizar segundo momento con gradiente actual:  $v \leftarrow \beta_2 v + (1 - \beta_2)g_t^2$ 
10:   $\theta_t \leftarrow \theta_{t-1} - \eta m$ 
11: end for return  $\theta_T$ 
```

AdEMAMix (Mezcla de Promedios Móviles Exponenciales). AdEMAMix reemplaza el EMA único de gradientes de Adam por una **mezcla de dos EMA**: uno de decadencia rápida y otro de decadencia lenta. Esto aborda la observación de que los gradientes siguen siendo

informativos durante decenas de miles de pasos, no solo cientos:

$$m_{1,t} = \beta_1 m_{1,t-1} + (1 - \beta_1) g_t \quad (\text{EMA rápido, } \beta_1 \approx 0,9) \quad (13)$$

$$m_{2,t} = \beta_3 m_{2,t-1} + (1 - \beta_3) g_t \quad (\text{EMA lento, } \beta_3 \approx 0,9999) \quad (14)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (15)$$

$$m_t = m_{1,t} + \alpha_t m_{2,t} \quad (\text{mezcla con peso programado } \alpha_t) \quad (16)$$

$$\theta_{t+1} = \theta_t - \eta \frac{m_t}{\sqrt{v_t} + \epsilon} \quad (17)$$

El peso de la mezcla α_t típicamente aumenta durante el entrenamiento, permitiendo que el EMA rápido proporcione adaptación inmediata mientras que el EMA lento acumula correlaciones de gradiente a largo plazo. Empíricamente, un modelo de 1.3B parámetros en 101B tokens alcanza una pérdida similar a AdamW en 197B tokens (ganancia del 95 % en eficiencia de datos), lo que sugiere que el EMA lento reduce significativamente el olvido del modelo.

Algorithm 6 AdEMAMix (Mezcla de Promedios Móviles Exponenciales)

Require: Parámetros iniciales θ_0 , tasa de aprendizaje η , β_1 (rápido), β_3 (lento), β_2 (segundo momento)

Require: Peso de mezcla α_t (típicamente creciente), tolerancia ϵ

Ensure: Parámetros actualizados θ_T

```

1: Inicializar:  $m_1 \leftarrow 0$  (EMA rápido),  $m_2 \leftarrow 0$  (EMA lento),  $v \leftarrow 0$ 
2: for  $t = 1, 2, \dots, T$  do
3:    $g_t \leftarrow \nabla f(\theta_{t-1})$ 
4:   if  $\text{decadencia\_de\_peso} > 0$  then
5:      $\theta_{t-1} \leftarrow \theta_{t-1}(1 - \eta\lambda)$ 
6:   end if
7:    $m_1 \leftarrow \beta_1 m_1 + (1 - \beta_1) g_t$  ▷ EMA rápido
8:    $m_2 \leftarrow \beta_3 m_2 + (1 - \beta_3) g_t$  ▷ EMA lento
9:    $v \leftarrow \beta_2 v + (1 - \beta_2) g_t^2$ 
10:   $m_t \leftarrow m_1 + \alpha_t m_2$  ▷ Mezcla
11:   $\theta_t \leftarrow \theta_{t-1} - \eta(m_t / (\sqrt{v} + \epsilon))$ 
12: end for return  $\theta_T$ 

```

MARS (Haciendo Brillar las Tasas de Aprendizaje Adaptativas). MARS combina métodos de gradiente preconditionado (ej., AdamW) con **reducción de varianza** mediante momentum recursivo estocástico escalado. La innovación central es una estimación del gradiente con varianza reducida:

$$c_t = g_t + \gamma(c_{t-1} - g_{t-1}) \quad (\text{estimación recursiva tipo SVRG}) \quad (18)$$

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) c_t \quad (19)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) c_t^2 \quad (20)$$

$$\theta_{t+1} = \theta_t - \eta \frac{m_t}{\sqrt{v_t} + \epsilon} \quad (21)$$

donde $\gamma \in [0, 0.1, 0.1]$ controla cuánta información de gradiente histórico se retiene. El gradiente con varianza reducida c_t actúa como un filtro de ruido implícito, amplificando las direcciones de señal consistentes y amortiguando el ruido contradictorio. MARS-AdamW supera consistentemente a AdamW por márgenes significativos en el preentrenamiento de GPT-2, lo que sugiere que la reducción de varianza mejora sustancialmente la convergencia en el entrenamiento estocástico por mini-lotes.

Algorithm 7 MARS (Haciendo Brillar las Tasas de Aprendizaje Adaptativas)

Require: Parámetros iniciales θ_0 , tasa de aprendizaje η , β_1, β_2 , decadencia de peso λ

Require: Coeficiente de reducción de varianza $\gamma \in [0, 0.1, 0, 1]$

Ensure: Parámetros actualizados θ_T

```
1: Inicializar:  $m \leftarrow 0$ ,  $v \leftarrow 0$ ,  $c \leftarrow 0$  (gradiente con varianza reducida),  $g_{-1} \leftarrow 0$ 
2: for  $t = 1, 2, \dots, T$  do
3:    $g_t \leftarrow \nabla f(\theta_{t-1})$ 
4:   Reducción de varianza:  $c \leftarrow g_t + \gamma(c - g_{t-1})$ 
5:    $m \leftarrow \beta_1 m + (1 - \beta_1)c$ 
6:    $v \leftarrow \beta_2 v + (1 - \beta_2)c^2$ 
7:   if  $\text{decadencia\_de\_peso} > 0$  then
8:      $\theta_{t-1} \leftarrow \theta_{t-1}(1 - \eta\lambda)$ 
9:   end if
10:   $\theta_t \leftarrow \theta_{t-1} - \eta(m/(\sqrt{v} + \epsilon))$ 
11:   $g_{t-1} \leftarrow g_t$ 
12: end for return  $\theta_T$ 
```

Optimizadores Cautelosos (Cautious AdamW, Cautious Lion). El marco Cauteloso aplica una **modificación de una línea** a cualquier optimizador basado en momentum: enmascarar la actualización de modo que solo se apliquen las dimensiones donde el momentum y la dirección del gradiente coinciden:

$$\text{mask}_i = \begin{cases} 1 & \text{si } m_t[i] \cdot g_t[i] > 0 \quad (\text{acuerdo}) \\ 0 & \text{en caso contrario} \end{cases} \quad (22)$$

$$u_t = \left(\frac{m_t}{\sqrt{v_t} + \epsilon} \right) \odot \text{mask} \quad (\text{enmascaramiento elemento a elemento}) \quad (23)$$

$$\theta_{t+1} = \theta_t - \eta u_t \quad (24)$$

La intuición: el momentum m_t estima la dirección del gradiente a partir del historial; el gradiente actual g_t es la señal instantánea. Cuando ambos coinciden, el optimizador está seguro y debe actualizar agresivamente. Cuando discrepan, el optimizador está en conflicto—la tendencia histórica apunta hacia una región que pudo haber sido buena antes, pero la evidencia actual la contradice. Al enmascarar las dimensiones en conflicto, Cauteloso se vuelve más conservador y evita pasos corruptos. Empíricamente, este simple enmascaramiento logra hasta **1.47× de aceleración** en el preentrenamiento de Llama y MAE mientras preserva las garantías de convergencia.

Algorithm 8 Cautious AdamW (Enmascaramiento de Actualización por Consenso)

Require: Parámetros iniciales θ_0 , tasa de aprendizaje η , β_1, β_2 , decadencia de peso λ

Ensure: Parámetros actualizados θ_T

```
1: Inicializar:  $m \leftarrow 0, v \leftarrow 0$ 
2: for  $t = 1, 2, \dots, T$  do
3:    $g_t \leftarrow \nabla f(\theta_{t-1})$ 
4:    $m \leftarrow \beta_1 m + (1 - \beta_1)g_t$ 
5:    $v \leftarrow \beta_2 v + (1 - \beta_2)g_t^2$ 
6:   Máscara de consenso:  $\text{mask}_i \leftarrow 1$  si  $m[i] \cdot g_t[i] > 0$ , si no 0 ▷ Acuerdo
7:   Actualización base Adam:  $u \leftarrow m/(\sqrt{v} + \epsilon)$ 
8:   Aplicar máscara:  $u_{\text{masked}} \leftarrow u \odot \text{mask}$  ▷ Elemento a elemento
9:   if  $\text{decadencia\_de\_peso} > 0$  then
10:     $\theta_{t-1} \leftarrow \theta_{t-1}(1 - \eta\lambda)$ 
11:   end if
12:    $\theta_t \leftarrow \theta_{t-1} - \eta u_{\text{masked}}$ 
13: end for return  $\theta_T$ 
```

1.5. Optimizadores de Lote Grande, Eficientes en Memoria y Sin Tasa de Aprendizaje

LAMB (Layer-wise Adaptive Moments optimizer for Batch training). LAMB extiende Adam con **escalado de tasa adaptativo por capas** (inspirado en LARS), permitiendo entrenamiento estable con tamaños de lote extremos (e.g., 64K en BERT):

$$m_t^L = \beta_1 m_{t-1}^L + (1 - \beta_1)g_t^L \quad (\text{momento de primer orden por capas}) \quad (25)$$

$$v_t^L = \beta_2 v_{t-1}^L + (1 - \beta_2)(g_t^L)^2 \quad (26)$$

$$u_{\text{adam}}^L = \frac{m_t^L}{\sqrt{v_t^L} + \epsilon} + \lambda \theta_{t-1}^L \quad (\text{paso adaptativo base con decaimiento}) \quad (27)$$

$$\phi^L = \frac{\|\theta_{t-1}^L\|_2}{\|u_{\text{adam}}^L\|_2} \quad (\text{razón de confianza: normalización por capas}) \quad (28)$$

$$\theta_t^L = \theta_{t-1}^L - \eta \cdot \phi^L \cdot u_{\text{adam}}^L \quad (29)$$

La **razón de confianza** $\phi^L = \|\theta_{t-1}^L\|_2 / \|u_{\text{adam}}^L\|_2$ normaliza el paso de actualización efectivo en relación con la magnitud de los pesos. En lotes muy grandes, el ruido estocástico del gradiente puede superar a la señal, desestabilizando las tasas de aprendizaje por capas. Al escalar las actualizaciones proporcionalmente a las normas de los pesos, LAMB asegura cambios relativos en lugar de absolutos, evitando que actualizaciones pequeñas y seguras en vectores grandes dominen. LAMB preserva los beneficios de generalización de lotes pequeños mientras permite entrenamiento eficiente con lotes grandes.

Algorithm 9 LAMB (Layer-wise Adaptive Moments optimizer for Batch training)

Require: Parámetros iniciales θ_0 , tasa de aprendizaje η , β_1, β_2 , decaimiento de pesos λ

Ensure: Parámetros actualizados θ_T

```
1: Inicializar: Para cada capa  $L$ :  $m^L \leftarrow 0$ ,  $v^L \leftarrow 0$ 
2: for  $t = 1, 2, \dots, T$  do
3:   for cada capa  $L$  do
4:      $g_t^L \leftarrow \nabla f(\theta_t^L)$ 
5:      $m^L \leftarrow \beta_1 m^L + (1 - \beta_1) g_t^L$ 
6:      $v^L \leftarrow \beta_2 v^L + (1 - \beta_2) (g_t^L)^2$ 
7:     Adam base:  $u_{\text{adam}}^L \leftarrow m^L / (\sqrt{v^L} + \epsilon)$ 
8:     if weight_decay > 0 then
9:        $u_{\text{adam}}^L \leftarrow u_{\text{adam}}^L + \lambda \theta_{t-1}^L$ 
10:    end if
11:    Razón de confianza:  $\phi^L \leftarrow \|\theta_{t-1}^L\|_2 / \|u_{\text{adam}}^L\|_2$  si ambos son no nulos, sino 1
12:     $\theta_t^L \leftarrow \theta_{t-1}^L - \eta \phi^L u_{\text{adam}}^L$ 
13:  end for
14: end for return  $\theta_T$ 
```

Schedule-Free AdamW. Los métodos Schedule-Free eliminan el diseño explícito de programación de la receta de optimización. El algoritmo mantiene dos flujos de parámetros: z_t (exploración) y x_t (promedio suavizado):

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (\text{varianza}) \quad (30)$$

$$z_t = z_{t-1} (1 - \eta\lambda) - \eta \frac{g_t}{\sqrt{v_t} + \epsilon} \quad (\text{paso adaptativo con decaimiento}) \quad (31)$$

$$c_t = \frac{1}{t+1} \quad (\text{coeficiente de promediado, decrece con el tiempo}) \quad (32)$$

$$x_t = (1 - c_t) x_{t-1} + c_t z_t \quad (\text{promediado de iteraciones}) \quad (33)$$

$$y_t = (1 - \beta) z_t + \beta x_t \quad (\text{interpolación para el punto de evaluación}) \quad (34)$$

El coeficiente de promediado $c_t = 1/(t+1)$ implementa una programación de tasa de aprendizaje implícita sin especificar explícitamente el número total de pasos T . Este marco unifica la programación y el promediado de iteraciones: el algoritmo explora mediante z_t mientras acumula dirección estable mediante x_t . El punto de evaluación y_t (usado para el cómputo del gradiente) interpola entre exploración y estabilidad. Schedule-Free logra convergencia de última generación en optimización convexa, aprendizaje profundo a gran escala y aprendizaje por refuerzo, eliminando un hiperparámetro importante.

Algorithm 10 Schedule-Free AdamW

Require: Parámetros iniciales θ_0 , tasa de aprendizaje η (fija), β_1, β_2 , decaimiento de pesos λ

Ensure: Parámetros actualizados θ_T o promediado x_T

```
1: Inicializar:  $z \leftarrow \theta_0$  (exploración),  $x \leftarrow \theta_0$  (promedio),  $v \leftarrow 0$ ,  $t \leftarrow 0$ 
2: for  $t = 1, 2, \dots, T$  do
3:    $g_t \leftarrow \nabla f(y_{t-1})$  (gradiente en el punto de interpolación)
4:    $v \leftarrow \beta_2 v + (1 - \beta_2) g_t^2$  ▷ Varianza
5:   Decaimiento de pesos sobre  $z$ :  $z \leftarrow z(1 - \eta\lambda)$ 
6:    $z \leftarrow z - \eta g_t / (\sqrt{v} + \epsilon)$  ▷ Paso adaptativo sobre el gradiente crudo
7:    $c_t \leftarrow 1/(t+1)$  ▷ Coeficiente de promediado
8:    $x \leftarrow (1 - c_t) x + c_t z$  ▷ Promediado de iteraciones
9:    $y_t \leftarrow (1 - \beta_1) z + \beta_1 x$  ▷ Interpolación para la siguiente evaluación
10: end for return  $x_T$  o  $y_T$ 
```

Adafactor. Adafactor reduce la memoria del optimizador **factorizando** estadísticas de segundo momento para parámetros con forma de matriz, almacenando solo acumuladores de filas y columnas en lugar de estados de varianza densos. Para una matriz de gradiente $G_t \in \mathbb{R}^{m \times n}$:

$$R_t = \beta_2 R_{t-1} + (1 - \beta_2)(G_t^2) \mathbf{1}_n^T \quad (\text{varianza por filas}) \quad (35)$$

$$C_t = \beta_2 C_{t-1} + (1 - \beta_2) \mathbf{1}_m^T (G_t^2) \quad (\text{varianza por columnas}) \quad (36)$$

$$\hat{V}_t = \frac{R_t C_t}{\mathbf{1}_n^T R_t} \quad (\text{varianza reconstruida mediante producto exterior}) \quad (37)$$

$$U_t = \frac{G_t}{\sqrt{\hat{V}_t} + \epsilon} \quad (\text{paso adaptativo normalizado}) \quad (38)$$

$$\hat{U}_t = \frac{U_t}{\max(1, \text{RMS}(U_t))} \quad (\text{recorte de estabilidad}) \quad (39)$$

En lugar de almacenar $m \times n$ valores de segundo momento, Adafactor almacena solo $m + n$ acumuladores, reduciendo la memoria de $O(n_{\text{params}})$ a $O(\sqrt{n_{\text{params}}})$. Esto es más atractivo cuando la VRAM está dominada por el estado del optimizador en lugar de las activaciones. La contrapartida es una expresividad de optimización reducida y posible inestabilidad en algunas tareas.

Algorithm 11 Adafactor (Estadísticas de Segundo Momento Factorizadas)

Require: Matriz de gradiente $G_t \in \mathbb{R}^{m \times n}$, tasa de aprendizaje η , $\beta_2 \approx 0,999$

Ensure: Parámetros actualizados

- 1: Inicializar: Acumuladores de fila $R \leftarrow \epsilon \mathbf{1}_m^T$, Columna $C \leftarrow \epsilon \mathbf{1}_n$
 - 2: **for** $t = 1, 2, \dots, T$ **do**
 - 3: $G_t \leftarrow \nabla f(\theta_t)$ (gradiente matricial)
 - 4: $R \leftarrow \beta_2 R + (1 - \beta_2)(G_t^2) \mathbf{1}_n^T$ ▷ Productos internos por filas
 - 5: $C \leftarrow \beta_2 C + (1 - \beta_2) \mathbf{1}_m^T (G_t^2)$ ▷ Productos internos por columnas
 - 6: Varianza reconstruida: $\hat{V} \leftarrow (R \cdot C) / (\mathbf{1}_n^T R)$ ▷ Mediante producto exterior
 - 7: Paso adaptativo normalizado: $U_t \leftarrow G_t / (\sqrt{\hat{V}} + \epsilon)$
 - 8: Recorte RMS por fila: $\hat{U}_t \leftarrow U_t / \max(1, \text{RMS}(U_t))$
 - 9: $\theta_{t+1} \leftarrow \theta_t - \eta \hat{U}_t$
 - 10: **end for**
-

GaLore (Gradient Low-Rank Projection). GaLore proyecta gradientes 2D en un subespacio de bajo rango antes de la optimización, reduciendo la memoria del estado del optimizador mientras mantiene el escalado adaptativo del paso. Para una matriz de parámetros 2D $W \in \mathbb{R}^{m \times n}$:

$$U, S, V = \text{SVD}(G_t) \quad (\text{calcular descomposición singular}) \quad (40)$$

$$P \in \mathbb{R}^{n \times r} \quad \text{o} \quad P^T \in \mathbb{R}^{r \times m} \quad (\text{seleccionar los } r \text{ vectores singulares superiores}) \quad (41)$$

$$G_{\text{low}} = P^T G_t \quad (\text{proyectar al espacio de bajo rango}) \quad (42)$$

$$\Delta_{\text{low}} = \text{Adam}(G_{\text{low}}) \quad (\text{optimizar en el espacio comprimido}) \quad (43)$$

$$\Delta = P \Delta_{\text{low}} \quad (\text{reconstruir en el espacio original}) \quad (44)$$

Al proyectar en un subespacio de rango r (típicamente $r \ll \min(m, n)$), el estado del optimizador se reduce de $O(mn)$ a $O(r(m+n))$ para parámetros 2D. Este enfoque complementario de ahorro de memoria es especialmente relevante cuando el modelo es demasiado grande para el estado completo del optimizador. GaLore funciona sinérgicamente con otras técnicas y ha mostrado resultados empíricos sólidos en modelos de miles de millones de parámetros.

Algorithm 12 GaLore (Gradient Low-Rank Projection)

Require: Matriz de parámetros 2D $W \in \mathbb{R}^{m \times n}$, tasa de aprendizaje η , rango $r \ll \min(m, n)$

Ensure: Parámetros actualizados

```
1: Inicializar: Estado de Adam en el espacio de bajo rango (si es variante de rango 2)
2: for  $t = 1, 2, \dots, T$  do
3:    $G_t \leftarrow \nabla f(W_t)$ 
4:   if  $t \bmod K = 1$  then ▷ SVD periódico
5:      $U_t, S_t, V_t^\top \leftarrow \text{SVD}(G_t)$  (completa o delgada)
6:     Seleccionar proyección:  $P \leftarrow U_t[:, :r]$  o  $P \leftarrow V_t[:, :r]$ 
7:   end if
8:   Proyectar gradiente:  $G_{\text{low}} \leftarrow P^\top G_t$  (o  $G_t P^\top$  para proyección derecha)
9:   Ejecutar Adam en bajo rango:  $\Delta_{\text{low}} \leftarrow \text{Adam}(G_{\text{low}})$ 
10:  Reconstruir en espacio original:  $\Delta \leftarrow P \Delta_{\text{low}}$  (o  $\Delta_{\text{low}} P^\top$ )
11:   $W_t \leftarrow W_t - \eta \Delta$ 
12: end for
```

APOLLO, APOLLO-Mini, y Q-APOLLO. La familia APOLLO [67] parte de la observación de que el denominador elemento a elemento de AdamW puede **transformarse en una actualización estructurada de la tasa de aprendizaje**. En lugar de almacenar momentos densos para cada entrada de parámetro, APOLLO proyecta un gradiente matricial $G_t \in \mathbb{R}^{m \times n}$ en un subespacio aleatorio compacto y rastrea momentos estilo Adam allí:

$$R_t = P_t G_t \quad \text{o} \quad R_t = G_t P_t^\top \quad (45)$$

$$M_t^R = \beta_1 M_{t-1}^R + (1 - \beta_1) R_t \quad (46)$$

$$V_t^R = \beta_2 V_{t-1}^R + (1 - \beta_2) R_t^2 \quad (47)$$

$$\tilde{R}_t = \frac{M_t^R}{\sqrt{V_t^R} + \epsilon} \quad (48)$$

El estado proyectado *no* se expande de vuelta a una actualización densa de bajo rango como en los métodos basados en SVD. En su lugar, APOLLO estima un tensor de escalado estructurado S_t en el espacio original. En la variante estándar de APOLLO, ese escalado es **por canales**: cada fila o canal recibe su propia razón de norma. En APOLLO-Mini, el escalado se reduce a un único escalar **por tensor**, correspondiente al caso extremo de rango 1 descrito en el artículo. La actualización de parámetros resultante es, por tanto, similar a Adam en adaptación pero mucho más cercana a SGD en costo de estado:

$$W_t \leftarrow (1 - \eta\lambda)W_{t-1} - \eta\alpha(G_t \odot S_t)$$

donde α es el factor de escala adicional usado para estabilizar variantes altamente comprimidas.

La contribución práctica del artículo es doble. Primero, APOLLO reemplaza el costoso SVD repetido con **proyección aleatoria gaussiana** actualizada periódicamente, de modo que la carga computacional es multiplicación de matrices ordinaria en lugar de descomposición espectral. Segundo, el optimizador es inusualmente tolerante a la compresión extrema: incluso **APOLLO-Mini**, que mantiene solo estado auxiliar de rango 1, sigue siendo competitivo o mejor que AdamW en los experimentos de preentrenamiento reportados, mientras se acerca al costo de memoria de SGD.

Algorithm 13 APOLLO / APOLLO-Mini Escalado de Gradiente Estructurado

Require: Matriz de pesos $W \in \mathbb{R}^{m \times n}$ con $m \leq n$, tasa de aprendizaje η , factor de escala α , tasas de decaimiento (β_1, β_2) , decaimiento de pesos λ , rango r , intervalo de actualización de proyección T

Ensure: Parámetros actualizados W_T

```
1: Inicializar momentos proyectados:  $M^R \leftarrow 0, V^R \leftarrow 0$ , paso  $t \leftarrow 0$ 
2: repeat
3:   Calcular gradiente:  $G_t \leftarrow \nabla_W \phi(W_t)$ 
4:   if  $t \bmod T = 0$  then
5:     Muestrear proyector gaussiano  $P_t \sim \mathcal{N}(0, 1/r)$  con una semilla nueva
6:   end if
7:   Proyectar gradiente:  $R_t \leftarrow P_t G_t$   $\triangleright$  o  $G_t P_t^\top$  según la disposición
8:   Actualizar momentos proyectados AdamW:  $M_t^R, V_t^R \leftarrow \text{AdamWState}(R_t; \beta_1, \beta_2)$ 
9:   Normalizar estado proyectado:  $\tilde{R}_t \leftarrow M_t^R / (\sqrt{V_t^R} + \epsilon)$ 
10:  if APOLLO then
11:     $S_t \leftarrow \text{diag}(s_1^R, \dots, s_m^R)$ , donde  $s_i^R = \|\tilde{R}_t[i, :]\|_2 / \|R_t[i, :]\|_2$ 
12:  else
13:     $S_t \leftarrow s_t^R$ , donde  $s_t^R = \|\tilde{R}_t\|_2 / \|R_t\|_2$ 
14:  end if
15:  Actualizar pesos:  $W_t \leftarrow (1 - \eta\lambda)W_{t-1} - \eta\alpha(G_t \odot S_t)$ 
16:   $t \leftarrow t + 1$ 
17: until convergencia
```

APOLLO-Mini. APOLLO-Mini es el miembro de la familia optimizado para **eficiencia de memoria extrema**. El artículo lo motiva argumentando que, en un espacio compacto de rango 1, el escalado por canales se vuelve demasiado ruidoso, por lo que la actualización se simplifica aún más a un único escalar por tensor. La pérdida de granularidad se compensa parcialmente con el factor de escala explícito α (el artículo discute valores como 128 en este régimen), proporcionando un punto útil en la frontera de Pareto de optimización: estado muy pequeño, sin sobrecarga de SVD, y aún un comportamiento sólido en preentrenamiento.

Q-APOLLO. El artículo de APOLLO también enfatiza que la familia se combina naturalmente con **cuantización** para entrenamiento con memoria ultra-baja. Este repositorio convierte esa observación de sistemas en una variante concreta del optimizador, **q_apollo**. En la implementación, los momentos de primer y segundo orden de bajo rango de APOLLO se almacenan en forma cuantizada junto con escalas y desplazamientos por tensor, y se descuantizan solo cuando es necesario para el siguiente paso. Q-APOLLO preserva, por tanto, la lógica de gradiente proyectado de APOLLO mientras reduce la precisión del estado restante del optimizador, convirtiéndolo en el miembro más agresivo en ahorro de memoria de la pila local de optimizadores.

Anon (Adaptivity Non-restricted Optimizer with Novel convergence technique). Anon [2] introduce adaptividad ajustable continuamente mediante un único parámetro $\gamma \in \mathbb{R}$, cerrando la brecha entre comportamientos tipo SGD ($\gamma = 0$) y tipo Adam ($\gamma = 1$), y extrapolando más allá de ambos. La innovación central es el mecanismo Incremental Delay Update (IDU), que permite convergencia estable a través de todo el espectro de adaptividad de valores reales:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (\text{momento de primer orden}) \quad (49)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (\text{momento de segundo orden}) \quad (50)$$

$$\bar{v}_t = \frac{v_t}{1 - \beta_2^{\max(t,1)}} + \epsilon \quad (\text{momento de segundo orden con corrección de sesgo}) \quad (51)$$

$$\text{fixed_lr}_k = \begin{cases} \bar{v}_k^{-\gamma/2} & \text{si } k = 0 \\ \sqrt{2 / \left(\frac{1}{\text{fixed_lr}_{k-1}^2} + \bar{v}_k^\gamma \right)} & \text{si } k > 0 \end{cases} \quad (52)$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{1 - \beta_1^t} \cdot m_t \odot \text{fixed_lr}_{\lfloor \log_2 t \rfloor} \quad (53)$$

A diferencia de los optimizadores adaptativos estándar, IDU actualiza el preconditionador solo en potencias de dos ($t = 2^k$), acumulando estadísticas de gradiente entre actualizaciones y agregándolas recursivamente. Este acumulador suave y multiescala evita el seguimiento estricto del máximo de AMSGrad mientras estabiliza demostrablemente la convergencia para cualquier $\gamma \in \mathbb{R}$. El parámetro γ controla directamente la adaptividad: $\gamma > 1$ acelera la salida de puntos de silla (favorable para arquitecturas complejas), $\gamma = 1$ recupera el comportamiento tipo Adam, $\gamma = 0$ produce escalado tipo SGD, y $\gamma < 0$ se sesga hacia mínimos más planos (beneficioso para arquitecturas clásicas como CNNs).

Anon mantiene tres buffers de estado $(m, v, \text{fixed_lr})$ *con costo* $O(n)$ *por paso*, *situados en la misma clase de complejidad* para problemas convexos y $O(\ln T / \sqrt{T})$ para entornos no convexos, igualando las garantías de los optimizadores adaptativos convencionales mientras soporta todo el rango de adaptividad de valores reales.

Algorithm 14 Anon (Adaptivity Non-restricted Optimizer con IDU)

Require: Parámetros iniciales θ_0 , tasa de aprendizaje η , β_1, β_2 , tolerancia ϵ , adaptividad $\gamma \in \mathbb{R}$

Ensure: Parámetros actualizados θ_T

```

1: Inicializar:  $m \leftarrow 0$ ,  $v \leftarrow 0$ ,  $\text{fixed\_lr} \leftarrow 0$ , paso  $t \leftarrow 0$ , contador IDU  $k \leftarrow -1$ 
2: for  $t = 1, 2, \dots, T$  do
3:    $g_t \leftarrow \nabla f(\theta_{t-1})$ 
4:    $m \leftarrow \beta_1 m + (1 - \beta_1) g_t$ 
5:    $v \leftarrow \beta_2 v + (1 - \beta_2) g_t^2$ 
6:   if  $t = 2^k$  then
7:      $k \leftarrow k + 1$ 
8:      $\bar{v} \leftarrow v / (1 - \beta_2^{\max(t/2, 1)}) + \epsilon$ 
9:     if  $k = 0$  then
10:       $\text{fixed\_lr} \leftarrow \bar{v}^{-\gamma/2}$ 
11:     else
12:       $\text{fixed\_lr} \leftarrow \sqrt{2 / (1 / \text{fixed\_lr}^2 + \bar{v}^\gamma)}$ 
13:     end if
14:      $v \leftarrow 0$ 
15:   end if
16:    $\theta_t \leftarrow \theta_{t-1} - \frac{\eta}{1 - \beta_1^t} \cdot m \odot \text{fixed\_lr}$ 
17: end for return  $\theta_T$ 

```

Prodigy (Approximating the Distance Estimate). Prodigy adapta la escala efectiva del paso mediante una estadística tipo distancia acumulativa, eliminando la necesidad de ajuste explícito de la tasa de aprendizaje. El algoritmo mantiene una estimación de distancia acumu-

lativa:

$$u_t = \frac{g_t}{\sqrt{v_t} + \epsilon} \quad (\text{paso adaptativo no normalizado}) \quad (54)$$

$$s_t = s_{t-1} + \langle u_t, \theta_t - \theta_0 \rangle \quad (\text{distancia signada acumulativa}) \quad (55)$$

$$d_t = \text{máx}(d_{t-1}, d_0 + d_{\text{coef}} \cdot s_t) \quad (\text{estimación de distancia con cota inferior}) \quad (56)$$

$$\theta_{t+1} = \theta_t - \eta \cdot d_t \cdot u_t \quad (57)$$

donde d_0 es una cota inicial y d_{coef} controla qué tan agresivamente se adapta la estimación. La estimación de distancia d_t captura la magnitud combinada de gradientes pasados ponderada por el desplazamiento real de los parámetros, implementando una tasa de aprendizaje efectiva implícita. Este escalado consciente de la distancia logra convergencia robusta en distintas escalas de problemas y tamaños de lote sin requerir una programación de la tasa de aprendizaje. Prodigy reduce la sensibilidad a hiperparámetros estimando los tamaños de paso a partir de la geometría de optimización en lugar del conocimiento previo específico del problema.

Algorithm 15 Prodigy (Approximating the Distance Estimate)

Require: Parámetros iniciales θ_0 , tasa de aprendizaje η , coeficiente d_{coef} , distancia inicial $d_0 > 0$

Ensure: Parámetros actualizados θ_T

- 1: Inicializar: $s \leftarrow 0$ (distancia signada acumulativa), $d \leftarrow d_0$
 - 2: **for** $t = 1, 2, \dots, T$ **do**
 - 3: $g_t \leftarrow \nabla f(\theta_{t-1})$
 - 4: $v_t \leftarrow \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$ ▷ Segundo momento
 - 5: $u_t \leftarrow g_t / (\sqrt{v_t} + \epsilon)$ ▷ Adaptativo no normalizado
 - 6: Actualizar distancia: $s \leftarrow s + \langle u_t, \theta_t - \theta_0 \rangle$ ▷ Distancia signada
 - 7: $d \leftarrow \text{máx}(d, d_0 + d_{\text{coef}} \cdot s)$ ▷ Distancia con cota inferior
 - 8: $\theta_t \leftarrow \theta_{t-1} - \eta \cdot d \cdot u_t$ ▷ Actualización escalada por distancia
 - 9: **end for** **return** θ_T
-

1.6. Optimizadores de Segundo Orden, Geométricos y de Ortogonalidad

Algorithm 16 Shampoo (Precondicionamiento Matricial mediante Productos de Kronecker)

Require: Parámetros iniciales θ_0 , tasa de aprendizaje η , intervalo de eigendescomposición K

Ensure: Parámetros actualizados θ_T

- 1: Inicializar: $L \leftarrow \epsilon I_m$, $R \leftarrow \epsilon I_n$ (matrices de Gram izquierda y derecha)
 - 2: **for** $t = 1, 2, \dots, T$ **do**
 - 3: $G \leftarrow \nabla f(\theta_{t-1})$ ▷ Gradiente
 - 4: Actualizar matrices de Gram: $L \leftarrow L + GG^\top$, $R \leftarrow R + G^\top G$
 - 5: **if** $t \bmod K == 0$ **then**
 - 6: $Q_L, \Lambda_L \leftarrow \text{eigh}(L)$ ▷ Eigendescomposición de L
 - 7: $Q_R, \Lambda_R \leftarrow \text{eigh}(R)$ ▷ Eigendescomposición de R
 - 8: $L^{-1/4} \leftarrow Q_L (\Lambda_L + \epsilon)^{-1/4} Q_L^\top$
 - 9: $R^{-1/4} \leftarrow Q_R (\Lambda_R + \epsilon)^{-1/4} Q_R^\top$
 - 10: **end if**
 - 11: $\Delta\theta \leftarrow L^{-1/4} G R^{-1/4}$ ▷ Gradiente precondicionado
 - 12: $\theta_t \leftarrow \theta_{t-1} - \eta \cdot \Delta\theta$
 - 13: **end for** **return** θ_T
-

Shampoo (Precondicionamiento Matricial mediante Productos de Kronecker). Shampoo es un **método estructurado de segundo orden** que calcula preconditionadores matriciales a partir de estadísticas de producto exterior con estructura de Kronecker. Para una matriz

de parámetros 2D $W \in \mathbb{R}^{m \times n}$:

$$L_t = L_{t-1} + G_t G_t^\top \quad (\text{matriz de Gram izquierda/filas, } m \times m) \quad (58)$$

$$R_t = R_{t-1} + G_t^\top G_t \quad (\text{matriz de Gram derecha/columnas, } n \times n) \quad (59)$$

$$L_t^{-1/4} = Q_L(\Lambda_L)^{-1/4} Q_L^\top \quad (\text{mediante eigendescomposición}) \quad (60)$$

$$R_t^{-1/4} = Q_R(\Lambda_R)^{-1/4} Q_R^\top \quad (61)$$

$$\Delta W_t = L_t^{-1/4} G_t R_t^{-1/4} \quad (\text{gradiente preconditionado}) \quad (62)$$

Shampoo aproxima el preconditionador de matriz completa de Adagrad $H^{-1/2}$ (donde H es el Hessiano) utilizando estructura factorizada de Kronecker. En lugar de almacenar un preconditionador de $(mn) \times (mn)$, Shampoo mantiene dos matrices más pequeñas ($m \times m$ y $n \times n$), capturando correlaciones entre parámetros dentro y entre grupos. El método ha demostrado ser efectivo a escala (sistemas de producción de Google) y es adecuado para arquitecturas transformer con estructura de alto rango. Las eigendescomposiciones se realizan periódicamente (e.g., cada $K = 10$ pasos) para amortizar el costo. Contrapartida: mayor cómputo por paso y eigendescomposición $O(m^3 + n^3)$ periódica frente a mejor condicionamiento y convergencia acelerada.

SOAP (Shampoo with Adam in eigenbasis). SOAP es una variante simplificada de Shampoo que desacopla el preconditionamiento del seguimiento de momento. En lugar de álgebra matricial compleja sobre gradientes preconditionados, SOAP ejecuta Adam estándar en la base de eigenvectores de los preconditionadores de Shampoo:

$$L_t = L_{t-1} + G_t G_t^\top, \quad R_t = R_{t-1} + G_t^\top G_t \quad (\text{acumulación de Gram}) \quad (63)$$

$$Q_L, \Lambda_L = \text{eigh}(L_t), \quad Q_R, \Lambda_R = \text{eigh}(R_t) \quad (\text{eigendescomposición periódica}) \quad (64)$$

$$G_t^{\text{rot}} = Q_L^\top G_t Q_R \quad (\text{rotar gradiente a la base de eigenvectores}) \quad (65)$$

$$m_t^{\text{rot}} = \beta_1 m_{t-1}^{\text{rot}} + (1 - \beta_1) G_t^{\text{rot}} \quad (66)$$

$$v_t^{\text{rot}} = \beta_2 v_{t-1}^{\text{rot}} + (1 - \beta_2) (G_t^{\text{rot}})^2 \quad (\text{Adam en el espacio rotado}) \quad (67)$$

$$U_t^{\text{rot}} = \frac{m_t^{\text{rot}}}{\sqrt{v_t^{\text{rot}} + \epsilon}}, \quad U_t = Q_L U_t^{\text{rot}} Q_R^\top \quad (\text{rotar de vuelta}) \quad (68)$$

La idea clave: todo el seguimiento de momento ocurre en la base de eigenvectores bien condicionada, simplificando la estabilidad numérica y el análisis teórico. SOAP combina los beneficios de Shampoo (estructura de curvatura explícita) y Adam (mecánica de momento probada), siendo más limpio y a menudo más estable que Shampoo completo. Las eigendescomposiciones se recalculan cada K pasos, amortizando el costo.

Algorithm 17 SOAP (Shampoo with Adam in Eigenbasis)

Require: Parámetros iniciales θ_0 , tasa de aprendizaje η , β_1, β_2 , intervalo de eigendescomposición K

Ensure: Parámetros actualizados θ_T

```
1: Inicializar:  $L \leftarrow \epsilon I_m, R \leftarrow \epsilon I_n, m \leftarrow 0, v \leftarrow 0$ 
2: for  $t = 1, 2, \dots, T$  do
3:    $G \leftarrow \nabla f(\theta_{t-1})$  ▷ Gradiente
4:   Actualizar Gram:  $L \leftarrow L + GG^\top, R \leftarrow R + G^\top G$ 
5:   if  $t \bmod K == 0$  then
6:      $Q_L, \Lambda_L \leftarrow \text{eigh}(L), Q_R, \Lambda_R \leftarrow \text{eigh}(R)$ 
7:   end if
8:    $G^{\text{rot}} \leftarrow Q_L^\top G Q_R$  ▷ Rotar gradiente a la base de eigenvectores
9:    $m \leftarrow \beta_1 m + (1 - \beta_1) G^{\text{rot}}$  ▷ Media móvil exponencial (corregir sesgo externamente)
10:   $v \leftarrow \beta_2 v + (1 - \beta_2) (G^{\text{rot}})^2$  ▷ Segundo momento (corregir sesgo externamente)
11:   $u \leftarrow m / (\sqrt{v} + \epsilon)$  ▷ Paso adaptativo en la base de eigenvectores
12:   $\Delta\theta \leftarrow Q_L u Q_R^\top$  ▷ Rotar de vuelta al espacio de parámetros
13:   $\theta_t \leftarrow \theta_{t-1} - \eta \cdot \Delta\theta$ 
14: end for return  $\theta_T$ 
```

Lion (EvoLved Sign Momentum). Lion logra un **sobrecosto de memoria mínimo** al usar actualizaciones basadas en signo en lugar de escalado adaptativo completo:

$$c_t = \beta_1 m_t + (1 - \beta_1) g_t \quad (\text{entrada de momento}) \quad (69)$$

$$\theta_{t+1} = \theta_t - \eta (\text{sign}(c_t) + \lambda \theta_t) \quad (\text{actualización con operador signo}) \quad (70)$$

$$m_{t+1} = \beta_2 m_t + (1 - \beta_2) g_t \quad (\text{acumulación de momento}) \quad (71)$$

Todas las operaciones de escalado elemento a elemento se reemplazan con `sign()`, que produce $\{-1, 0, +1\}$. Esto reduce drásticamente la memoria en comparación con métodos tipo Adam: Lion almacena solo el buffer de momento, sin varianza de segundo momento. La contrapartida: las actualizaciones basadas en signo sacrifican la adaptación de tasa de aprendizaje elemento a elemento que hace efectivo a Adam. Lion se posiciona no como un optimizador universalmente superior sino como una alternativa especializada de baja memoria y alto rendimiento para escenarios donde la memoria de activaciones domina y se puede sacrificar algo de sofisticación del optimizador. Funciona bien en regímenes de lotes grandes y cuando el rendimiento del hardware es la restricción principal.

Algorithm 18 Lion (EvoLved Sign Momentum)

Require: Parámetros iniciales θ_0 , tasa de aprendizaje η , β_1, β_2 , decaimiento de pesos λ

Ensure: Parámetros actualizados θ_T

```
1: Inicializar:  $m \leftarrow 0$  (buffer de momento)
2: for  $t = 1, 2, \dots, T$  do
3:    $g \leftarrow \nabla f(\theta_{t-1})$  ▷ Gradiente
4:    $c \leftarrow \beta_1 m + (1 - \beta_1) g$  ▷ Entrada de momento
5:    $\theta_t \leftarrow \theta_{t-1} - \eta (\text{sign}(c) + \lambda \theta_{t-1})$  ▷ Actualización basada en signo con decaimiento de pesos
6:    $m \leftarrow \beta_2 m + (1 - \beta_2) g$  ▷ Acumulación de momento (desacoplada)
7: end for return  $\theta_T$ 
```

Sophia (Second-Order Hessian Information with Optimized Approximation). Sophia utiliza **estimaciones diagonales del Hessiano** para escalado consciente de la curvatura sin

el costo de memoria y cómputo de los métodos densos de segundo orden.

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (\text{momento de primer orden}) \quad (72)$$

$$h_t = \beta_2 h_{t-(k-1)} + (1 - \beta_2) \hat{h}_t \quad (\text{Hessiano diagonal, actualizado cada } k \text{ pasos}) \quad (73)$$

$$\text{clip}(x, C) = \min(\max(x, -C), C) \quad (\text{recorte elemento a elemento}) \quad (74)$$

$$\theta_{t+1} = \theta_t - \eta \cdot \text{clip}\left(\frac{m_t}{\max(\gamma h_t, \epsilon)}, 1\right) \quad (75)$$

donde $\hat{h}_t$ es una estimación diagonal (e.g., estimador de traza de Hutchinson) y γ es un factor de escala. El Hessiano diagonal h_t captura la curvatura local, permitiendo que el optimizador tome pasos más pequeños en direcciones pronunciadas y pasos más grandes en direcciones planas. La operación de recorte $\text{clip}(\cdot, 1)$ evita que los pasos adaptativos exploten. Sophia pertenece a la familia de métodos conscientes de la curvatura que buscan mejor condicionamiento sin el costo $O(n^2)$ o $O(n^3)$ de los métodos completos de segundo orden. Funciona particularmente bien en escenarios de ajuste fino de segunda pasada.

Algorithm 19 Sophia (Hessiano Diagonal con Actualizaciones Recortadas)

Require: Parámetros iniciales θ_0 , tasa de aprendizaje η , β_1, β_2 , frecuencia de actualización del Hessiano k , umbral de recorte C

Ensure: Parámetros actualizados θ_T

```

1: Inicializar:  $m \leftarrow 0$ ,  $h \leftarrow \epsilon$  (estimación diagonal del Hessiano)
2: for  $t = 1, 2, \dots, T$  do
3:    $g \leftarrow \nabla f(\theta_{t-1})$ 
4:    $m \leftarrow \beta_1 m + (1 - \beta_1) g$   $\triangleright$  Momento de primer orden (corregir sesgo externamente)
5:   if  $t \bmod k == 0$  then
6:      $\hat{h} \leftarrow \text{HutchinsonEstimate}(g)$   $\triangleright$  Hessiano diagonal mediante Hutchinson
7:      $h \leftarrow \beta_2 h + (1 - \beta_2) \hat{h}$   $\triangleright$  Actualizar estimación del Hessiano
8:   end if
9:    $u \leftarrow \frac{m}{\max(\gamma h, \epsilon)}$   $\triangleright$  Paso adaptativo (elemento a elemento)
10:   $u \leftarrow \text{clip}(u, C)$   $\triangleright$  Recorte elemento a elemento a  $[-C, C]$ 
11:   $\theta_t \leftarrow \theta_{t-1} - \eta \cdot u$ 
12: end for return  $\theta_T$ 

```

Muon y Turbo-Muon (Optimizadores Basados en Ortogonalidad). Muon y Turbo-Muon son **optimizadores orientados a la ortogonalidad** que reformulan la geometría de actualización utilizando iteraciones polinomiales de Newton-Schulz para ortogonalización. Para una matriz de parámetros 2D W con gradiente G_t :

$$X_0 = \frac{G_t}{\|G_t\|_F + \epsilon} \quad (\text{gradiente normalizado}) \quad (76)$$

$$A_k = X_k X_k^\top \quad (\text{Gramiana}) \quad (77)$$

$$B_k = b A_k + c A_k^2 \quad (\text{paso polinomial, coeficientes } b, c \text{ de Newton-Schulz}) \quad (78)$$

$$X_{k+1} = a X_k + B_k X_k \quad (\text{iteración } k = 0, 1, \dots, 4) \quad (79)$$

$$W_{t+1} = W_t - \eta X_K \quad (\text{actualización con dirección ortogonalizada}) \quad (80)$$

Muon realiza 5 iteraciones de Newton-Schulz para producir una dirección aproximadamente ortogonal, asegurando que las actualizaciones respeten restricciones geométricas. Turbo-Muon añade un paso de **precondicionamiento casi-ortogonal** (AOL) antes de las iteraciones, reduciendo el número de pasos de ortogonalización requeridos a 4. La justificación: las actualizaciones ortogonales preservan las normas durante el entrenamiento, evitando la deriva de norma observada en

métodos basados en momento. Estos optimizadores muestran potencial en entrenamiento a gran escala pero requieren kernels CUDA personalizados para eficiencia. Contrapartida: alto cómputo por paso (multiplicaciones de matrices e iteraciones polinomiales) frente a una geometría de actualización fundamentalmente mejor condicionada.

Algorithm 20 Muon (Ortogonalización de Newton-Schulz)

Require: Parámetros iniciales θ (matrices), tasa de aprendizaje η , iteraciones Newton-Schulz K

Ensure: Parámetros actualizados θ

```

1: for cada matriz de parámetros 2D  $W$  do
2:    $G \leftarrow \nabla f(W)$  ▷ Gradiente
3:    $X_0 \leftarrow \frac{G}{\|G\|_{F+\epsilon}}$  ▷ Normalizar gradiente por norma de Frobenius
4:   for  $k = 0, 1, \dots, K - 1$  do
5:      $A_k \leftarrow X_k X_k^\top$  ▷ Gramiana (costo:  $O(mn^2)$  o  $O(m^2n)$  para matrices altas/anchas)
6:      $B_k \leftarrow (3/2)A_k - (1/2)A_k^2$  ▷ Coeficientes de Newton-Schulz
7:      $X_{k+1} \leftarrow B_k X_k$  ▷ Paso polinomial
8:   end for
9:    $W \leftarrow W - \eta X_K$  ▷ Actualizar con dirección ortogonalizada
10: end for return  $\theta$  actualizado

```

Algorithm 21 Turbo-Muon (Ortogonalización Precondicionada con AOL)

Require: Parámetros iniciales θ (matrices), tasa de aprendizaje η , iteraciones Newton-Schulz K' (típicamente 4)

Ensure: Parámetros actualizados θ

```

1: for cada matriz de parámetros 2D  $W$  do
2:    $G \leftarrow \nabla f(W)$  ▷ Gradiente
3:    $X_0 \leftarrow \frac{G}{\|G\|_{F+\epsilon}}$  ▷ Normalizar gradiente
4:   AOL (Almost-Orthogonal preconditioner): ▷ Paso de preconditionamiento
5:    $P \leftarrow (1,5I - 0,5X_0 X_0^\top)$  ▷ Matriz de preconditionamiento
6:    $X_0 \leftarrow P X_0$  ▷ Inicio preconditionado
7:   for  $k = 0, 1, \dots, K' - 1$  do
8:      $A_k \leftarrow X_k X_k^\top$ 
9:      $B_k \leftarrow (3/2)A_k - (1/2)A_k^2$ 
10:     $X_{k+1} \leftarrow B_k X_k$  ▷ Iteraciones reducidas gracias al preconditionamiento AOL
11:   end for
12:    $W \leftarrow W - \eta X_{K'}$  ▷ Actualizar con dirección ortogonalizada preconditionada
13: end for return  $\theta$  actualizado

```

Grupo	Métodos	Objetivo Principal	Interpretación desde el Estudio
Línea base clásica	SGD, AdamW, RADam	estabilidad y referencias base	Definen el piso de comparación para las afirmaciones de optimizadores más nuevos.
Rediseño de momento	Adan, AdEMAMix, MARS, Cautious AdamW	adaptación de primer orden más rápida o segura	Mejor cuando la velocidad de convergencia o la estabilidad ante gradiente ruidoso es la preocupación principal.

Grupo	Métodos	Objetivo Principal	Interpretación desde el Estudio
Simplificación de lotes grandes y programación	LAMB, Schedule-Free AdamW	robustez operacional a escala	Reducen la fragilidad debida al crecimiento del tamaño de lote o la ingeniería de programación.
Eficiencia de memoria	Adafactor, GaLore, APOLLO, APOLLO-Mini, Q-APOLLO, Lion	reducción del estado del optimizador	Más útil cuando la VRAM está dominada por el estado del optimizador en lugar de las activaciones; los métodos de la familia APOLLO reemplazan los momentos densos de AdamW con escalado estructurado proyectado o cuantizado.
Consciente de curvatura	Shampoo, Sophia	mejor condicionamiento	Preferir cuando una geometría más rica justifica la sobrecarga de implementación y cómputo.
Orientado a geometría	Muon, Turbo-Muon	estructura de actualización ortogonalizada	Opciones especializadas para geometría matricial y modelado de representaciones.

2. Transformadores Densos, Recurrentes y Aumentados con Memoria

Este anexo sintetiza las familias densas, recurrentes y aumentadas con memoria de transformers. El campo ha evolucionado hacia varios paradigmas principales: atención densa global con variantes, compresión de cabezas clave-valor mediante atención por consultas agrupadas, compresión de estado recurrente con decaimiento, modelos de espacio de estado selectivos, integración numérica de profundidad continua y arquitecturas aumentadas con memoria. Comprender esta taxonomía ilumina las compensaciones fundamentales entre expresividad, costo computacional, huella de memoria y simplicidad de despliegue.

2.1. Atención Densa de Referencia: Estándar y Sigmoide

2.1.1. Atención Softmax Estándar

La atención multi-cabeza estándar [51] calcula la similitud de producto punto escalado entre embeddings de tokens:

- 1: **Entrada:** Matriz de consulta $\mathbf{Q} \in \mathbb{R}^{n \times d}$, Matriz de clave $\mathbf{K} \in \mathbb{R}^{n \times d}$, Matriz de valor $\mathbf{V} \in \mathbb{R}^{n \times d}$
- 2: $\text{puntajes} \leftarrow \frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}} \in \mathbb{R}^{n \times n}$ ▷ calcular similitudes por pares
- 3: $\text{pesos_atencion} \leftarrow \text{softmax}(\text{puntajes}, \text{eje} = 1) \in \mathbb{R}^{n \times n}$ ▷ normalizar sobre las claves
- 4: **Salida:** $\mathbf{Y} = \text{pesos_atencion} \cdot \mathbf{V} \in \mathbb{R}^{n \times d}$ ▷ combinación ponderada de valores

Para generación autorregresiva, el almacenamiento en caché KV guarda claves y valores pasados para evitar el recálculo $\mathcal{O}(n^2)$. Sin embargo, este caché de crecimiento lineal ocupa $\sim n \cdot d_{\text{oculto}}$ bytes, lo que puede consumir cientos de gigabytes para modelos de miles de millones de parámetros. La atención estándar logra una **expresividad perfecta** dentro de la ventana de contexto—cualquier token puede atender a cualquier otro token con pesos aprendidos—pero paga el precio del cómputo denso.

- **Complejidad de entrenamiento:** $\mathcal{O}(n^2 \cdot d)$ tiempo, $\mathcal{O}(n^2)$ espacio (materialización de la matriz de atención).
- **Complejidad de inferencia:** $\mathcal{O}(n)$ tiempo por token, $\mathcal{O}(n)$ espacio (caché KV).

- **Fortalezas:** Expresividad incomparable; recuperación perfecta del historial; altamente paralelizable.
- **Debilidades:** El cuello de botella cuadrático prohíbe contextos muy largos; el caché KV domina la memoria durante la generación.

2.1.2. Atención Sigmoides

La atención sigmoides reemplaza el softmax por filas con una activación sigmoides elemento a elemento [42]:

- 1: **Entrada:** $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ como arriba, sesgo aprendible $\mathbf{b} \in \mathbb{R}^{n \times n}$
- 2: $\text{logits} \leftarrow \frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}} + \mathbf{b}$
- 3: $\text{pesos_atencion} \leftarrow \sigma(\text{logits})$ ▷ sigmoides elemento a elemento, no softmax
- 4: **Salida:** $\mathbf{Y} = \text{pesos_atencion} \odot \mathbf{V}$

A diferencia de softmax, la sigmoides no impone una distribución de probabilidad (los pesos no necesitan sumar 1), lo que permite una independencia de tokens más fuerte. El análisis teórico mediante mezcla de expertos muestra que la sigmoides alcanza una complejidad muestral superior: convergencia $\mathcal{O}(n^{-0.51})$ para expertos ReLU frente a $\mathcal{O}(n^{-0.24})$ de softmax. Sin embargo, el entrenamiento empírico reveló inestabilidades de gradiente a escala. El remedio es la **norma híbrida**—agregar normalización después de la salida de atención—que estabiliza los gradientes sin sacrificar los beneficios teóricos del enrutamiento elemento a elemento.

- **Complejidad de entrenamiento:** $\mathcal{O}(n^2 \cdot d)$ (idéntico al estándar), pero las operaciones elemento a elemento permiten una aceleración de inferencia del 17 % mediante FlashSigmoid.
- **Complejidad de inferencia:** $\mathcal{O}(n)$ por token con caché KV (asintóticamente igual, pero con factores constantes más bajos).
- **Fortalezas:** Supera la competencia de suma cero; evita la sincronización por filas; implementación eficiente en hardware.
- **Debilidades:** Requiere estabilización cuidadosa (norma híbrida); inestabilidad de entrenamiento a grandes escalas sin pérdida auxiliar.

2.1.3. Atención por Consultas Agrupadas (GQA)

La atención por consultas agrupadas (GQA) [1] tiende un puente entre la atención multi-cabeza (MHA) y la atención multi-consulta (MQA) al particionar las cabezas de consulta en grupos, cada uno compartiendo una sola cabeza clave-valor. Si h es el número total de cabezas de atención, h_k es el número de cabezas clave-valor, y $G = h/h_k$ es el tamaño del grupo, entonces cada grupo de G cabezas de consulta comparte una cabeza KV:

- 1: **Entrada:** $\mathbf{Q} \in \mathbb{R}^{n \times h \cdot d_h}, \mathbf{K} \in \mathbb{R}^{n \times h_k \cdot d_h}, \mathbf{V} \in \mathbb{R}^{n \times h_k \cdot d_h}$
- 2: $\mathbf{K}_{\text{completo}} \leftarrow \text{repeat_interleave}(\mathbf{K}, G)$ ▷ expandir de h_k a h cabezas
- 3: $\mathbf{V}_{\text{completo}} \leftarrow \text{repeat_interleave}(\mathbf{V}, G)$
- 4: $\text{puntajes} \leftarrow \frac{\mathbf{Q}\mathbf{K}_{\text{completo}}^\top}{\sqrt{d_h}} \in \mathbb{R}^{n \times n}$ ▷ producto punto estándar tras expansión
- 5: $\text{pesos_atencion} \leftarrow \text{softmax}(\text{puntajes}, \text{eje} = 1)$
- 6: **Salida:** $\mathbf{Y} = \text{pesos_atencion} \cdot \mathbf{V}_{\text{completo}} \in \mathbb{R}^{n \times h \cdot d_h}$

GQA interpola suavemente entre MHA ($h_k = h, G = 1$) y MQA ($h_k = 1, G = h$). La ventaja clave es un balance ajustable entre la huella del caché KV y la capacidad representacional. Cada cabeza KV adicional consume $2 \cdot d_h \cdot n$ bytes extra en el caché, por lo que reducir h_k produce ahorros proporcionales de memoria.

- **Complejidad de entrenamiento:** $\mathcal{O}(n^2 \cdot h \cdot d_h)$ (idéntico a MHA tras la expansión de cabezas).
- **Complejidad de inferencia:** $\mathcal{O}(n)$ por token, espacio de caché KV $\mathcal{O}(h_k \cdot n \cdot d_h)$.
- **Fortalezas:** Balance flexible de cabezas; probado a escala (Llama 2 y 3 usan $h_k = 8$ cabezas KV con $h = 32$ cabezas de consulta); reduce el caché KV en factor $G\times$; soporta inferencia rápida con FlashAttention.
- **Debilidades:** Menos cabezas KV pueden reducir la capacidad representacional en modelos pequeños; requerir que h_k divida a h restringe las opciones arquitectónicas.

2.2. Arquitecturas Recurrentes y Retentivas

2.2.1. Redes Retentivas (RetNet)

RetNet [49] unifica tres paradigmas de cómputo: entrenamiento paralelo, inferencia recurrente y despliegue por fragmentos. Su innovación central es el **mecanismo de retención**, que utiliza una matriz de decaimiento exponencial fija para modelar la importancia temporal:

- 1: **Forma paralela (entrenamiento):**
- 2: $\text{matriz_decaimiento}[i, j] \leftarrow \gamma^{i-j}$ para $i \geq j$, si no 0 ▷ decaimiento exponencial causal
- 3: $\text{matriz_decaimiento}[i, j] \leftarrow 0$ para $i < j$ ▷ enmascaramiento causal
- 4: $\mathbf{Y}_{\text{paralelo}} \leftarrow (\mathbf{QK}^\top \odot \text{matriz_decaimiento})\mathbf{V}$ ▷ multiplicación elemento a elemento con decaimiento
- 1: **Forma recurrente (inferencia):**
- 2: **for** $t = 1, \dots, n$ **do**
- 3: $\mathbf{s}_t \leftarrow \gamma \mathbf{s}_{t-1} + \mathbf{k}_t \mathbf{v}_t^\top$ ▷ actualización de estado con decaimiento
- 4: $\mathbf{y}_t \leftarrow \mathbf{q}_t \mathbf{s}_t$ ▷ salida mediante interacción consulta-estado
- 5: **end for**

El escalar de decaimiento $\gamma \in (0, 1)$ controla la ventana temporal. RetNet utiliza **retención multi-escala** con diferentes valores de γ por cabeza (ej., $\gamma = 1 - 2^{-5}, 1 - 2^{-6}, \dots$), permitiendo dependencias a corto y largo plazo simultáneamente. El modo recurrente por fragmentos divide las secuencias en fragmentos, procesa cada fragmento en paralelo y enhebra un estado recurrente entre fragmentos.

- **Complejidad de entrenamiento:** $\mathcal{O}(n^2 \cdot d)$ (paralela), o $\mathcal{O}(n \cdot c \cdot d)$ (recurrente por fragmentos con tamaño de fragmento c).
- **Complejidad de inferencia:** $\mathcal{O}(1)$ por token, $\mathcal{O}(d^2)$ espacio de estado (matriz fija).
- **Fortalezas:** Inferencia en tiempo constante; triple paradigma de cómputo; consolidación multi-escala.
- **Debilidades:** El decaimiento fijo impone un sesgo inductivo rígido; puede truncar patrones de largo alcance aprendidos.

2.2.2. Mamba: Modelos de Espacio de Estado Selectivos

Mamba [18] enmarca la recurrencia como un sistema dinámico de tiempo continuo con parámetros dependientes de la entrada, logrando tanto entrenamiento lineal como inferencia en tiempo constante:

- 1: **Dinámica continua:** $h'(t) = \mathbf{A}h(t) + \mathbf{B}x(t), \quad y(t) = \mathbf{C}h(t)$
- 2: **Discretización con tamaño de paso Δ_t :**
- 3: $\bar{\mathbf{A}}_t \leftarrow \exp(\Delta_t \mathbf{A})$

```

4:  $\bar{\mathbf{B}}_t \leftarrow (\Delta_t \mathbf{A})^{-1}(\exp(\Delta_t \mathbf{A}) - \mathbf{I})\Delta_t \mathbf{B}_t$ 
5: Actualización recurrente:
6: for  $t = 1, \dots, n$  do
7:    $\Delta_t \leftarrow \text{softplus}(\text{Lineal}(x_t))$  ▷ tamaño de paso dependiente de la entrada
8:    $\mathbf{B}_t \leftarrow \text{Lineal}(x_t), \quad \mathbf{C}_t \leftarrow \text{Lineal}(x_t)$  ▷ matrices de proyección dependientes de la entrada
9:    $h_t \leftarrow \bar{\mathbf{A}}_t h_{t-1} + \bar{\mathbf{B}}_t x_t$  ▷ transición de estado
10:   $y_t \leftarrow \mathbf{C}_t h_t$  ▷ proyección de salida
11: end for

```

La innovación crítica es que $\mathbf{A}$ (la matriz del sistema) *no* depende de la entrada, pero Δ_t , $\mathbf{B}_t$ y $\mathbf{C}_t$ sí, haciendo que el sistema sea **variante en el tiempo**. Esta selectividad permite al modelo *ignorar* información irrelevante estableciendo Δ_t cerca de cero, creando efectivamente una compuerta. Un algoritmo de escaneo paralelo consciente del hardware implementa la recurrencia eficientemente en GPUs fusionando el cómputo dentro de SRAM, evitando el costoso ancho de banda de HBM.

- **Complejidad de entrenamiento:** $\mathcal{O}(n \cdot d)$ mediante escaneo consciente del hardware (lineal en la longitud de la secuencia).
- **Complejidad de inferencia:** $\mathcal{O}(1)$ por token, $\mathcal{O}(d)$ estado (vector oculto).
- **Fortalezas:** Logra entrenamiento lineal e inferencia constante simultáneamente; práctico en secuencias largas (millones de tokens).
- **Debilidades:** La compresión del vector de estado puede debilitar la copia exacta y la recuperación asociativa densa en comparación con la atención completa.

2.3. Transformers de Profundidad Continua: Integración EDO

2.3.1. Transformer EDO

El Transformer EDO [64] interpreta la profundidad de la red como integración numérica de un sistema dinámico continuo, utilizando solucionadores Runge-Kutta de orden superior para reducir el error de truncamiento:

- 1: **Formulación continua:** $\frac{dh(t)}{dt} = f_\theta(h(t), t)$ donde f_θ es la subred del transformer
- 2: **Aproximación discreta Runge-Kutta-4:**
- 3: $k_1 \leftarrow f_\theta(h_t, t)$
- 4: $k_2 \leftarrow f_\theta(h_t + \frac{1}{2}k_1, t + \frac{1}{2}\Delta t)$
- 5: $k_3 \leftarrow f_\theta(h_t + \frac{1}{2}k_2, t + \frac{1}{2}\Delta t)$
- 6: $k_4 \leftarrow f_\theta(h_t + k_3, t + \Delta t)$
- 7: $h_{t+1} \leftarrow h_t + \frac{\Delta t}{6}(k_1 + 2k_2 + 2k_3 + k_4)$ ▷ combinación ponderada

En lugar de las conexiones residuales simples de Euler $h_{t+1} = h_t + f_\theta(h_t)$, el bloque RK4 calcula cuatro evaluaciones intermedias y las combina con los pesos clásicos de RK4. Para evitar gradientes que se desvanecen, la arquitectura introduce **compuertas aprendidas** que interpolan entre aproximaciones intermedias:

- 1: $g \leftarrow \sigma(\text{Lineal}([k_1, k_2, k_3, k_4]))$ ▷ compuerta aprendible
- 2: $h_{t+1} \leftarrow h_t + g \cdot k_1 + (1 - g) \cdot k_2$ ▷ refinamiento interpolado

Esta formulación reduce el número efectivo de parámetros mediante el uso compartido de pesos—la misma f_θ se evalúa múltiples veces—proporcionando un refinamiento de trayectoria más rico.

- **Complejidad de entrenamiento:** $\mathcal{O}(k \cdot n^2 \cdot d)$ donde k es el orden RK (4 para RK4).

- **Complejidad de inferencia:** $\mathcal{O}(k \cdot n)$ por token (mayor sobrecarga constante).
- **Fortalezas:** Precisión significativamente mayor en tareas de generación (BLEU de última generación); eficiente en parámetros mediante uso compartido de pesos.
- **Debilidades:** Alto costo de cómputo por paso; la latencia de inferencia aumenta por un factor constante k ; requiere compuertas complejas para la estabilidad.

2.4. Memoria en Tiempo de Prueba: Titans

La arquitectura Titans [4] introduce una dimensión ortogonal: en lugar de pesos estáticos, el modelo mantiene una memoria aprendible que se *actualiza durante la inferencia* basada en una señal impulsada por sorpresa:

- 1: **Atención local a corto plazo:** Aplicar atención estándar o dispersa dentro de una ventana de contexto fija c
- 2: $y_t^{\text{local}} \leftarrow \text{Atencion}(q_t, k_{[t-c:t]}, v_{[t-c:t]})$
- 3: **Señal de actualización de memoria (sorpresa):**
- 4: $S_t \leftarrow \eta_t S_{t-1} - \theta_t \nabla_\ell(\mathcal{M}_{t-1}; x_t)$ $\triangleright$ sorpresa como gradiente de la pérdida respecto a la memoria
- 5: $\mathcal{M}_t \leftarrow (1 - \alpha_t) \mathcal{M}_{t-1} + S_t$ $\triangleright$ memoria actualizada mediante EMA
- 6: **Recuperación de memoria a largo plazo:**
- 7: $y_t^{\text{memoria}} \leftarrow \mathcal{M}_t^*(q_t)$ $\triangleright$ consultar módulo de memoria para información recuperada
- 8: **Combinación de salida:**
- 9: $y_t \leftarrow y_t^{\text{local}} + \text{compuerta}(y_t^{\text{memoria}})$ $\triangleright$ combinar memoria local y recuperada

El módulo de memoria $\mathcal{M}$ se actualiza literalmente durante el paso hacia adelante calculando gradientes de una pérdida asociativa y aplicando pasos de SGD con momento. Las tasas de decaimiento $\eta_t, \theta_t, \alpha_t$ son en sí mismas dependientes de la entrada, permitiendo al modelo cambiar de paradigma de memoria cuando el contexto cambia.

- **Complejidad de entrenamiento:** Aproximadamente $\mathcal{O}(c^2 + n \cdot f)$ donde c es la ventana local y f es la sobrecarga de actualización de memoria.
- **Complejidad de inferencia:** $\mathcal{O}(c^2)$ atención local más $\mathcal{O}(1)$ recuperación de memoria por token.
- **Fortalezas:** Maneja longitudes de contexto extremas; permite recuperación asociativa real; la memoria se adapta a la entrada.
- **Debilidades:** Significativamente más complejo; la inferencia incluye cálculos de gradientes; mayor sobrecarga de coordinación.

2.5. Despacho de Mezclador Guiado por Patrón

El código base selecciona un mezclador por bloque a partir del `layer_pattern` configurado. La lógica de despacho impone la política sin entrenamiento para bloques de solo evaluación y enruta a la implementación de familia correspondiente.

2.6. Comparación y Síntesis Arquitectónica

Algorithm 22 Paso Adelante del Mezclador Guiado por Patrón (Conceptual)

Require: Estados ocultos H , patrón P , índice de capa ℓ

```
1:  $m \leftarrow P[\ell \bmod |P|]$ 
2: if  $m \in \{\text{fasa\_attn}, \text{sparge\_attn}\}$  y el modelo está en modo entrenamiento then
3:   lanzar error de configuración/tiempo de ejecución (bloque sin entrenamiento en modo
   entrenar)
4: else if  $m = \text{standard\_attn}$  then
5:    $H \leftarrow \text{softmax-attention}(H)$ 
6: else if  $m = \text{sigmoid\_attn}$  then
7:    $H \leftarrow \text{sigmoid-attention}(H)$ 
8: else if  $m \in \{\text{retnet}, \text{retnet\_attn}\}$  then
9:    $H \leftarrow \text{retention}(H)$ 
10: else if  $m = \text{mamba}$  then
11:    $H \leftarrow \text{selective-ssm}(H)$ 
12: else if  $m = \text{ode}$  then
13:    $H \leftarrow \text{rk-step}(H)$ 
14: else if  $m$  es una clave de atención dispersa then
15:    $H \leftarrow \text{sparse-attention-family}(H)$ 
16: else if  $m$  es una clave de atención con compuerta then
17:    $H \leftarrow \text{gated-attention-family}(H)$ 
18: else
19:    $H \leftarrow \text{memory-augmented-attn}(H)$ 
20: end if
21: return  $H$ 
```

Arquitectura	Entrenar	Inferir	Estado	Característica Clave
Atención Es-tándar	$O(n^2d)$	$O(n)$ caché	$O(nd)$	Expresividad perfecta, enrutamiento completo de tokens, cuello de botella KV.
GQA	$O(n^2d)$	$O(n)$ caché	$O(h_k \cdot n \cdot d_h)$	Compresión KV ajustable, probado a escala, reducción de caché en factor $G \times$.
Atención Sigmoid	$O(n^2d)$	$O(n)$ caché	$O(nd)$	Compuerta elemento a elemento, eficiente en hardware, estable a escala con norma híbrida.
RetNet	$O(n^2d)$	$O(1)$	$O(d^2)$	Decaimiento multi-escala, triple modo de cómputo, patrón de olvido fijo.
Mamba	$O(nd)$	$O(1)$	$O(d)$	Dinámica selectiva dependiente de la entrada, entrenamiento e inferencia lineales, escaneo de hardware.
Transformer EDO	$O(kn^2d)$	$O(kn)$	$O(nd)$	Refinamiento por integración numérica, uso compartido de pesos por etapas, mayor precisión.
Titans	$O(c^2 + nf)$	$O(c^2 + 1)$	$O(1)$	Adaptación de memoria en tiempo de prueba, contexto extremo, actualizaciones de parámetros durante inferencia.

El campo exhibe una progresión clara a lo largo de dos ejes: (i) **eficiencia computacional**, pasando de $O(n^2)$ a $O(n)$ u $O(1)$, y (ii) **adaptabilidad de memoria**, pasando de pesos estáticos a modelos dinámicos actualizados en tiempo de prueba. La atención estándar sigue siendo la línea base de expresividad; Mamba y RetNet representan la frontera práctica de eficiencia; Titans introduce un eje de innovación ortogonal (aprendizaje en tiempo de prueba). La elección de arquitectura refleja la restricción fundamental de ingeniería: expresividad versus costo de despliegue. La familia de atención latente, que comprime el caché KV en un latente de rango bajo, se cubre en el Apéndice 3.

3. Familias de Atención Latente y de Rango Bajo

Este anexo cubre la familia de mecanismos de atención de compresión latente / de rango bajo de KV. Estas arquitecturas comprimen el estado clave-valor por token en un vector latente de rango bajo que es la única cantidad materializada en el caché KV durante la decodificación, luego reconstruyen las claves y valores completos de todas las cabezas al vuelo. La familia unifica diseños de consulta agrupada, latente, factorizado y latente temporal bajo un cuello de botella de rango bajo común.

3.1. Atención Latente Multi-Cabeza (MLA)

La Atención Latente Multi-Cabeza (MLA, por sus siglas en inglés) [34] comprime el estado clave-valor por token en un único vector latente de rango bajo c_{KV} de rango r_{kv} , el cual es la única cantidad materializada en el caché KV durante la decodificación. Un par de matrices de up-proyección reconstruye las claves y valores completos de todas las cabezas al vuelo, mientras que RoPE se aplica a los tensores de consulta y clave descomprimidos para preservar la estructura posicional. Los autores muestran que el ancho latente óptimo de Pareto se sitúa cerca de $r_{kv} = d_{\text{oculto}}/2$, punto en el cual MLA reduce a la mitad el caché KV pero iguala la calidad de la atención multi-cabeza (MHA) en modelos pequeños.

3.1.1. Formulación Matemática

$$c_{KV} = W_{DKV} x \in \mathbb{R}^{r_{kv}}, \quad k = W_{UK} c_{KV}, \quad v = W_{UV} c_{KV}, \quad q = W_Q x.$$

$$\tilde{q} = \text{RoPE}(q), \quad \tilde{k} = \text{RoPE}(k), \quad \text{atencion} = \text{softmax}\left(\frac{\tilde{q} \tilde{k}^\top}{\sqrt{d_h}}\right) v.$$

3.1.2. Pseudocódigo Algorítmico

- 1: **Entrada:** estado oculto $x \in \mathbb{R}^{n \times d}$, rango r_{kv} , proyecciones $W_{DKV}, W_{UK}, W_{UV}, W_Q$
- 2: $c_{KV} \leftarrow W_{DKV} x$ ▷ down-proyección a latente de rango r_{kv} (en caché en inferencia)
- 3: $k \leftarrow W_{UK} c_{KV}, \quad v \leftarrow W_{UV} c_{KV}$ ▷ up-proyección a cabezas completas
- 4: $q \leftarrow W_Q x$
- 5: $\tilde{q} \leftarrow \text{RoPE}(q), \quad \tilde{k} \leftarrow \text{RoPE}(k)$ ▷ aplicar embeddings rotatorios
- 6: pesos $\leftarrow \text{softmax}(\tilde{q} \tilde{k}^\top / \sqrt{d_h})$
- 7: **Salida:** $Y = \text{pesos} \cdot v$

3.1.3. Características Clave

- **Complejidad de entrenamiento:** $\mathcal{O}(n^2 \cdot d)$ (atención completa sobre cabezas descomprimidas).
- **Complejidad de inferencia:** $\mathcal{O}(n)$ por token, espacio de caché KV $\mathcal{O}(r_{kv} \cdot n)$ (sólo latente).

- **Entrenable:** totalmente entrenable; r_{kv} es un hiperparámetro (óptimo de Pareto $\approx d/2$).
- **Fortalezas:** reduce a la mitad el caché KV frente a MHA con calidad equivalente; compatible con RoPE; cuello de botella latente limpio.
- **Limitaciones:** la up-proyección añade FLOPs de decodificación; el rango latente es un presupuesto global fijo compartido entre cabezas.

3.2. Atención Latente por Consultas Agrupadas (GQLA)

La Atención Latente por Consultas Agrupadas (GQLA, por sus siglas en inglés) [35] es una modificación mínima de MLA que expone dos rutas de decodificación algebraicamente equivalentes sobre los mismos pesos entrenados: una ruta MQA-absorb (que fusiona la up-proyección en la consulta, óptima en hardware tipo H100) y una ruta GQA-expanded (que materializa claves/valores completos, óptima en hardware H20 / predicción multi-token). Las dos rutas son matemáticamente idénticas porque la up-proyección es lineal, de modo que $\text{softmax}(q_{\text{absorbido}} \cdot c_{KV}^\top) = \text{softmax}((q \cdot W_{UK}^\top) \cdot c_{KV}^\top)$, permitiendo un despacho adaptativo al hardware sin reentrenamiento.

3.2.1. Formulación Matemática

Mismo latente que MLA:

$$c_{KV} = W_{DKV} x, \quad k = W_{UK} c_{KV}, \quad v = W_{UV} c_{KV}, \quad q = W_Q x.$$

$$\text{Ruta A (MQA-absorb): } \text{atencion} = \text{softmax}\left(\frac{(q W_{UK}^\top) c_{KV}^\top}{\sqrt{d_h}}\right) (W_{UV} c_{KV}).$$

$$\text{Ruta B (GQA-expanded): } \text{atencion} = \text{softmax}\left(\frac{q k^\top}{\sqrt{d_h}}\right) v, \quad k = W_{UK} c_{KV}, \quad v = W_{UV} c_{KV}.$$

3.2.2. Pseudocódigo Algorítmico

- 1: **Entrada:** x , pesos $W_{DKV}, W_{UK}, W_{UV}, W_Q$, bandera de hardware $hw \in \{H100, H20\}$
- 2: $c_{KV} \leftarrow W_{DKV} x$ ▷ latente compartido, en caché
- 3: $q \leftarrow W_Q x$
- 4: **if** $hw == H100$ **then** ▷ ruta MQA-absorb
- 5: $q_{\text{abs}} \leftarrow q W_{UK}^\top$ ▷ absorber up-proyección en la consulta
- 6: pesos $\leftarrow \text{softmax}(q_{\text{abs}} c_{KV}^\top / \sqrt{d_h})$
- 7: $Y \leftarrow \text{pesos} \cdot (W_{UV} c_{KV})$
- 8: **else** ▷ ruta GQA-expanded
- 9: $k \leftarrow W_{UK} c_{KV}, \quad v \leftarrow W_{UV} c_{KV}$
- 10: pesos $\leftarrow \text{softmax}(q k^\top / \sqrt{d_h})$
- 11: $Y \leftarrow \text{pesos} \cdot v$
- 12: **end if**
- 13: **Salida:** Y

3.2.3. Características Clave

- **Complejidad de entrenamiento:** $\mathcal{O}(n^2 \cdot d)$, idéntica a MLA.
- **Complejidad de inferencia:** $\mathcal{O}(n)$ por token, estado latente $\mathcal{O}(r_{kv} \cdot n)$; ruta elegida por hardware.
- **Entrenable:** se entrena una sola vez; las dos rutas de decodificación reutilizan los mismos pesos.

- **Fortalezas:** decodificación adaptativa al hardware sin pérdida de calidad; la equivalencia algebraica garantiza consistencia.
- **Limitaciones:** mismas compensaciones de rango latente que MLA; la selección de ruta es una heurística de despliegue.

3.3. Atención Multi-Cabeza de Rango Bajo (MLRA)

La Atención Multi-Cabeza de Rango Bajo (MLRA, por sus siglas en inglés) [31] refactoriza el latente único de MLA en L sub-cabezas disjuntas cada una de rango r/L , de modo que el caché KV puede partitionarse entre L dispositivos para una decodificación tensor-paralela de 4 vías (cada dispositivo carga sólo $1/L$ del caché). Las up-proyecciones por sub-cabeza reconstruyen la porción correspondiente de las cabezas completas y se concatenan, logrando una aceleración de decodificación de $2.8\times$ sobre MLA preservando el presupuesto latente global.

3.3.1. Formulación Matemática

$$\begin{aligned}
c_{KV} &= [c_1, c_2, \dots, c_L], \quad c_i \in \mathbb{R}^{r/L}, \quad c_{KV} = W_{DKV} x. \\
k_i &= W_{UK}^{(i)} c_i, \quad v_i = W_{UV}^{(i)} c_i, \quad k = [k_1; \dots; k_L], \quad v = [v_1; \dots; v_L]. \\
\text{atencion} &= \text{softmax}\left(\frac{q k^\top}{\sqrt{d_h}}\right) v.
\end{aligned}$$

3.3.2. Pseudocódigo Algorítmico

- 1: **Entrada:** x , número de sub-cabezas L , proyecciones por sub-cabeza $W_{UK}^{(i)}, W_{UV}^{(i)}$
- 2: $c_{KV} \leftarrow W_{DKV} x = [c_1, \dots, c_L]$ ▷ particionar latente en L bloques
- 3: **for** $i = 1, \dots, L$ (en paralelo entre L dispositivos) **do**
- 4: $k_i \leftarrow W_{UK}^{(i)} c_i, \quad v_i \leftarrow W_{UV}^{(i)} c_i$
- 5: **end for**
- 6: $k \leftarrow \text{concat}(k_1, \dots, k_L), \quad v \leftarrow \text{concat}(v_1, \dots, v_L)$
- 7: pesos $\leftarrow \text{softmax}(q k^\top / \sqrt{d_h})$
- 8: **Salida:** $Y = \text{pesos} \cdot v$

3.3.3. Características Clave

- **Complejidad de entrenamiento:** $\mathcal{O}(n^2 \cdot d)$ (atención completa sobre cabezas concatenadas).
- **Complejidad de inferencia:** $\mathcal{O}(n)$ por token, estado $\mathcal{O}(r_{kv} \cdot n)$ particionado entre L dispositivos.
- **Entrenable:** totalmente entrenable; L controla la granularidad de partición.
- **Fortalezas:** aceleración de decodificación de $2.8\times$; particionamiento limpio tensor-paralelo de 4 vías del caché; preserva el presupuesto latente total.
- **Limitaciones:** requiere que L divida a r_{kv} ; más matrices de proyección que MLA; all-reduce inter-dispositivo en la salida.

3.4. Atención Tucker

La Atención Tucker [24] unifica MHA, GQA y MLA bajo una única factorización estilo Tucker de los tensores de pesos de consulta/clave/valor con rangos independientes $q_{\text{rank}}, k_{\text{rank}}, v_{\text{rank}}$. Se recupera MHA cuando todos los rangos igualan el tamaño oculto, GQA cuando $k_{\text{rank}} = v_{\text{rank}} < d$, y MLA cuando $k_{\text{rank}} = v_{\text{rank}}$ es igual al rango latente. Como la factorización es una repas lineal de pesos, el método es totalmente compatible con FlashAttention y RoPE, y produce un orden de magnitud menos parámetros para métricas comparables.

3.4.1. Formulación Matemática

$$\mathbf{Q} = W_{Q,\text{core}} W_{Q,\text{factor}} x, \quad \mathbf{K} = W_{K,\text{core}} W_{K,\text{factor}} x, \quad \mathbf{V} = W_{V,\text{core}} W_{V,\text{factor}} x.$$

$$\text{atencion} = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_h}}\right) \mathbf{V}.$$

Casos especiales: MHA $\Leftrightarrow q_{\text{rank}} = k_{\text{rank}} = v_{\text{rank}} = d$; GQA $\Leftrightarrow k_{\text{rank}} = v_{\text{rank}} < d$; MLA $\Leftrightarrow k_{\text{rank}} = v_{\text{rank}} = r_{kv}$.

3.4.2. Pseudocódigo Algorítmico

- 1: **Entrada:** x , rangos $q_{\text{rank}}, k_{\text{rank}}, v_{\text{rank}}$, matrices core/factor
- 2: $\mathbf{Q} \leftarrow W_{Q,\text{core}} W_{Q,\text{factor}} x$ ▷ consulta factorizada de rango q_{rank}
- 3: $\mathbf{K} \leftarrow W_{K,\text{core}} W_{K,\text{factor}} x$ ▷ clave factorizada de rango k_{rank}
- 4: $\mathbf{V} \leftarrow W_{V,\text{core}} W_{V,\text{factor}} x$ ▷ valor factorizado de rango v_{rank}
- 5: **Aplicar RoPE a $\mathbf{Q}, \mathbf{K}$ si se usa codificación posicional**
- 6: pesos $\leftarrow \text{softmax}(\mathbf{Q}\mathbf{K}^\top / \sqrt{d_h})$ ▷ compatible con FlashAttention
- 7: **Salida:** $\mathbf{Y} = \text{pesos} \cdot \mathbf{V}$

3.4.3. Características Clave

- **Complejidad de entrenamiento:** $\mathcal{O}(n^2 \cdot d)$ sobre los tensores reconstruidos.
- **Complejidad de inferencia:** $\mathcal{O}(n)$ por token; tamaño de caché gobernado por k_{rank} .
- **Entrenable:** los rangos son hiperparámetros; la factorización es una repas de pesos directa.
- **Fortalezas:** unifica MHA/GQA/MLA; reducción de parámetros de un orden de magnitud con métricas equivalentes; compatible con FlashAttention/RoPE.
- **Limitaciones:** dos matmuls por proyección añaden latencia; la selección de rango es un problema de ajuste por tarea.

3.5. Atención de Cabezas Entrelazadas (IHA)

La Atención de Cabezas Entrelazadas (IHA, por sus siglas en inglés) [15] construye P pseudo-cabezas por cabeza original (típicamente $P = H$) donde cada consulta/clave/valor pseudo es una combinación lineal aprendida de todas las H cabezas originales. Esto induce hasta P^2 patrones de atención distintos por cabeza con una sobrecarga de $\mathcal{O}(H^2 P)$. Teóricamente, IHA necesita sólo $\Theta(\sqrt{k} n^2)$ parámetros frente a $\Theta(k n^2)$ de MHA en la tarea Polinomial; empíricamente entrega +10–20 % en RULER de recuperación multi-clave, +5,8 % en GSM8K y +2,8 % en MATH-500.

3.5.1. Formulación Matemática

$$q_{\text{pseudo}}[p] = \sum_{h=1}^H \text{mix}_q[p, h] q[h], \quad k_{\text{pseudo}}[p] = \sum_h \text{mix}_k[p, h] k[h], \quad v_{\text{pseudo}}[p] = \sum_h \text{mix}_v[p, h] v[h].$$

$$\text{atencion}_p = \text{softmax}\left(\frac{q_{\text{pseudo}}[p] k_{\text{pseudo}}[p]^\top}{\sqrt{d_h}}\right) v_{\text{pseudo}}[p], \quad \mathbf{Y}[h] = \frac{1}{P} \sum_p \text{atencion}_p[h].$$

3.5.2. Pseudocódigo Algorítmico

- 1: **Entrada:** por cabeza $q[h], k[h], v[h]$, matrices de mezcla $\text{mix}_q, \text{mix}_k, \text{mix}_v \in \mathbb{R}^{P \times H}$
- 2: **for** $p = 1, \dots, P$ **do**
- 3: $q_p \leftarrow \sum_h \text{mix}_q[p, h] q[h]$ ▷ mezcla aprendida entre cabezas
- 4: $k_p \leftarrow \sum_h \text{mix}_k[p, h] k[h], \quad v_p \leftarrow \sum_h \text{mix}_v[p, h] v[h]$
- 5: $\text{atencion}_p \leftarrow \text{softmax}(q_p k_p^\top / \sqrt{d_h}) v_p$
- 6: **end for**
- 7: **Salida:** $\mathbf{Y}[h] \leftarrow \frac{1}{P} \sum_p \text{atencion}_p[h]$ ▷ promediar salidas pseudo-cabeza por cabeza

3.5.3. Características Clave

- **Complejidad de entrenamiento:** $\mathcal{O}(n^2 \cdot H \cdot P)$ con $P \approx H$, es decir $\mathcal{O}(n^2 \cdot H^2)$.
- **Complejidad de inferencia:** $\mathcal{O}(n)$ por token con caché KV; sobrecarga extra de mezcla $\mathcal{O}(H^2 P)$.
- **Entrenable:** las matrices de mezcla se aprenden; P es un hiperparámetro.
- **Fortalezas:** eficiencia de parámetros $\Theta(\sqrt{k})$ en Polinomial; grandes ganancias en recuperación y razonamiento.
- **Limitaciones:** costo de mezcla cuadrático en cabezas; más patrones de atención a computar; latencia añadida con H grande.

3.6. Atención laTenT por Cabezas Agrupadas (GTA)

La Atención laTenT por Cabezas Agrupadas (GTA, por sus siglas en inglés) [48] combina dos compresiones: (1) un *mapa de atención compartido* — un tensor de puntajes de atención por grupo de cabezas, reutilizado entre todas las cabezas del grupo, reduciendo el caché de claves; y (2) un *decodificador de valores no lineal* — el caché de valores se comprime a un latente mediante una down-proyección y se reconstruye por una up-proyección con compuerta SiLU antes de la proyección de salida. GTA reduce los FLOPs de atención hasta 62.5% frente a GQA, encoge el caché KV hasta 70%, y produce una aceleración de inferencia de $2\times$ de extremo a extremo.

3.6.1. Formulación Matemática

$$q_g = \text{mean}(q[\text{grupo } g]), \quad k_g = \text{mean}(k[\text{grupo } g]), \quad \text{atencion}_g = \text{softmax}\left(\frac{q_g k_g^\top}{\sqrt{d_h}}\right) \quad (\text{reutilizado en el grupo}).$$

$$v_{\text{latente}} = W_{DV} x, \quad v = \text{SiLU}(W_{UV} v_{\text{latente}}), \quad \mathbf{Y} = \text{atencion}_g \cdot v.$$

3.6.2. Pseudocódigo Algorítmico

```

1: Entrada:  $q, k$  por cabeza, oculto  $x$ , partición de grupos  $\{g\}$ 
2:  $v_{\text{latente}} \leftarrow W_{DV} x$  ▷ comprimir valores a latente (en caché)
3:  $v \leftarrow \text{SiLU}(W_{UV} v_{\text{latente}})$  ▷ decodificación no lineal de valores
4: for cada grupo  $g$  do
5:    $q_g \leftarrow \text{mean}(q[g]), \quad k_g \leftarrow \text{mean}(k[g])$  ▷ consultas/claves compartidas por grupo
6:    $\text{atencion}_g \leftarrow \text{softmax}(q_g k_g^\top / \sqrt{d_h})$ 
7:    $\mathbf{Y}[g] \leftarrow \text{atencion}_g \cdot v[g]$  ▷ un mapa reutilizado para todas las cabezas en  $g$ 
8: end for
9: Salida:  $\mathbf{Y}$ 

```

3.6.3. Características Clave

- **Complejidad de entrenamiento:** $\mathcal{O}(n^2 \cdot G \cdot d_h)$ donde G es el número de grupos (frente a H cabezas en GQA).
- **Complejidad de inferencia:** $\mathcal{O}(n)$ por token; caché de valores $\mathcal{O}(r_v \cdot n)$, caché de claves reducido por compartición de grupo.
- **Entrenable:** la asignación de grupo y el rango latente son entrenables/hiperparámetros.
- **Fortalezas:** hasta 62.5 % de reducción de FLOPs de atención, 70 % de encogimiento de caché KV, 2× de aceleración de extremo a extremo.
- **Limitaciones:** el mapa compartido reduce la diversidad por cabeza; el decodificador no lineal añade latencia; el agrupamiento debe elegirse con cuidado.

3.7. Atención Latente Temporal Multi-Cabeza (MTLA)

La Atención Latente Temporal Multi-Cabeza (MTLA, por sus siglas en inglés) [13] extiende MLA a lo largo del eje temporal: una hyper-red fusiona dinámicamente m vectores temporalmente adyacentes del caché KV en un único slot, reduciendo la longitud temporal por un factor de m . Una máscara causal consciente del stride mantiene el entrenamiento paralelo consistente con la inferencia autorregresiva, produciendo una aceleración de decodificación de 5.3× y una reducción de memoria GPU de 8.3× frente a MHA en traducción de voz En-De.

3.7.1. Formulación Matemática

$$c_{KV}^{(t)} = W_{DKV} x^{(t)} \in \mathbb{R}^{r_{kv}} \text{ (latente por token).}$$

$$\bar{c}_{KV}^{(j)} = \sum_{i=0}^{m-1} \alpha_i c_{KV}^{(sj+i)}, \quad \alpha_i = \text{sigmoid}(\text{HyperNet}(c_{KV}^{(sj+i)})),$$

donde s es el stride y m la ventana de fusión. Las consultas atienden sobre slots fusionados $\bar{c}_{KV}$ con una máscara causal consciente del stride M_{stride} :

$$\text{atencion} = \text{softmax}\left(\frac{q \bar{k}^\top}{\sqrt{d_h}} + M_{\text{stride}}\right) \bar{v}, \quad \bar{k} = W_{UK} \bar{c}_{KV}, \quad \bar{v} = W_{UV} \bar{c}_{KV}.$$

3.7.2. Pseudocódigo Algorítmico

```

1: Entrada: latentes por token  $c_{KV}^{(t)}$ , ventana de fusión  $m$ , stride  $s$ 
2: Fase de fusión (hyper-red):
3: for cada slot fusionado  $j = 0, 1, \dots$  do
4:    $\alpha_i \leftarrow \text{sigmoid}(\text{HyperNet}(c_{KV}^{(sj+i)}))$  para  $i = 0, \dots, m-1$ 
5:    $\bar{c}_{KV}^{(j)} \leftarrow \sum_{i=0}^{m-1} \alpha_i c_{KV}^{(sj+i)}$   $\triangleright$  promedio con compuerta sobre la ventana
6: end for
7:  $\bar{k} \leftarrow W_{UK} \bar{c}_{KV}$ ,  $\bar{v} \leftarrow W_{UV} \bar{c}_{KV}$ 
8: pesos  $\leftarrow \text{softmax}(q \bar{k}^\top / \sqrt{d_h} + M_{\text{stride}})$   $\triangleright$  máscara causal consciente del stride
9: Salida:  $\mathbf{Y} = \text{pesos} \cdot \bar{v}$ 

```

3.7.3. Características Clave

- **Complejidad de entrenamiento:** $\mathcal{O}(n^2 \cdot d/m)$ sobre la secuencia fusionada (longitud $\lceil n/m \rceil$) más costo de la hyper-red.
- **Complejidad de inferencia:** $\mathcal{O}(n/m)$ por token sobre slots fusionados; estado $\mathcal{O}(r_{kv} \cdot n/m)$.
- **Entrenable:** la hyper-red y la ventana de fusión m se aprenden/eligen; la máscara consciente del stride garantiza consistencia.
- **Fortalezas:** aceleración de decodificación de $5.3\times$, reducción de memoria GPU de $8.3\times$ frente a MHA; compresión temporal ortogonal a la compresión de rango.
- **Limitaciones:** la ventana de fusión puede difuminar detalle temporal fino; la hyper-red añade cómputo; el diseño de la máscara no es trivial.

3.8. Comparación y Síntesis Arquitectónica

3.9. Atención Convolutiva Comprimida (CCA)

La Atención Convolutiva Comprimida (CCA, por sus siglas en inglés) [16] adopta un enfoque fundamentalmente diferente para la atención latente: en lugar de comprimir el caché KV y luego up-proyectar al ancho completo antes de atender (como MLA/GQLA/MLRA), CCA proyecta consultas, claves y valores a un latente compartido y realiza la operación de atención *completa* dentro de él. No hay matrices de up-proyección para Q/K/V — sólo una única up-proyección de salida W_O mapea la salida latente de vuelta al flujo residual. Esto reduce simultáneamente parámetros, caché KV, y FLOPs de atención por el factor de compresión $C = E/\tilde{e}$, mientras que MLA sólo reduce el caché.

Para hacer viable la atención en el latente comprimido, CCA introduce tres innovaciones: (1) dos convoluciones causales sobre el tensor empaquetado q/k (una convolución de secuencia depth-wise y una convolución agrupada de canales por cabeza), (2) q-k-mean (añade la media pre-convolución de q y k a los valores post-convolución), y (3) value-shift (cada cabeza recibe la mitad de sus valores del token actual y la mitad del token anterior mediante dos proyecciones independientes). Tras estos pasos, se aplica normalización L2 de QK y una temperatura de clave aprendible β , RoPE se aplica *directamente en el latente* (sin cabeza RoPE separada), y se computa la atención softmax estándar.

3.9.1. Formulación Matemática

$$\begin{aligned}
\tilde{q}_{\text{pre}} &= \tilde{W}_Q x, & \tilde{k}_{\text{pre}} &= \tilde{W}_K x, & \tilde{e} &= E/C. \\
\tilde{q} &= \text{Conv}(\tilde{q}_{\text{pre}}) + \frac{1}{2}(\tilde{q}_{\text{pre}} + \tilde{k}_{\text{pre}}), & \tilde{k} &= \text{Conv}(\tilde{k}_{\text{pre}}) + \frac{1}{2}(\tilde{q}_{\text{pre}} + \tilde{k}_{\text{pre}}).
\end{aligned}$$

$$\tilde{v} = [\tilde{W}_{V_1} x_t \parallel \tilde{W}_{V_2} x_{t-1}], \quad \hat{q} = \frac{\tilde{q}\sqrt{d_h}}{\|\tilde{q}\|}, \quad \hat{k} = \frac{\tilde{k}\sqrt{d_h}}{\|\tilde{k}\|} e^\beta.$$

$$\text{atencion} = \text{softmax}\left(\frac{\text{RoPE}(\hat{q}) \text{RoPE}(\hat{k})^\top}{\sqrt{d_h}}\right) \tilde{v}, \quad \mathbf{Y} = W_O \text{atencion}.$$

3.9.2. Características Clave

- **Complejidad de entrenamiento:** $\mathcal{O}(n^2 \cdot d/C)$ (atención en latente comprimido, $C \times$ menos FLOPs que MHA).
- **Complejidad de inferencia:** $\mathcal{O}(n)$ por token, espacio de caché KV $\mathcal{O}(\tilde{e} \cdot n)$.
- **Entrenable:** C , tamaños de kernel de conv, y los tres trucos (convs, qk-mean, value-shift) son configurables.
- **Fortalezas:** Reduce params, caché Y FLOPs simultáneamente; RoPE se integra nativamente; $>2\times$ menos params que MLA a igual compresión.
- **Limitaciones:** No elimina la cuadrática S^2 (sólo la divide por C); conv/qk-mean/v-shift añaden sesgo inductivo; se necesita kernel fusionado para la aceleración teórica.

3.10. Atención Convolutiva Comprimida por Consultas Agrupadas (CCG-QA)

La Atención Convolutiva Comprimida por Consultas Agrupadas (CCGQA, por sus siglas en inglés) [16] extiende CCA con compartición de cabezas clave/valor estilo GQA aplicada *dentro* del latente comprimido, y **desacopla** las tasas de compresión de query y KV: las consultas se comprimen por C_1 y las claves/valores por $C_2 \geq C_1$. La dimensión latente por cabeza d_h debe coincidir entre cabezas de query y clave, imponiendo la restricción $C_2/C_1 = n_h/n_{kv}$. Las cabezas KV se comparten por grupo (replicadas para coincidir con las cabezas de query), y el sesgo qk-mean usa operadores de broadcast/reducción B_{group} (replicar cabezas KV a cabezas query) y E_{group} (promediar cabezas query dentro de cada grupo). CCGQA logra la mejor pérdida reportada a $8\times$ reducción de caché KV sin pérdida de calidad frente a MHA.

3.10.1. Características Clave

- **Complejidad de entrenamiento:** $\mathcal{O}(n^2 \cdot d/C_1)$ (el término S^2 escala por $1/C_1$; los términos de proyección por $1/C_1 + 1/C_2$).
- **Complejidad de inferencia:** $\mathcal{O}(n)$ por token, espacio de caché KV $\mathcal{O}(\tilde{e}_{kv} \cdot n)$ (comprimido por C_2 , compartido por grupo).
- **Entrenable:** C_1 , C_2 , n_{kv} , y todos los trucos CCA son configurables. Restricción: $C_2/C_1 = n_h/n_{kv}$.
- **Fortalezas:** Compresión desacoplada permite frontera de Pareto suave; misma intensidad aritmética que GQA; mejor pérdida a $8\times$ reducción de caché.
- **Limitaciones:** Misma cuadrática S^2 (sólo reducida por C_1); se necesita kernel fusionado; dos proyecciones de valor añaden complejidad.

3.11. Comparación y Síntesis Arquitectónica

Arquitectura	Entrenar	Inferir	Estado	Característica Clave
MLA	$O(n^2d)$	$O(n)$ caché	$O(r_{kv}n)$	KV latente de rango $r_{kv} \approx d/2$, reduce a la mitad el caché, iguala calidad MHA.
GQLA	$O(n^2d)$	$O(n)$ caché	$O(r_{kv}n)$	Mismos pesos MLA, dos rutas de decodificación algebraicamente equivalentes adaptativas al hardware.
MLRA	$O(n^2d)$	$O(n)$ caché	$O(r_{kv}n)$ particionado	Latente dividido en L sub-cabezas para decodificación tensor-paralela de 4 vías, $2.8\times$ de aceleración.
Tucker	$O(n^2d)$	$O(n)$ caché	$O(k_{\text{rank}}n)$	Factorización Tucker unifica MHA/GQA/MLA, un orden de magnitud menos parámetros.
IHA	$O(n^2H^2)$	$O(n)$ caché	$O(Hnd_h)$	Pseudo-cabezas como mezclas aprendidas de cabezas, $\Theta(\sqrt{k})$ parámetros, +10–20 % RULER.
GTA	$O(n^2Gd_h)$	$O(n)$ caché	$O(r_vn)$	Mapa compartido por grupo + decodificador de valores SiLU, 62.5 % menos FLOPs, 70 % menos caché.
MTLA	$O(n^2d/m)$	$O(n/m)$	$O(r_{kv}n/m)$	Fusión temporal de m slots KV, $5.3\times$ decodificación, $8.3\times$ reducción de memoria.
CCA	$O(n^2d/C)$	$O(n)$ caché	$O(\tilde{d}n)$	Atención completa en latente comprimido, $C\times$ menos FLOPs + params + caché; convs + qk-mean + value-shift.
CCGQA	$O(n^2d/C_1)$	$O(n)$ caché	$O(\tilde{d}_{kv}n)$	CCA + GQA desacoplada en latente; compresión $C_2 \geq C_1$; mejor pérdida a $8\times$ reducción de caché.

La familia latente muestra una progresión clara a lo largo de dos ejes: (i) **compresión de rango**, pasando de cabezas completas (MHA) a un latente compartido (MLA) a sub-cabezas latentes particionadas (MLRA), y (ii) **adaptividad de despliegue**, pasando de una sola ruta de decodificación a despacho adaptativo al hardware (GQLA) y fusión temporal de caché (MTLA). La Atención Tucker proporciona el marco algebraico unificador: MHA, GQA y MLA son todos casos especiales de una factorización Tucker con elecciones de rango apropiadas. CCA y CCGQA toman un enfoque fundamentalmente diferente: en lugar de comprimir el caché KV y luego up-proyectar para atención a ancho completo (MLA/GQLA/MLRA), realizan atención *completamente dentro* del espacio latente comprimido, reduciendo no sólo el caché sino también los FLOPs de atención por el factor de compresión. CCGQA además desacopla las tasas de compresión de query y KV, logrando la mejor pérdida reportada a $8\times$ reducción de caché KV.

4. Mecanismos de Atención Dispersa Exhaustivos

4.1. Resumen Ejecutivo

Los mecanismos de atención dispersa abordan la prohibitiva complejidad computacional y de memoria $\mathcal{O}(n^2)$ de la atención de producto punto escalado estándar al restringir el conjunto

de pares clave-valor atendidos. La atención dispersa moderna abarca un rico espacio de diseño distinguido por cuatro dimensiones ortogonales: (i) **patrón de dispersidad** (geométrico fijo, dependiente de datos o basado en frecuencia), (ii) **capacidad de entrenamiento** (entrenado de extremo a extremo o solo inferencia), (iii) **unidad de dispersidad** (tokens individuales, ventanas locales o bloques), y (iv) **mecanismo** (enmascaramiento, selección, filtrado o compresión). Este anexo sintetiza siete familias principales de atención dispersa—Sparse Transformer, Longformer, BigBird, SparseK, NSA, SpargeAttn y FASA—proporcionando formulaciones matemáticas, pseudocódigo algorítmico y comparaciones arquitectónicas exhaustivas.

4.2. Sparse Transformer: Patrones Estrideados y Fijos Factorizados

4.2.1. Formulación Matemática

Sparse Transformer [11] reduce la complejidad cuadrática a $\mathcal{O}(n\sqrt{n})$ factorizando la atención dispersa en dos cabezas dispersas complementarias. Sea n la longitud de la secuencia y $l = \lfloor \sqrt{n} \rfloor$ el stride.

Cabeza de atención estrideada : Cada posición i atiende a cada l -ésima posición anterior:

$$\mathcal{A}_i^{(\text{estrideada})} = \{j : j \leq i, (i - j) \bmod l = 0\}$$

Cabeza de atención fija : Cada posición i atiende a posiciones dentro de su bloque local más columnas de resumen fijas:

$$\mathcal{A}_i^{(\text{fija})} = \{j : \lfloor j/l \rfloor = \lfloor i/l \rfloor\} \cup \{j : j \bmod l \in \{l - c, \dots, l - 1\}\}$$

donde c es un hiperparámetro que controla el número de columnas de resumen (típicamente $c = 1$ o $c = 2$).

El cálculo de atención para cada cabeza h sigue las reglas estándar de producto punto escalado restringido al conjunto disperso:

$$\text{Attn}_h(Q, K, V)_i = \text{softmax} \left(\frac{q_i^h K_{\mathcal{A}_i}^h \top}{\sqrt{d_k}} \right) V_{\mathcal{A}_i}^h$$

La idea clave es que con las dos cabezas factorizadas operando en paralelo a través de una profundidad de L capas transformer, cada posición puede alcanzar cualquier otra posición a través de un camino de longitud máxima $L + 1$, preservando la alcanzabilidad de largo alcance con una fracción del costo de la atención densa.

4.2.2. Pseudocódigo Algorítmico

Algorithm 23 Atención Sparse Transformer: Cabezas Duales Estrideada + Fija

Require: Consulta $\mathbf{Q} \in \mathbb{R}^{n \times d}$, Clave $\mathbf{K} \in \mathbb{R}^{n \times d}$, Valor $\mathbf{V} \in \mathbb{R}^{n \times d}$

Require: Stride $l = \lfloor \sqrt{n} \rfloor$, columnas de resumen $c \in \{1, 2\}$

Ensure: Salida $\mathbf{Y} \in \mathbb{R}^{n \times d}$

```

1: Cabeza 1: Atención Estrideada
2: for  $i = 1, \dots, n$  do
3:    $\mathcal{A}_i \leftarrow \{j : j \leq i \text{ y } (i - j) \bmod l = 0\}$  ▷ Cada  $l$ -ésima posición
4:    $\text{puntajes}_i \leftarrow \mathbf{q}_i \mathbf{K}_{\mathcal{A}_i}^\top / \sqrt{d_k}$ 
5:    $\text{attn}_i \leftarrow \text{softmax}(\text{puntajes}_i)$ 
6:    $\mathbf{y}_i^{(1)} \leftarrow \text{attn}_i \mathbf{V}_{\mathcal{A}_i}$ 
7: end for
8: Cabeza 2: Atención de Bloque Fijo + Resumen
9: for  $i = 1, \dots, n$  do
10:   $\text{inicio\_bloque} \leftarrow \lfloor i/l \rfloor \cdot l$ ,  $\text{fin\_bloque} \leftarrow \min(i + 1, \text{inicio\_bloque} + l)$ 
11:   $\mathcal{A}_i \leftarrow [\text{inicio\_bloque}, \text{fin\_bloque}) \cup \{\text{columnas de las últimas } c \text{ columnas}\}$ 
12:   $\text{puntajes}_i \leftarrow \mathbf{q}_i \mathbf{K}_{\mathcal{A}_i}^\top / \sqrt{d_k}$ 
13:   $\text{attn}_i \leftarrow \text{softmax}(\text{puntajes}_i)$ 
14:   $\mathbf{y}_i^{(2)} \leftarrow \text{attn}_i \mathbf{V}_{\mathcal{A}_i}$ 
15: end for
16: Concatenar:  $\mathbf{Y} = [\mathbf{y}^{(1)} \parallel \mathbf{y}^{(2)}]$  y proyectar mediante capa lineal de salida return  $\mathbf{Y}$ 

```

4.2.3. Características Clave

- **Complejidad:** $\mathcal{O}(n\sqrt{n} \cdot d)$ durante el entrenamiento; $\mathcal{O}(n)$ por token durante la generación.
- **Entrenable:** Sí; retropropagación completa a través de las posiciones seleccionadas.
- **Patrón de dispersidad:** Fijo e independiente de los datos; los patrones no se adaptan al contenido.
- **Fortaleza:** Alcanzabilidad estructurada que garantiza dependencias de largo alcance; demostrado efectivo en secuencias largas (Enwik8, imágenes de texto).
- **Limitación:** Los patrones fijos pueden pasar por alto relaciones importantes pero no contiguas; requiere kernels CUDA personalizados para eficiencia práctica.

4.3. Longformer: Ventana Deslizante, Dilatación y Tokens Globales

4.3.1. Formulación Matemática

Longformer [5] logra una complejidad lineal $\mathcal{O}(n \cdot w)$ combinando tres patrones de atención complementarios.

Atención de ventana deslizante : Vecindario local de tamaño w :

$$\mathcal{A}_i^{(\text{ventana})} = \{j : |i - j| \leq w/2\}$$

Ventana deslizante dilatada : Atención con ventana y dilatación d , saltando stride $d + 1$:

$$\mathcal{A}_i^{(\text{dilatada})} = \{j : |i - j| \leq (w/2)(d + 1) \text{ y } (i - j) \bmod (d + 1) = 0\}$$

Atención global : Tokens globales designados (ej., [CLS]) atienden a todas las posiciones y todos atienden a ellos:

$$\mathcal{A}_i^{(\text{global})} = \begin{cases} \{1, \dots, n\} & \text{si } i \in \mathcal{G} \\ \mathcal{A}_i^{(\text{ventana})} \cup \mathcal{G} & \text{en otro caso} \end{cases}$$

Los tres patrones se aplican mediante proyecciones separadas. La atención local y global pueden usar las mismas proyecciones o unas separadas específicas para cada tarea.

4.3.2. Pseudocódigo Algorítmico

Algorithm 24 Longformer: Ventana Deslizante + Dilatada + Global

Require: Consulta $\mathbf{Q} \in \mathbb{R}^{n \times d}$, Clave $\mathbf{K}$, Valor $\mathbf{V}$, tamaño de ventana w , dilatación d , índices de tokens globales $\mathcal{G}$

Ensure: Salida $\mathbf{Y}$

```

1: for  $i = 1, \dots, n$  do
2:   if  $i \in \mathcal{G}$  then
3:      $\mathcal{A}_i \leftarrow \{1, \dots, n\}$  ▷ Token global atiende a todos
4:   else
5:     Ventana local:  $\mathcal{A}^{(\text{local})} \leftarrow [i - w/2, i + w/2] \cap [1, n]$ 
6:     Desplazamientos dilatados:  $\mathcal{A}^{(\text{dilatada})} \leftarrow \{j : (i - j) \bmod (d + 1) = 0\} \cap \mathcal{A}^{(\text{rango dilatado})}$ 
7:     Combinar:  $\mathcal{A}_i \leftarrow \mathcal{A}^{(\text{local})} \cup \mathcal{A}^{(\text{dilatada})} \cup \mathcal{G}$ 
8:   end if
9:    $\text{puntajes}_i \leftarrow \mathbf{q}_i \mathbf{K}_{\mathcal{A}_i}^\top / \sqrt{d_k}$ 
10:   $\text{attn}_i \leftarrow \text{softmax}(\text{puntajes}_i)$ 
11:   $\mathbf{y}_i \leftarrow \text{attn}_i \mathbf{V}_{\mathcal{A}_i}$ 
12: end for return  $\mathbf{Y}$ 

```

4.3.3. Características Clave

- **Complejidad:** $\mathcal{O}(n \cdot w)$ donde w es el tamaño de ventana (lineal cuando $w \ll n$).
- **Entrenable:** Sí; reemplazo directo para la atención estándar.
- **Cobertura:** Con L capas y tamaño de ventana w , el campo receptivo crece a $L \times w$, cubriendo toda la secuencia con redes poco profundas.
- **Fortaleza:** Práctico para tareas a nivel de documentos; la dilatación expande los campos receptivos locales con sobrecarga mínima; los tokens globales proporcionan correspondencia consulta-documento.
- **Limitación:** El tamaño de ventana sigue siendo un hiperparámetro de diseño; subóptimo para tareas que requieren patrones de atención densos.

4.4. BigBird: Grafo Disperso Aleatorio, Local y Global

4.4.1. Formulación Matemática

BigBird [62] preserva la aproximación universal y la completitud de Turing utilizando una combinación fundamentada de tres patrones de dispersidad.

Conexiones aleatorias : Cada posición se conecta a r posiciones muestreadas aleatoriamente:

$$\mathcal{A}_i^{(\text{aleatoria})} = \text{MuestraAleatoria}(\{1, \dots, n\}, r)$$

Ventana local : Vecindario de ventana deslizante:

$$\mathcal{A}_i^{(\text{local})} = \{j : |i - j| \leq w/2\}$$

Tokens globales : Ancas globales específicas de la tarea:

$$\mathcal{A}_i^{(\text{global})} = \begin{cases} \{1, \dots, n\} & \text{si } i \in \mathcal{G} \\ \mathcal{A}_i^{(\text{aleatoria})} \cup \mathcal{A}_i^{(\text{local})} \cup \mathcal{G} & \text{en otro caso} \end{cases}$$

La máscara dispersa compuesta M es:

$$M_{ij} = \mathbf{1}[j \in \mathcal{A}_i^{(\text{aleatoria})} \cup \mathcal{A}_i^{(\text{local})} \cup \mathcal{A}_i^{(\text{global})}]$$

En la práctica, BigBird agrupa tokens en bloques de tamaño b y aplica la decisión de dispersidad a nivel de bloque, permitiendo implementaciones eficientes de bloques dispersos.

4.4.2. Garantía Teórica

Los autores demuestran que con $O(1)$ conexiones aleatorias y tokens globales, el grafo disperso mantiene las mismas propiedades de aproximación universal y completitud de Turing que la atención densa. Formalmente, cualquier función computable puede representarse y cualquier secuencia de operaciones puede simularse dentro del grafo de conectividad de BigBird.

4.4.3. Pseudocódigo Algorítmico

Algorithm 25 BigBird: Atención Dispersa por Bloques Aleatoria + Local + Global

Require: Consulta $\mathbf{Q}$, Clave $\mathbf{K}$, Valor $\mathbf{V}$, tamaño de bloque b , enlaces aleatorios por bloque r , índices globales $\mathcal{G}$

Ensure: Salida $\mathbf{Y}$

```

1: Reformar en bloques:  $n_b = \lceil n/b \rceil$  bloques
2: for bloque  $i = 0, \dots, n_b - 1$  do
3:   for posición  $j$  en bloque  $i$  do
4:     Ventana local:  $\mathcal{A}_j^{(\text{local})} =$  posiciones cercanas en el bloque  $\cup$  posiciones frontera
5:     Muestreo aleatorio de bloques: Muestrear  $r$  bloques uniformemente al azar, agregar todas las posiciones de esos bloques
6:     Global: Agregar todas las posiciones de tokens globales
7:      $\mathcal{A}_j \leftarrow \mathcal{A}_j^{(\text{local})} \cup \mathcal{A}_j^{(\text{aleatoria})} \cup \mathcal{G}$ 
8:   end for
9: end for
10: for  $i = 1, \dots, n$  do
11:    $\text{puntajes}_i \leftarrow \mathbf{q}_i \mathbf{K}_{\mathcal{A}_i}^\top / \sqrt{d_k}$ 
12:    $\text{attn}_i \leftarrow \text{softmax}(\text{puntajes}_i)$ 
13:    $\mathbf{y}_i \leftarrow \text{attn}_i \mathbf{V}_{\mathcal{A}_i}$ 
14: end for return  $\mathbf{Y}$ 
```

4.4.4. Características Clave

- **Complejidad:** $\mathcal{O}(n)$ o casi lineal en implementación óptima de bloques dispersos.
- **Entrenable:** Sí.
- **Base teórica:** Mantiene demostrablemente la aproximación universal y la completitud de Turing con conectividad dispersa.

- **Fortaleza:** Rendimiento sólido en documentos largos para preguntas-respuestas, resúmenes y tareas de genómica.
- **Limitación:** El muestreo aleatorio introduce varianza y no determinismo; el ajuste de los hiperparámetros r, w, g sigue dependiendo de la tarea.

4.5. Atención SparseK: Selección Diferencial Top- k

4.5.1. Formulación Matemática

SparseK [33] permite dispersidad aprendible mediante un **operador top- k diferenciable**. Una red de puntuación ϕ_θ evalúa la importancia de cada par clave-valor:

$$u_j = \phi_\theta(\mathbf{k}_j, \mathbf{q}_i) \in \mathbb{R}$$

El **operador SparseK** selecciona los puntajes top- k manteniéndose diferenciable. Calcula un umbral $\tau(u)$ tal que la suma de los puntajes activos sea igual a k :

$$\text{SparseK}(u, k)_j = \max(u_j - \tau(u), 0), \quad \text{donde} \quad \sum_j \max(u_j - \tau, 0) = k$$

El umbral τ se encuentra mediante bisección. La atención se convierte en:

$$m_j = \text{SparseK}(u, k)_j, \quad \text{Attn}_i = \text{softmax} \left(\frac{\mathbf{q}_i K_{\text{sel}}^\top}{\sqrt{d_k}} \right) V_{\text{sel}}$$

donde $K_{\text{sel}}, V_{\text{sel}}$ contienen solo las entradas top- k (aquellas con $m_j > 0$).

4.5.2. Pseudocódigo Algorítmico

Algorithm 26 SparseK: Atención Top- k Diferenciable

Require: Consulta $\mathbf{q} \in \mathbb{R}^d$, Matriz de clave $\mathbf{K} \in \mathbb{R}^{n \times d}$, Matriz de valor $\mathbf{V} \in \mathbb{R}^{n \times d}$

Require: Red de puntuación ϕ_θ , dispersidad objetivo k

Ensure: Salida de atención $\mathbf{y}$

```

1: Calcular puntajes de importancia:
2: for  $j = 1, \dots, n$  do
3:    $u_j \leftarrow \phi_\theta(\mathbf{k}_j, \mathbf{q})$ 
4: end for
5: Calcular top- $k$  diferenciable mediante umbral:
6:  $u_{\text{ordenado}} \leftarrow \text{ordenar}(u, \text{descendente} = \text{Verdadero})$ 
7:  $\text{suma\_acum} \leftarrow \text{suma\_acumulada}(u_{\text{ordenado}})$ 
8: Encontrar  $\rho = \max\{i : u_{\text{ordenado}}[i] > 0 \text{ y } \text{suma\_acum}[i] \leq k\}$ 
9:  $\tau \leftarrow (u_{\text{ordenado}}[\rho] - k) / (\rho + 1)$  ▷ Umbral para  $k$  elementos activos
10:  $m \leftarrow \max(u - \tau, 0)$  ▷ Máscara de selección diferenciable
11: Aplicar máscara y calcular atención:
12:  $K_{\text{sel}} \leftarrow K[m > 0], V_{\text{sel}} \leftarrow V[m > 0]$ 
13:  $\text{puntajes} \leftarrow \mathbf{q} K_{\text{sel}}^\top / \sqrt{d_k}$ 
14:  $\text{attn} \leftarrow \text{softmax}(\text{puntajes})$ 
15:  $\mathbf{y} \leftarrow \text{attn} \cdot V_{\text{sel}}$  return  $\mathbf{y}$ 

```

4.5.3. Características Clave

- **Complejidad:** $\mathcal{O}(n \cdot d)$ durante el entrenamiento (lineal en la longitud de la secuencia); $\mathcal{O}(k)$ por token durante la generación.

- **Entrenable:** Sí; flujo de gradiente de extremo a extremo a través del operador SparseK.
- **Generación incremental:** Soporta generación autorregresiva eficiente con memoria constante.
- **Fortaleza:** Se integra perfectamente en arquitecturas LLM existentes; mínimo ajuste fino necesario.
- **Limitación:** El acceso a memoria disperso de la selección top- k puede limitar la eficiencia de caché en algunos hardwares; la red de puntuación añade sobrecarga.

4.6. NSA (Native Sparse Attention): Ramas Jerárquicas Alineadas con el Hardware

4.6.1. Formulación Matemática

NSA [60] descompone la atención dispersa en tres ramas paralelas que se combinan mediante compuertas aprendidas.

Rama de compresión : Un MLP aprendible φ comprime bloques de clave-valor:

$$\tilde{K}_t^{\text{cmp}} = [\varphi(k_{id+1:id+l})]_{1 \leq i \leq \lfloor (t-l)/d \rfloor}, \quad \tilde{V}_t^{\text{cmp}} = [\varphi(v_{id+1:id+l})]$$

Rama de selección : Los bloques de alta importancia se seleccionan basándose en puntajes comprimidos:

$$p_t^{\text{cmp}} = \text{softmax}(q_t^\top \tilde{K}_t^{\text{cmp}} / \sqrt{d}), \quad I_t = \text{TopK}(p_t^{\text{cmp}}, n), \quad \tilde{K}_t^{\text{sel}} = \text{Recolectar}(K, I_t)$$

Rama de ventana deslizante : Contexto local fijo:

$$\tilde{K}_t^{\text{win}} = K_{t-w:t}, \quad \tilde{V}_t^{\text{win}} = V_{t-w:t}$$

Combinación con compuertas : Las tres salidas de atención se combinan mediante compuertas aprendidas:

$$\alpha_t^{(\text{cmp})} = \sigma(\text{Lineal}_{\text{cmp}}(q_t)), \quad \alpha_t^{(\text{sel})} = \sigma(\text{Lineal}_{\text{sel}}(q_t)), \quad \alpha_t^{(\text{win})} = \sigma(\text{Lineal}_{\text{win}}(q_t))$$

$$y_t = \alpha_t^{(\text{cmp})} \cdot \text{Attn}(q_t, \tilde{K}^{\text{cmp}}, \tilde{V}^{\text{cmp}}) + \alpha_t^{(\text{sel})} \cdot \text{Attn}(q_t, \tilde{K}^{\text{sel}}, \tilde{V}^{\text{sel}}) + \alpha_t^{(\text{win})} \cdot \text{Attn}(q_t, \tilde{K}^{\text{win}}, \tilde{V}^{\text{win}})$$

4.6.2. Pseudocódigo Algorítmico

Algorithm 27 NSA: Ramas Comprimida + Seleccionada + Ventana con Compuertas Aprendidas

Require: Consulta $\mathbf{q}_t$, secuencias de Clave/Valor almacenadas en caché, tamaño de bloque l , stride d , conteo de selección n , ventana w

Require: MLP de compresión φ , redes de compuerta $\text{Lineal}_{\text{cmp}}$, $\text{Lineal}_{\text{sel}}$, $\text{Lineal}_{\text{win}}$

Ensure: Salida $\mathbf{y}_t$

```

1: Rama de compresión:
2: for  $i = 1$  to  $\lfloor t/d \rfloor$  do
3:    $k_i^{\text{cmp}} \leftarrow \varphi(\mathbf{K}_{id+1:id+l})$  ▷ Comprimir bloque mediante MLP
4:    $v_i^{\text{cmp}} \leftarrow \varphi(\mathbf{V}_{id+1:id+l})$ 
5: end for
6:  $\tilde{\mathbf{K}}^{\text{cmp}} \leftarrow [k_1^{\text{cmp}}, \dots, k_{\lfloor t/d \rfloor}^{\text{cmp}}]$ ,  $\tilde{\mathbf{V}}^{\text{cmp}} \leftarrow [v_1^{\text{cmp}}, \dots, v_{\lfloor t/d \rfloor}^{\text{cmp}}]$ 
7:  $\mathbf{y}_t^{(\text{cmp})} \leftarrow \text{SDPA}(\mathbf{q}_t, \tilde{\mathbf{K}}^{\text{cmp}}, \tilde{\mathbf{V}}^{\text{cmp}})$ 
8: Rama de selección:
9: Calcular puntajes sobre comprimidos:  $\mathbf{s} \leftarrow \text{softmax}(\mathbf{q}_t \tilde{\mathbf{K}}^{\text{cmp}\top} / \sqrt{d})$ 
10: Seleccionar los  $n$  bloques más importantes:  $I \leftarrow \text{TopK}(\mathbf{s}, n)$ 
11: Recolectar bloques completos:  $\tilde{\mathbf{K}}^{\text{sel}} \leftarrow [\mathbf{K}_{I_i d+1:I_i d+l}]_i$ ,  $\tilde{\mathbf{V}}^{\text{sel}} \leftarrow [\mathbf{V}_{I_i d+1:I_i d+l}]_i$ 
12:  $\mathbf{y}_t^{(\text{sel})} \leftarrow \text{SDPA}(\mathbf{q}_t, \tilde{\mathbf{K}}^{\text{sel}}, \tilde{\mathbf{V}}^{\text{sel}})$ 
13: Rama de ventana:
14:  $\mathbf{y}_t^{(\text{win})} \leftarrow \text{SDPA}(\mathbf{q}_t, \mathbf{K}_{t-w:t}, \mathbf{V}_{t-w:t})$ 
15: Fusión con compuertas:
16:  $\alpha^{(\text{cmp})} \leftarrow \sigma(\text{Lineal}_{\text{cmp}}(\mathbf{q}_t))$ ,  $\alpha^{(\text{sel})} \leftarrow \sigma(\text{Lineal}_{\text{sel}}(\mathbf{q}_t))$ ,  $\alpha^{(\text{win})} \leftarrow \sigma(\text{Lineal}_{\text{win}}(\mathbf{q}_t))$ 
17:  $\mathbf{y}_t \leftarrow \alpha^{(\text{cmp})} \mathbf{y}_t^{(\text{cmp})} + \alpha^{(\text{sel})} \mathbf{y}_t^{(\text{sel})} + \alpha^{(\text{win})} \mathbf{y}_t^{(\text{win})}$  return  $\mathbf{y}_t$ 

```

4.6.3. Características Clave

- **Complejidad:** $\mathcal{O}(t/d + n \cdot l + w)$ tokens atendidos por paso (significativamente reducido frente a $\mathcal{O}(t)$ para densa).
- **Entrenable:** Sí; entrenamiento directo con todas las ramas diferenciables.
- **Alineación con hardware:** Diseñado para utilización eficiente de núcleos tensoriales; la compresión y selección reducen la presión sobre el ancho de banda de memoria.
- **Fortaleza:** $11.6\times$ de aceleración en decodificación y $9\times$ de aceleración hacia adelante en secuencias de 64k; supera a la atención completa en muchas tareas.
- **Limitación:** Requiere kernels Triton/CUDA personalizados; la arquitectura multi-rama compleja aumenta la sobrecarga de ingeniería.

4.7. FASA: Atención Dispersa Consciente de la Frecuencia

4.7.1. Formulación Matemática

FASA [54] es un **método en tiempo de inferencia sin entrenamiento** que explota la estructura del embedding de posición rotatorio (RoPE). Bajo RoPE, el embedding del token se rota por $\theta_i = B^{-2(i-1)/d}$, descomponiendo el espacio d -dimensional en $d/2$ fragmentos de frecuencia (FC).

Idea clave : Solo un subconjunto pequeño ($< 1\%$) de los FC importa para la conciencia contextual; la mayoría codifica patrones posicionales.

Identificación de fragmento de frecuencia dominante : Para cada capa l y cabeza h , identificar FC dominantes mediante concordancia contextual (CA):

$$\text{CA}^{l,h,i} = \frac{|\text{TopK}(\alpha^{l,h}) \cap \text{TopK}(\alpha^{l,h,i})|}{K}$$

donde $\alpha^{l,h}$ es la máscara de atención completa y $\alpha^{l,h,i}$ es la máscara usando solo el FC i . Los FC dominantes tienen CA alta—contribuyen significativamente al patrón de atención final.

Eta de predicción de importancia de tokens (TIP) : Usando solo FC dominantes, calcular importancia ligera por token:

$$S_t^{l,h} = \sum_{i \in \mathcal{I}_{\text{dom}}^{l,h}} \alpha^{l,h,i}(\mathbf{q}_t, \mathbf{K}_{1:t}), \quad \mathcal{T}_t = \text{TopK-Índices}(S_t, N_{\text{fac}})$$

Eta de cálculo de atención enfocada (FAC) : Calcular atención de precisión completa en los tokens seleccionados:

$$\hat{\alpha}_{\text{FAC}} = \text{softmax} \left(\frac{\mathbf{q}_t \mathbf{K}_{\mathcal{T}_t}^\top}{\sqrt{d}} \right), \quad \mathbf{o}_t = \hat{\alpha}_{\text{FAC}} \mathbf{V}_{\mathcal{T}_t}$$

4.7.2. Pseudocódigo Algorítmico

Algorithm 28 FASA: Atención Dispersa Consciente de la Frecuencia (en Tiempo de Inferencia)

Require: Consulta actual $\mathbf{q}_t$, Clave/Valor almacenados en caché $\mathbf{K}_{1:t}, \mathbf{V}_{1:t}$, base RoPE B , FC dominantes $\mathcal{I}_{\text{dom}}$

Require: Presupuesto TIP N_{tip} , presupuesto FAC N_{fac}

Ensure: Salida $\mathbf{o}_t$

```

1: Eta 1: Predicción de Importancia de Tokens (TIP)
2: for capa  $l$  y cabeza  $h$  do
3:   for posición  $i = 1$  to  $t$  do
4:     Calcular importancia desde FC dominantes:  $s_i \leftarrow 0$ 
5:     for  $fc \in \mathcal{I}_{\text{dom}}^{l,h}$  do
6:       Rotar consulta y clave en FC  $fc$ :  $\mathbf{q}_i^{(fc)}, \mathbf{k}_i^{(fc)}$ 
7:        $s_i^{(fc)} \leftarrow \text{softmax}(\mathbf{q}_i^{(fc)} \mathbf{k}_i^{(fc)\top} / \sqrt{d})$ 
8:        $s_i \leftarrow s_i + s_i^{(fc)}$ 
9:     end for
10:  end for
11:   $\mathcal{T}_t \leftarrow \text{TopK-ndices}([s_1, \dots, s_t], N_{\text{fac}})$  ▷ Seleccionar tokens principales
12: end for
13: Eta 2: Cálculo de Atención Enfocada (FAC)
14:  $\text{puntajes}_{\text{fac}} \leftarrow \mathbf{q}_t \mathbf{K}_{\mathcal{T}_t}^\top / \sqrt{d}$  ▷ Producto punto de precisión completa
15:  $\alpha_{\text{fac}} \leftarrow \text{softmax}(\text{puntajes}_{\text{fac}})$  ▷ Softmax completo
16:  $\mathbf{o}_t \leftarrow \alpha_{\text{fac}} \mathbf{V}_{\mathcal{T}_t}$  ▷ Suma ponderada de valores return  $\mathbf{o}_t$ 

```

4.7.3. Características Clave

- **Complejidad:** TIP es $\mathcal{O}(t \cdot N_{\text{tip}})$; FAC es $\mathcal{O}(N_{\text{fac}} \cdot d)$.
- **Entrenable:** No; optimización de inferencia sin entrenamiento.

- **Aplicabilidad:** Requiere modelos basados en RoPE; extendido a ALiBi y MLA con modificaciones.
- **Fortaleza:** Precisión cercana a la óptima con ≤ 256 tokens de entre millones; hasta $2.56\times$ de aceleración en decodificación; ortogonal a la cuantización.
- **Limitación:** Requiere calibración fuera de línea; la identificación de FC dominantes añade sobrecarga de preprocesamiento.

4.8. SpargeAttn: Filtrado de Dos Etapas a Nivel de Bloques

4.8.1. Formulación Matemática

SpargeAttn [65] es un **método universal sin entrenamiento** que predice qué bloques de la matriz de atención contendrán valores insignificantes y omite el cálculo para esos bloques.

Etapas 1 — Predicción dispersa : Calcular importancia proxy de bajo costo $\hat{s}_{ij}$ para el bloque (i, j) :

$$\hat{s}_{ij} = f_{\text{pred}}(\mathbf{Q}_i, \mathbf{K}_j; \text{hiperparámetros}), \quad \text{omitir si } \hat{s}_{ij} < \epsilon_1$$

Los predictores comunes incluyen similitud de media de bloque o estadísticas de autosimilitud.

Etapas 2 — Filtrado consciente de softmax : Después de calcular $\tilde{P}_{ij} = \text{softmax}(Q_i K_j^\top / \sqrt{d})$, verificar si la probabilidad máxima del bloque es insignificante en relación con el máximo de softmax en ejecución:

$$\text{omitir } P_{ij} V_j \quad \text{si} \quad \max(\tilde{P}_{ij}) < e^{m_{\text{antiguo}} - m_{\text{nuevo}}} \cdot \epsilon_2$$

donde $m_{\text{antiguo}}, m_{\text{nuevo}}$ son máximos del softmax en línea en ejecución (calculados mediante numéricos estilo FlashAttention).

4.8.2. Pseudocódigo Algorítmico

Algorithm 29 SparseAttn: Filtrado Disperso de Dos Etapas por Bloques

Require: Bloques de consulta $\mathbf{Q}_i$ para $i = 1, \dots, n_b$, Bloques de clave $\mathbf{K}_j$ para $j = 1, \dots, n_b$, Bloques de valor $\mathbf{V}_j$

Require: Umbral de predicción ϵ_1 , umbral de softmax ϵ_2 , tamaño de bloque b_s

Ensure: Salida $\mathbf{Y}$

```

1: Inicializar: máximo de softmax en línea  $m_{\text{global}} \leftarrow -\infty$ 
2: for fila de bloque  $i = 1$  to  $n_b$  do
3:   Inicializar: salida de fila de bloque  $\mathbf{Y}_i \leftarrow 0$ , máximo local  $m_i \leftarrow -\infty$ 
4:   for columna de bloque  $j = 1$  to  $n_b$  do
5:     Etapa 1: Predicción Dispersa
6:     Calcular importancia proxy:  $\hat{s}_{ij} \leftarrow f_{\text{pred}}(\mathbf{Q}_i, \mathbf{K}_j)$ 
7:     if  $\hat{s}_{ij} < \epsilon_1$  then
8:       omitir este bloque  $[i, j]$  ▷ Terminación temprana
9:       continuar ▷ Pasar al siguiente bloque
10:    end if
11:    Etapa 2: Calcular y Filtrar
12:     $\tilde{P}_{ij} \leftarrow \text{softmax}(\mathbf{Q}_i \mathbf{K}_j^\top / \sqrt{d})$ 
13:     $m_{\text{bloque}} \leftarrow \text{máx}(\tilde{P}_{ij})$ 
14:    if  $m_{\text{bloque}} < e^{m_i - m_{\text{global}}} \cdot \epsilon_2$  then
15:      El bloque contribuye insignificamente; omitir ▷ Poda consciente de softmax
16:      continuar
17:    end if
18:    Actualizar máximo global:  $m_i \leftarrow \text{máx}(m_i, m_{\text{bloque}})$ 
19:    Acumular:  $\mathbf{Y}_i \leftarrow \mathbf{Y}_i + \tilde{P}_{ij} \mathbf{V}_j$ 
20:  end for
21:   $m_{\text{global}} \leftarrow \text{máx}(m_{\text{global}}, m_i)$ 
22: end for return  $\mathbf{Y}$ 

```

4.8.3. Características Clave

- **Complejidad:** Empírica $\mathcal{O}(n^2 \cdot s)$ donde s es la fracción de bloques no omitidos (típicamente 0.2–0.5).
- **Entrenable:** No; aceleración plug-and-play para modelos existentes.
- **Universalidad:** Funciona en modelos de lenguaje, modelos de difusión de imágenes y generación de video.
- **Fortaleza:** 2.5–5× de aceleración comparado con métodos densos o dispersos anteriores; compatible con cuantización (integración SageAttention).
- **Limitación:** La aceleración depende de la dispersidad inherente; la granularidad a nivel de bloque puede pasar por alto patrones más finos; se requiere ajuste de umbral por familia de modelos.

4.9. MSA (MiniMax Sparse Attention): Atención Dispersa por Bloques con Rama de Índice Ligera

4.9.1. Formulación Matemática

MSA [25] es un mecanismo de atención dispersa por bloques construido sobre atención agrupada por consultas (GQA) que combina una **Rama de Índice** ligera para la selección de bloques

con una **Rama Principal** para atención softmax dispersa por bloques exacta sobre los bloques seleccionados. Sea N la longitud de la secuencia, $B_k = 128$ el tamaño de bloque, y $k = 16$ el número de bloques seleccionados por consulta (lo que yield un presupuesto fijo por consulta de $k \cdot B_k = 2048$ tokens independientemente de N).

Rama de Índice : Una proyección de baja dimensionalidad produce consultas de índice $\mathbf{q}_t^{\text{idx}} \in \mathbb{R}^{d_{\text{idx}}}$ y representantes de bloque $\mathbf{r}_b \in \mathbb{R}^{d_{\text{idx}}}$ para cada bloque KV b . Los puntajes de índice por grupo GQA son:

$$s_{t,b}^{(g)} = \mathbf{q}_t^{\text{idx},(g)} \cdot \mathbf{r}_b, \quad \mathcal{I}_t^{(g)} = \text{TopK} \left(\{s_{t,b}^{(g)}\}_{b=1}^{N/B_k}, k \right)$$

La entrada del índice está **desconectada** (stop-gradient), de modo que los gradientes no fluyen hacia atrás a través de la entrada del indexador; la Rama de Índice se entrena en su lugar mediante una pérdida de alineación KL entre su distribución de atención y la distribución de la Rama Principal. Debido a que softmax preserva el orden, los puntajes crudos de índice se alimentan directamente al operador top- k *sin* un softmax explícito (**top- k sin exponencial**), reduciendo el costo numérico.

Bloque local forzado : El bloque que contiene la posición de consulta t **siempre** se incluye en $\mathcal{I}_t^{(g)}$, garantizando cobertura de contexto local.

Rama Principal : La atención estándar de producto punto escalado se calcula exactamente sobre la unión de los bloques seleccionados:

$$\mathbf{o}_t^{(h)} = \text{softmax} \left(\frac{\mathbf{q}_t^{(h)} \mathbf{K}_{\mathcal{I}_t}^{(h)\top}}{\sqrt{d_h}} \right) \mathbf{V}_{\mathcal{I}_t}^{(h)}$$

donde $\mathbf{K}_{\mathcal{I}_t}, \mathbf{V}_{\mathcal{I}_t}$ reúnen los tensores de clave/valor de precisión completa de los bloques seleccionados. La selección se realiza independientemente por grupo GQA, de modo que diferentes grupos de consulta dentro de la misma capa pueden atender a diferentes subconjuntos de bloques.

4.9.2. Pseudocódigo Algorítmico

Algorithm 30 MSA: Atención Dispersa MiniMax con Ramas de Índice + Principal

Require: Consulta $\mathbf{Q} \in \mathbb{R}^{N \times d}$, Clave $\mathbf{K}$, Valor $\mathbf{V}$, tamaño de bloque $B_k = 128$, bloques top- k $k = 16$

Require: Proyección de índice W_{idx} (entrada desconectada), representantes de bloque $\{\mathbf{r}_b\}$

Ensure: Salida $\mathbf{Y} \in \mathbb{R}^{N \times d}$

- 1: Calcular representantes de bloque: $\mathbf{r}_b \leftarrow \text{mean}(\mathbf{K}_{bB_k:(b+1)B_k})$ para $b = 1, \dots, N/B_k$
 - 2: **for** posición de consulta $t = 1, \dots, N$ **do**
 - 3: **for** cada grupo GQA g **do**
 - 4: $\mathbf{q}_t^{\text{idx}} \leftarrow \text{sg}(W_{\text{idx}} \mathbf{x}_t)$ ▷ consulta de índice desconectada
 - 5: Calcular puntajes de bloque: $s_b \leftarrow \mathbf{q}_t^{\text{idx}} \cdot \mathbf{r}_b$
 - 6: Identificar bloque local: $b_{\text{local}} \leftarrow \lfloor t/B_k \rfloor$
 - 7: $\mathcal{I}_t^{(g)} \leftarrow \text{TopK}(\{s_b\}, k) \cup \{b_{\text{local}}\}$ ▷ top- k sin exp + local forzado
 - 8: **end for**
 - 9: Reunir KV seleccionado: $\tilde{\mathbf{K}}_t \leftarrow \text{Gather}(\mathbf{K}, \mathcal{I}_t)$, $\tilde{\mathbf{V}}_t \leftarrow \text{Gather}(\mathbf{V}, \mathcal{I}_t)$
 - 10: Rama principal SDPA: $\mathbf{y}_t \leftarrow \text{softmax}(\mathbf{q}_t \tilde{\mathbf{K}}_t^\top / \sqrt{d_h}) \tilde{\mathbf{V}}_t$
 - 11: **end for**
 - 12: Agregar pérdida de alineación KL $\mathcal{L}_{\text{KL}} = \text{KL}(\text{dist índice} \parallel \text{dist principal})$ durante el entrenamiento **return** $\mathbf{Y}$
-

4.9.3. Características Clave

- **Complejidad:** Rama de índice $\mathcal{O}(H_{kv} \cdot d_{idx} \cdot N^2)$; Rama principal $\mathcal{O}(H_q \cdot d_h \cdot N \cdot k \cdot B_k)$. Presupuesto por consulta fijo en $k \cdot B_k = 2048$ tokens independientemente de N .
- **Entrenable:** Sí; de extremo a extremo (la Rama de Índice se entrena mediante una pérdida de alineación KL, no es solo inferencia).
- **Estado:** Caché KV $\mathcal{O}(N)$ (tamaño GQA); sin expansión de la dimensión de estado recurrente.
- **Fortaleza:** Desplegado en el modelo 109B MiniMax-M3, MSA logra una reducción de cómputo de atención por token de $28,4\times$ a contexto de 1M, con $14,2\times$ en prefill y $7,6\times$ en decodificación en GPUs H800.
- **Limitación:** La granularidad de selección es fija a nivel de bloque ($B_k = 128$); el bloque local forzado y la selección independiente por grupo aumentan la complejidad de ingeniería y requieren kernels personalizados de bloques dispersos.

4.10. SparDA (Atención Dispersa Desacoplada): Selección de Bloques Anticipada Guiada por Pronóstico

4.10.1. Formulación Matemática

SparDA [17] introduce un esquema de atención dispersa **desacoplado** que aumenta las proyecciones estándar por capa Q, K, V con una cuarta proyección, el **Pronóstico**, que predice los bloques KV que serán necesarios en la *siguiente* capa. Esta anticipación permite solapar el prefetch CPU-a-GPU de los bloques seleccionados de la siguiente capa con la ejecución de la capa actual, ocultando la latencia de offload que afecta a las implementaciones de atención dispersa en hardware con memoria limitada.

Proyección de Pronóstico : Para la entrada $\mathbf{x} \in \mathbb{R}^{N \times d}$,

$$\mathbf{F} = \mathbf{x} \mathbf{W}_F \in \mathbb{R}^{N \times H_{kv} \times d_f}$$

con **una cabeza de Pronóstico por grupo GQA**, lo que reduce la sobrecarga de selección frente a los selectores multi-cabeza usados en trabajos anteriores.

Puntuación y selección de bloques : Para cada consulta i y bloque KV b con representante $\mathbf{r}_b$:

$$S_{i,b} = \mathbf{F}_i \cdot \mathbf{r}_b, \quad \mathcal{I}_i = \text{TopK}(\{S_{i,b}\}_b, k)$$

Atención dispersa por bloques : La atención estándar de producto punto escalado se calcula entonces sobre los bloques seleccionados:

$$\mathbf{o}_i = \text{softmax} \left(\frac{\mathbf{q}_i \mathbf{K}_{\mathcal{I}_i}^\top}{\sqrt{d_h}} \right) \mathbf{V}_{\mathcal{I}_i}$$

Entrenamiento : SparDA añade menos del 0,5 % de parámetros adicionales. Solo las proyecciones de Pronóstico se entrenan, emparejando la distribución de atención del selector disperso original (congelado)—una destilación ligera que preserva el comportamiento del modelo base mientras habilita el prefetch anticipado.

4.10.2. Pseudocódigo Algorítmico

Algorithm 31 SparDA: Atención Dispersa Desacoplada con Prefetch de Pronóstico

Require: Entrada $\mathbf{x} \in \mathbb{R}^{N \times d}$, bloques de Clave/Valor en caché, bloques top- k k

Require: Proyecciones W_q, W_k, W_v, W_F ; selector base congelado π_{orig}

Ensure: Salida $\mathbf{Y}$; bloques prefetched para la siguiente capa

- 1: Calcular Pronóstico: $\mathbf{F} \leftarrow \mathbf{x} \mathbf{W}_F$ $\triangleright$ una cabeza por grupo GQA
 - 2: **for** consulta $i = 1, \dots, N$ **do**
 - 3: Calcular puntajes de bloque: $S_{i,b} \leftarrow \mathbf{F}_i \cdot \mathbf{r}_b$ para cada bloque b
 - 4: Seleccionar: $\mathcal{I}_i \leftarrow \text{TopK}(\{S_{i,b}\}, k)$
 - 5: **Prefetch** $\mathbf{K}_{\mathcal{I}_i}, \mathbf{V}_{\mathcal{I}_i}$ de CPU a GPU para la *siguiente* capa $\triangleright$ solapa con la capa actual
 - 6: **end for**
 - 7: Calcular SDPA dispersa por bloques sobre los bloques seleccionados de la capa actual: $\mathbf{Y} \leftarrow \text{BlockSparseSDPA}(\mathbf{Q}, \mathbf{K}, \mathbf{V}, \{\mathcal{I}_i\})$
 - 8: Pérdida de entrenamiento: $\mathcal{L} \leftarrow \text{KL}(\text{dist Pronóstico} \parallel \pi_{\text{orig}})$ $\triangleright$ solo se actualiza W_F **return** $\mathbf{Y}$
-

4.10.3. Características Clave

- **Complejidad:** Pronóstico $\mathcal{O}(H_{\text{kv}} \cdot d_f \cdot N \cdot N_{\text{bloques}})$; Atención principal $\mathcal{O}(H_q \cdot d_h \cdot N \cdot k \cdot B_k)$.
- **Entrenable:** Sí; solo las proyecciones de Pronóstico se entrenan ($< 0,5\%$ parámetros extra).
- **Fortaleza:** En dos modelos sparse-pretrained de 8B, SparDA logra hasta $1,25\times$ en prefill y $1,7\times$ en decodificación sobre el baseline de offload de atención dispersa, y hasta $5,3\times$ throughput de decodificación frente a un baseline disperso sin offload.
- **Limitación:** Requiere un pipeline de offload CPU–GPU y un selector base congelado; el beneficio anticipado depende de que la demanda de bloques de la siguiente capa sea predecible a partir del Pronóstico actual.

4.11. Resumen Comparativo

Método	Complejidad	Entrenable	Unidad	Año	Contribución Principal
Sparse Transformer	$\mathcal{O}(n\sqrt{nd})$	Sí	Patrón de token	2019	Máscaras fijas y estrideas factorizadas
Longformer	$\mathcal{O}(nwd)$	Sí	Local + global	2020	Escalado lineal con dilatación
BigBird	$\mathcal{O}(n)$	Sí	Grafo disperso	2020	Prueba de completitud teórica
SparseK	$\mathcal{O}(nd)$	Sí	Tokens top- k	2024	Selección diferenciable
NSA	$\mathcal{O}((t/d + nl' + w)d)$	Sí	Multi-rama	2025	Diseño de 3 ramas alineado con hardware
FASA	$\mathcal{O}(tN_{\text{tip}} + N_{\text{fac}}d)$	No	Fragmentos de frecuencia	2026	Conocimiento de frecuencia RoPE
SpargeAttn	$\mathcal{O}(n^2 s \cdot d)$	No	Filtro de bloques	2025	Filtrado universal de dos etapas
MSA	$\mathcal{O}(H_{\text{kv}} d_{\text{idx}} N^2 + H_q d_h N k B_k)$	Sí	Bloques top- k	2026	Bloques dispersos con rama índice (desplegado 109B)

Método	Complejidad Entrenable	Unidad	Año	Contribución Principal
SparDA	$O(H_{kv}d_fNN_b + H_qd_hNkB_k)$	Pronóstico de bloques	2026	Selección desacoplada con prefetch anticipado

4.12. Espacio de Diseño y Criterios de Selección

Los siete métodos de atención dispersa ocupan posiciones complementarias en un espacio de diseño multidimensional:

- **Patrones geométricos/fijos** (Sparse Transformer, Longformer): Simples de implementar y analizar, pero los patrones no se adaptan al contenido.
- **Selección aprendida** (SparseK, NSA): Permiten adaptación mediante redes de selección entrenables, a costa de mayor complejidad de implementación.
- **Aceleración sin entrenamiento** (FASA, SpargeAttn): Permiten aceleración directa de modelos existentes sin reentrenamiento, adecuados para sistemas desplegados.
- **Garantías teóricas** (BigBird): Proporcionan demostraciones formales de completitud expresiva, importantes para comprender los márgenes de seguridad.

Orientación de selección:

1. **Entrenamiento de nuevos modelos** donde los recursos lo permitan: NSA o SparseK para máxima aceleración de inferencia; BigBird para garantía teórica.
2. **Comprensión de documentos de contexto largo**: Longformer o FASA por simplicidad práctica; NSA para escala extrema.
3. **Aceleración de modelos existentes**: FASA para LLMs basados en RoPE; SpargeAttn para cualquier arquitectura.
4. **Entornos con recursos limitados**: Sparse Transformer por simplicidad y eficiencia probada a escala moderada.

5. Familias de Atención con Compuerta—Análisis Exhaustivo de Literatura

5.1. Resumen Ejecutivo: Compuertas para el Control de Memoria

El campo emergente de los *mecanismos de atención con compuerta* aborda un desafío fundamental en el modelado de secuencias: ¿cómo debe retenerse, olvidarse y actualizarse la información a medida que el modelo procesa nuevos tokens? Los estados recurrentes aditivos tradicionales acumulan toda la información sin borrado, lo que lleva a la saturación de memoria y a la incapacidad de adaptarse a contextos cambiantes. La atención softmax proporciona expresividad completa pero usa un caché KV de $\mathcal{O}(Ld)$ durante la inferencia, limitando el tamaño de la ventana de contexto. Los mecanismos con compuerta proporcionan un punto intermedio: control selectivo sobre la supervivencia de la información.

El principio unificador entre las arquitecturas con compuerta es que las compuertas controlan *qué información sobrevive*. Las compuertas operan en tres niveles distintos:

1. **Decaimiento del estado recurrente** (GLA, HGRN2): Compuertas multiplicativas sobre la matriz de memoria S_t controlan cuánto del estado anterior persiste en el siguiente paso.

2. **Intensidad de escritura en actualizaciones recurrentes** (DeltaNet, Gated DeltaNet, Gated DeltaNet-2): Las compuertas controlan la magnitud y la dirección canalizada de la nueva información escrita en la memoria.
3. **Sesgo de logits softmax** (FoX): Las compuertas inyectan sesgo de actualidad a nivel de token en el cálculo de logits de atención.
4. **Dispersidad post-atención** (Gated Softmax): Las compuertas escalan selectivamente los canales de salida de SDPA.

Este apéndice sintetiza ocho arquitecturas principales de atención con compuerta publicadas entre 2023 y 2026, analizando sus fundamentos matemáticos, características de eficiencia en hardware y compensaciones empíricas.

5.2. 1. Atención Lineal con Compuerta (GLA)

5.2.1. Fundamento Matemático

La Atención Lineal con Compuerta [56] aumenta la atención lineal estándar (que usa un estado recurrente con valor de matriz) con decaimiento multiplicativo dependiente de los datos. La atención lineal estándar reformula el mecanismo softmax tradicional como una recurrencia sobre estados ocultos:

$$\mathbf{S}_t = \mathbf{S}_{t-1} + \mathbf{v}_t \mathbf{k}_t^\top, \quad \mathbf{o}_t = \mathbf{S}_t \mathbf{q}_t$$

donde el estado $\mathbf{S}_t \in \mathbb{R}^{d_v \times d_k}$ acumula productos externos. Esta formulación puramente aditiva sufre de “sobrecarga de memoria”: la información se acumula monótonamente y no puede borrarse, degradando el rendimiento en tareas que requieren selección de contexto.

GLA introduce una **matriz de compuerta diagonal** $\mathbf{G}_t \in [0, 1]^{d_k \times d_k}$ dependiente de los datos:

$$\mathbf{S}_t = \mathbf{G}_t \odot \mathbf{S}_{t-1} + \mathbf{v}_t \mathbf{k}_t^\top, \quad \mathbf{o}_t = \mathbf{S}_t \mathbf{q}_t$$

La compuerta $\mathbf{G}_t$ se parametriza con una estructura de producto externo:

$$\mathbf{G}_t = \mathbf{1} \alpha_t^\top, \quad \alpha_t = \text{logsigmoid}(\mathbf{W}_{gk} \mathbf{x}_t) / c$$

donde $\mathbf{W}_{gk}$ es una proyección de rango bajo ($\mathbb{R}^{d \rightarrow d_k}$) y $c \approx 16$ es un normalizador. La activación logsigmoid mapea $\alpha_t \in \mathbb{R}$ a aproximadamente $[-1/2, 0]$, y la división por c contrae esto aún más para que esté cerca de la identidad en la inicialización. La compuerta resultante $G_t = 1 + \alpha_t \approx 1$ cerca de la inicialización, luego aprende a decaer ($\alpha_t \rightarrow -\infty$) información histórica.

Algorithm 32 Atención Lineal con Compuerta (Forma Recurrente)

Require: Entrada $\mathbf{x}_t$; estado anterior $\mathbf{S}_{t-1}$

Require: Proyecciones: W_q, W_k, W_v (estándar); W_{gk} (proyección de compuerta)

Ensure: Salida $\mathbf{o}_t$ y estado actualizado $\mathbf{S}_t$

- 1: $\mathbf{q}_t \leftarrow W_q \mathbf{x}_t$
 - 2: $\mathbf{k}_t \leftarrow W_k \mathbf{x}_t$
 - 3: $\mathbf{v}_t \leftarrow W_v \mathbf{x}_t$
 - 4: Calcular compuerta: $\alpha_t \leftarrow \text{logsigmoid}(W_{gk} \mathbf{x}_t) / 16$ ▷ Decaimiento por dimensión de clave
 - 5: Aplicar compuerta de producto externo: $\mathbf{G}_t \leftarrow \mathbf{1} \alpha_t^\top$ ▷ producto externo diagonal
 - 6: Decaer estado anterior: $\mathbf{S}_t^{\text{decaimiento}} \leftarrow \mathbf{G}_t \odot \mathbf{S}_{t-1}$ ▷ producto elemento a elemento
 - 7: Agregar nueva asociación: $\mathbf{S}_t \leftarrow \mathbf{S}_t^{\text{decaimiento}} + \mathbf{v}_t \mathbf{k}_t^\top$
 - 8: Recuperar mediante consulta: $\mathbf{o}_t^{\text{crudo}} \leftarrow \mathbf{S}_t \mathbf{q}_t$
 - 9: Aplicar compuerta de salida: $\mathbf{o}_t \leftarrow \text{GroupNorm}(\mathbf{o}_t^{\text{crudo}}) \otimes \text{SiLU}(W_g \mathbf{x}_t)$ **return** $\mathbf{o}_t, \mathbf{S}_t$
-

5.2.2. Entrenamiento Eficiente en Hardware

Para el entrenamiento paralelo, GLA usa un algoritmo por fragmentos que agrupa tokens en fragmentos C y calcula transiciones recurrentes en paralelo:

$$\mathbf{O}_{[t]} = \overleftarrow{\mathbf{Q}}_{[t]} \mathbf{S}_{[t]}^\top + \left(\mathbf{Q}_{[t]} \mathbf{K}_{[t]}^\top \odot \mathbf{\Gamma}_{[t]} \right) \mathbf{V}_{[t]}$$

donde $\overleftarrow{\mathbf{Q}}_{[t]}$ atiende al estado en el límite del fragmento (retrospectiva), y $\mathbf{\Gamma}_{[t]}$ es la máscara causal con escalado consciente del decaimiento:

$$\mathbf{\Gamma}_{[t],ij} = \begin{cases} \prod_{l=j}^{i-1} \alpha_l & \text{si } i > j \text{ (retrospectiva)} \\ \prod_{l=1}^{i-j} \alpha_l & \text{si } i \leq j \text{ (en-fragmento)} \end{cases}$$

Este diseño maximiza las operaciones matmul susceptibles de aceleración mediante núcleos tensoriales, logrando una complejidad total subcuadrática $\mathcal{O}(nLd^2/C)$ donde L es el número de cabezas de atención y C es el tamaño del fragmento.

5.2.3. Fortalezas y Limitaciones

Aspecto	Fortalezas	Limitaciones
Eficiencia computacional	Entrenamiento subcuadrático mediante paralelismo por fragmentos. Inferencia con memoria constante $\mathcal{O}(d^2)$.	Requiere kernels CUDA/Triton personalizados; no disponible en todos los frameworks.
Generalización de contexto	Extiende entrenamiento de 2K tokens a más de 20K tokens con degradación de perplejidad insignificante.	Aún rinde por debajo de softmax en benchmarks con mucha recuperación (aguja en el pajar).
Selectividad	Decaimiento dependiente de los datos por dimensión de clave.	La estructura de compuerta limita la selectividad por par clave-valor; no hay control directo sobre qué asociaciones específicas borrar.

5.3. 2. DeltaNet: Atención Lineal con Corrección de Errores

5.3.1. Fundamento Matemático

DeltaNet [57] aplica una **regla de aprendizaje delta** clásica a las actualizaciones de estado de la atención lineal. En lugar de acumulación aditiva, DeltaNet calcula la diferencia entre el valor predicho (recuperado de la memoria) y el valor objetivo, luego realiza una actualización de corrección de errores:

$$\mathbf{S}_t = \mathbf{S}_{t-1}(\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top) + \beta_t \mathbf{v}_t \mathbf{k}_t^\top$$

donde $\beta_t \in (0, 1)$ es un escalar de **intensidad de escritura** dependiente de los datos. Esta actualización puede descomponerse como una operación de borrado-escritura:

$$\mathbf{v}_t^{\text{anterior}} = \mathbf{S}_{t-1} \mathbf{k}_t \quad (\text{predicción actual}), \quad \mathbf{v}_t^{\text{actualizado}} = \beta_t \mathbf{v}_t + (1 - \beta_t) \mathbf{v}_t^{\text{anterior}} \quad (\text{mezcla anterior/nuevo})$$

de modo que:

$$\mathbf{S}_t = \mathbf{S}_{t-1} - \mathbf{v}_t^{\text{anterior}} \mathbf{k}_t^\top + \mathbf{v}_t^{\text{actualizado}} \mathbf{k}_t^\top$$

La idea clave es que esta regla de actualización minimiza una pérdida de ECM en línea en cada paso temporal:

$$\mathcal{L}_t(\mathbf{S}) = \frac{1}{2} \|\mathbf{S}\mathbf{k}_t - \mathbf{v}_t\|^2 \Rightarrow \nabla_{\mathbf{S}} \mathcal{L}_t = (\mathbf{S}\mathbf{k}_t - \mathbf{v}_t)\mathbf{k}_t^\top$$

Un paso de SGD con tasa de aprendizaje β_t produce la actualización de la regla delta. Esta conexión con el entrenamiento en tiempo de prueba (TTT) proporciona fundamentación teórica: DeltaNet optimiza directamente la predicción de valores en tiempo de inferencia.

5.3.2. Entrenamiento Paralelo Eficiente

La matriz de transición de DeltaNet $\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top$ es una matriz de Householder generalizada (actualización de rango 1 a la identidad). Para el entrenamiento por fragmentos, DeltaNet usa la **representación** $\mathbf{WY}$, que representa el producto de m de estas actualizaciones de rango 1 de forma compacta:

$$\prod_{i=1}^m (\mathbf{I} - \beta_i \mathbf{k}_i \mathbf{k}_i^\top) = \mathbf{I} - \mathbf{WY}^\top$$

donde $\mathbf{W}, \mathbf{Y} \in \mathbb{R}^{d \times m}$ se construyen recursivamente. Esta factorización permite paralelismo rico en matmul sin materializar el producto completo, manteniendo el entrenamiento por fragmentos eficiente.

Algorithm 33 Actualización de Estado de DeltaNet con Representación WY

Require: Fragmento de entrada $\mathbf{X}_{[t]}$; estado anterior $\mathbf{S}_{t-1}$

Require: Consultas normalizadas L2 $\hat{\mathbf{Q}}_t$, claves $\hat{\mathbf{K}}_t$, valores $\mathbf{V}_t$

Ensure: Salida $\mathbf{O}_{[t]}$ y estado actualizado $\mathbf{S}_t$

1: **Fase por fragmentos:**

2: **for** posición $i = 1$ to tamaño_fragmento **do**

3: Normalizar: $\hat{\mathbf{k}}_i \leftarrow \hat{\mathbf{K}}_t[i] / \|\hat{\mathbf{K}}_t[i]\|$, $\hat{\mathbf{q}}_i \leftarrow \hat{\mathbf{Q}}_t[i] / \|\hat{\mathbf{Q}}_t[i]\|$

4: Calcular intensidad de escritura: $\beta_i \leftarrow \text{sigmoid}(W_\beta \mathbf{x}_i)$

5: Predicción: $\hat{\mathbf{v}}^{\text{pred}} \leftarrow \mathbf{S}_{i-1}^{\text{en-fragmento}} \hat{\mathbf{k}}_i$

6: Mezcla consciente del error: $\mathbf{v}_i^{\text{actualización}} \leftarrow \beta_i (\mathbf{v}_i - \hat{\mathbf{v}}^{\text{pred}})$

7: Borrar-escribir: $\mathbf{S}_i^{\text{en-fragmento}} \leftarrow \mathbf{S}_{i-1}^{\text{en-fragmento}} - \hat{\mathbf{v}}^{\text{pred}} \hat{\mathbf{k}}_i^\top + (\beta_i \mathbf{v}_i + (1 - \beta_i) \hat{\mathbf{v}}^{\text{pred}}) \hat{\mathbf{k}}_i^\top$

8: Salida: $\mathbf{o}_i \leftarrow \mathbf{S}_i^{\text{en-fragmento}} \hat{\mathbf{q}}_i$

9: **end for**

10: **Fase entre fragmentos:** Usar representación WY de los productos de transición **return** $\mathbf{O}_{[t]}, \mathbf{S}_t$

5.3.3. Fortalezas y Limitaciones

Aspecto	Fortalezas	Limitaciones
Recuperación asociativa	Recuperación perfecta en el benchmark MQAR (Recuperación Asociativa Multi-Consulta); la semántica de corrección de errores se ajusta naturalmente a patrones de recuperación.	Sin olvido global, la memoria aún se satura en longitudes de secuencia extremas; requiere mecanismos de reinicio periódico para contexto ilimitado.
Teoría	Fundamentada en optimización de ECM en línea; conexión clara con el paradigma de entrenamiento en tiempo de prueba.	Requiere claves normalizadas L2 para estabilidad numérica; la normalización de doble ancho añade sobrecarga.
Rendimiento	La representación WY permite entrenamiento eficiente por fragmentos.	Ligeramente más lento que Mamba2 por token debido a matrices de transición más ricas (rango 1 en lugar de diagonales).

5.4. 3. Gated DeltaNet: Síntesis de Compuerta y Corrección de Errores

5.4.1. Fundamento Matemático

Gated DeltaNet [58] sintetiza las fortalezas de GLA y DeltaNet combinando una **compuerta de decaimiento** α_t (de GLA) con la **compuerta de intensidad de escritura** β_t (de DeltaNet):

$$\mathbf{S}_t = \mathbf{S}_{t-1} \left(\alpha_t (\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top) \right) + \beta_t \mathbf{v}_t \mathbf{k}_t^\top$$

Reescribiendo para mayor claridad:

$$\mathbf{S}_t = \alpha_t \mathbf{S}_{t-1} (\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top) + \beta_t \mathbf{v}_t \mathbf{k}_t^\top$$

Las dos compuertas son complementarias:

- $\alpha_t \rightarrow 0$ borra rápidamente el estado histórico: $\mathbf{S}_t \approx \beta_t \mathbf{v}_t \mathbf{k}_t^\top$ (reinicio de contexto).
- $\alpha_t \rightarrow 1$ recupera el comportamiento puro de regla delta: $\mathbf{S}_t \approx \mathbf{S}_{t-1} (\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top) + \beta_t \mathbf{v}_t \mathbf{k}_t^\top$ (actualizaciones dirigidas).
- $\beta_t \rightarrow 0$ omite la escritura pero decae: $\mathbf{S}_t \approx \alpha_t \mathbf{S}_{t-1}$ (olvido puro).
- $\beta_t \rightarrow 1$ reemplaza la memoria agresivamente: $\mathbf{S}_t \approx \alpha_t \mathbf{S}_{t-1} + \mathbf{v}_t \mathbf{k}_t^\top$ (reemplazo).

Desde una perspectiva de aprendizaje en línea, Gated DeltaNet corresponde a:

$$\min_{\mathbf{S}_t} \|\mathbf{S}_t - \alpha_t \mathbf{S}_{t-1}\|_F^2 - 2 \langle \mathbf{S}_t \mathbf{k}_t, \beta_t (\mathbf{v}_t - \alpha_t \mathbf{S}_{t-1} \mathbf{k}_t) \rangle$$

que introduce un decaimiento de pesos adaptativo α_t en una actualización tipo SGD—análogo al decaimiento de pesos desacoplado en la optimización de aprendizaje profundo.

Algorithm 34 Paso Adelante Paralelo por Fragmentos de Gated DeltaNet

Require: Fragmento de entrada $\mathbf{X}_{[i]}$; estado anterior $\mathbf{S}_{i-1}$; tamaño de fragmento C

Require: Proyecciones: $W_q, W_k, W_v, W_\alpha, W_\beta$

Ensure: Salida $\mathbf{O}_{[i]}$ y estado $\mathbf{S}_i$

1: **Cálculo en-fragmento:**

2: **for** $j = 1$ to C **do**

3: $\mathbf{q}_j \leftarrow W_q \mathbf{x}_j, \mathbf{k}_j \leftarrow W_k \mathbf{x}_j, \mathbf{v}_j \leftarrow W_v \mathbf{x}_j$

4: $\alpha_j \leftarrow \text{sigmoid}(W_\alpha \mathbf{x}_j), \beta_j \leftarrow \text{sigmoid}(W_\beta \mathbf{x}_j)$

5: Aplicar compuerta combinada: $\mathbf{S}_j \leftarrow \alpha_j \mathbf{S}_{j-1} (\mathbf{I} - \beta_j \mathbf{k}_j \mathbf{k}_j^\top) + \beta_j \mathbf{v}_j \mathbf{k}_j^\top$

6: $\mathbf{o}_j \leftarrow \mathbf{S}_j \mathbf{q}_j$

7: **end for**

8: **Recurrencia entre fragmentos:**

9: $\mathbf{S}_i \leftarrow \mathbf{S}_C$ del en-fragmento; propagar entre fragmentos mediante máscaras $\prod \alpha$ acumulativas

10: **Compuerta de salida:** $\mathbf{O}_{[i]} \leftarrow \text{GroupNorm}(\mathbf{O}^{\text{crudo}}) \otimes \text{SiLU}(W_g \mathbf{X}_{[i]})$ **return** $\mathbf{O}_{[i]}, \mathbf{S}_i$

5.4.2. Rendimiento Empírico y Compensaciones

Gated DeltaNet se ha integrado en los modelos de producción Qwen3-Next y Qwen3.5 de Alibaba. Empíricamente:

Tarea	Gated DeltaNet	Mamba2	DeltaNet	GLA
Modelado de Lenguaje	1,0 $\times$	1,08 $\times$	1,05 $\times$	1,12 $\times$
Recuperación Asociativa	1,0 $\times$	— — (<i>sinperfecto</i>)	1,0 $\times$	0,75
Extrapolación de Longitud	1,0 $\times$	0,98 $\times$	0,96 $\times$	0,94 $\times$
Rendimiento (tokens/seg)	0,85 $\times$	1,0 $\times$	0,82 $\times$	0,88 $\times$

Gated DeltaNet logra el mejor equilibrio entre diversas tareas a costa de un rendimiento ligeramente reducido debido a matrices de transición más ricas.

5.5. 4. HGRN2: Compuerta Jerárquica con Expansión de Producto Externo

5.5.1. Fundamento Matemático

HGRN2 [40] usa una expansión de estado basada en producto externo con **compuertas de olvido acotadas inferiormente de forma jerárquica**. La actualización de estado es:

$$\mathbf{S}_t = \text{diag}(\mathbf{g}_t) \cdot \mathbf{S}_{t-1} + \mathbf{v}_t \mathbf{k}_t^\top, \quad \mathbf{o}_t = \mathbf{S}_t \mathbf{q}_t$$

donde $\mathbf{g}_t \in [b_\ell, 1]^d$ es una compuerta de olvido con cota inferior para la capa ℓ . Las cotas satisfacen:

$$0 \leq b_{\ell_{\text{poco profunda}}} < b_{\ell_{\text{media}}} < b_{\ell_{\text{profunda}}} \leq 1$$

es decir, las cotas aumentan (se vuelven menos restrictivas) en las capas más profundas. La compuerta en sí se calcula como:

$$\mathbf{g}_t = b_\ell + (1 - b_\ell) \cdot \sigma(W_g \mathbf{x}_t)$$

donde σ es sigmoide. Cuando W_g produce logits débilmente negativos, $\mathbf{g}_t \approx b_\ell$ (retención forzada en capas superficiales). Cuando las salidas son fuertemente positivas, $\mathbf{g}_t \approx 1$ (refresco de memoria en cualquier capa).

La estructura jerárquica anima a diferentes capas a especializarse en diferentes escalas temporales: las capas superficiales modelan dependencias locales (alta retención debido a la cota b_ℓ), mientras que las capas profundas capturan estructura de largo alcance (compuerta flexible con cota superior alta).

Algorithm 35 Paso Adelante de HGRN2 con Cotas Jerárquicas

Require: Entrada $\mathbf{x}_t$; estado anterior $\mathbf{S}_{t-1}$; índice de capa $\ell \in [0, L - 1]$

Require: Cotas no decrecientes: $0 \leq b_0 < b_1 < \dots < b_{L-1} \leq 1$

Ensure: Salida $\mathbf{o}_t$ y estado $\mathbf{S}_t$

- 1: Cota de retención específica de capa: $b \leftarrow b_\ell$
 - 2: Proyectar: $\mathbf{q}_t \leftarrow W_q \mathbf{x}_t$, $\mathbf{k}_t \leftarrow W_k \mathbf{x}_t$, $\mathbf{v}_t \leftarrow W_v \mathbf{x}_t$
 - 3: Calcular compuerta de olvido con cota: $\mathbf{g}_t \leftarrow b + (1 - b) \cdot \sigma(W_g \mathbf{x}_t)$ $\triangleright$ Confinado a $[b, 1]$
 - 4: Multiplicación diagonal (vectorizada): $\mathbf{S}_t \leftarrow \mathbf{S}_{t-1} \circ \text{diag}(\mathbf{g}_t) + \mathbf{v}_t \mathbf{k}_t^\top$
 - 5: Recuperar: $\mathbf{o}_t \leftarrow \mathbf{S}_t \mathbf{q}_t$
 - 6: Normalización opcional: $\mathbf{o}_t \leftarrow \text{RMSNorm}(\mathbf{o}_t)$ **return** $\mathbf{o}_t$, $\mathbf{S}_t$
-

5.5.2. Escalado y Resultados Empíricos

A escala de 3B en 100B tokens, HGRN2 supera ligeramente a Mamba2 y a los Transformers con arquitectura LLaMA en modelado de lenguaje. La compuerta jerárquica proporciona modelado temporal multi-escala sin variantes arquitectónicas explícitas por capa. Sin embargo, la compuerta diagonal con valor vectorial (en oposición a la selección elemento a elemento) proporciona menos flexibilidad por elemento que las variantes de regla delta.

5.6. 5. Forgetting Transformer (FoX): Compuerta en el Espacio de Logits Softmax

5.6.1. Fundamento Matemático

Forgetting Transformer (FoX), propuesto por Lin et al. [28], incrusta una compuerta de olvido directamente en los logits de atención softmax. En lugar de reemplazar softmax con recurrencia lineal, FoX preserva la expresividad completa de softmax mientras añade control de actualidad mediante un sesgo de logit dependiente de los datos.

Para cada posición de token t y todas las claves $j \leq t$, calcular un escalar de compuerta de olvido:

$$f_t = \sigma(w_f^\top \mathbf{x}_t + b_f)$$

El sesgo de logit en la posición (i, j) es el producto acumulativo log-olvido:

$$d_{ij} = \sum_{l=j+1}^i \log f_l = \log \prod_{l=j+1}^i f_l$$

La salida de atención completa se convierte en:

$$\mathbf{O} = \text{softmax} \left(\frac{\mathbf{QK}^\top}{\sqrt{d_k}} + \mathbf{D} \right) \mathbf{V}$$

donde $\mathbf{D}_{ij} = d_{ij}$. Equivalentemente:

$$\text{Atencion}(i, j) = \frac{\exp(\mathbf{q}_i^\top \mathbf{k}_j / \sqrt{d_k} + d_{ij})}{\sum_{j'=1}^i \exp(\mathbf{q}_i^\top \mathbf{k}_{j'} / \sqrt{d_k} + d_{ij'})}$$

Esta ponderación reduce el peso de los tokens más antiguos multiplicativamente: un token de hace 10 pasos se escala por $\prod_{l=j+1}^i f_l$, que es típicamente mucho menor que 1 si $f_t < 1$ en promedio.

El mecanismo es matemáticamente equivalente a una **variante aprendible dependiente de los datos de ALiBi (Atención con Sesgos Lineales)**, donde trabajos anteriores usaban $d_{ij} = -\infty \cdot |i - j|$ fijo o $d_{ij} = -(i - j)$.

Algorithm 36 FoX: Atención con Olvido y Sesgo de Actualidad

Require: Consultas $\mathbf{Q}$, claves $\mathbf{K}$, valores $\mathbf{V}$; entrada $\mathbf{X}$

Require: Parámetros de compuerta de olvido: $w_f \in \mathbb{R}^d$, $b_f \in \mathbb{R}$

Ensure: Salida de atención $\mathbf{O}$

```
1: Calcular compuertas de olvido:
2: for  $t = 1$  to  $T$  do
3:    $f_t \leftarrow \sigma(w_f^\top \mathbf{x}_t + b_f)$  ▷ Probabilidad de olvido por token
4: end for
5: Calcular sesgos acumulativos de log-olvido:
6: for  $i = 1$  to  $T$  do
7:   for  $j = 1$  to  $i$  do
8:      $d_{ij} \leftarrow \sum_{l=j+1}^i \log f_l$  ▷ Producto acumulativo en espacio logarítmico
9:   end for
10: end for
11: SDPA estándar con sesgo:
12: Calcular puntajes:  $\mathbf{S} \leftarrow \mathbf{QK}^\top / \sqrt{d_k}$ 
13: Agregar sesgo:  $\mathbf{S} \leftarrow \mathbf{S} + \mathbf{D}$ 
14: Aplicar softmax:  $\mathbf{A} \leftarrow \text{softmax}(\mathbf{S}, \text{dim} = -1)$ 
15: Ponderar valores:  $\mathbf{O} \leftarrow \mathbf{AV}$  return  $\mathbf{O}$ 
```

5.6.2. Integración con FlashAttention

La ventaja clave de FoX es la compatibilidad con FlashAttention y las implementaciones optimizadas existentes. Los sesgos de olvido se añaden en el cálculo de logits softmax, que ya es una operación central en el algoritmo por bloques de FlashAttention. La sobrecarga es:

$$\text{Sobrecarga} \approx 0,5 - 2\% \text{ del tiempo de pared}$$

ya que la adición de sesgo se amortiza entre las operaciones matmul.

5.6.3. Fortalezas y Limitaciones

FoX logra resultados de última generación en extrapolación de longitud, recuperación casi perfecta de aguja en el pajar y comprensión superior de contexto largo en comparación con los Transformers. Sin embargo, sigue siendo $\mathcal{O}(L^2d)$ en entrenamiento y $\mathcal{O}(Ld)$ por paso en inferencia con caché KV, limitando las longitudes de secuencia extremas.

5.7. 6. Atención con Compuerta (Post-SDPA Sigmoides)

5.7.1. Fundamento Matemático

El Mejor Artículo de NeurIPS 2025 del equipo de Qwen propone aplicar una compuerta sigmoide **después** de la atención de producto punto escalado (SDPA):

$$\mathbf{Y} = \text{SDPA}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{QK}^\top}{\sqrt{d_k}}\right) \mathbf{V}$$

$$\mathbf{Y}' = \mathbf{Y} \odot \sigma(\mathbf{XW}_\theta)$$

donde el puntaje de compuerta $\sigma(\mathbf{XW}_\theta)$ puede tener forma $\mathbb{R}^{n \times q \times d_k}$ (compuerta elemento a elemento por cabeza de consulta) o $\mathbb{R}^{n \times q}$ (compuerta fija por cabeza). En más de 30 variantes y modelos MoE de 15B + densos de 1.7B entrenados en 3.5T tokens, el equipo encontró:

- La compuerta por cabeza añade solo $\sim 1,6M$ parámetros a un modelo de 15B (sobrecarga insignificante).

- Las compuertas efectivas son **dispersas**: activación media $\approx 0,116$ (es decir, el 88 % son cero o casi cero).
- Las compuertas actúan como filtros dependientes de la consulta, suprimiendo canales de bajo valor mientras preservan señales de alto valor.
- Las compuertas eliminan demostrablemente el fenómeno de **sumidero de atención**, donde el patrón de atención se vuelve dominado por un único token temprano.

Algorithm 37 Atención Softmax con Compuerta (Post-SDPA)

Require: Consultas $\mathbf{Q}$, Claves $\mathbf{K}$, Valores $\mathbf{V}$; entrada $\mathbf{X}$

Require: Matriz de proyección de compuerta $W_g \in \mathbb{R}^{d \rightarrow d_{\text{compuerta}}}$ donde $d_{\text{compuerta}} \in \{1, d_k\}$

Ensure: Salida con compuerta $\mathbf{Y}'$

```

1: SDPA estándar:
2: Calcular atención:  $\mathbf{A} \leftarrow \text{softmax}(\mathbf{QK}^\top / \sqrt{d_k})$ 
3: Ponderar valores:  $\mathbf{Y} \leftarrow \mathbf{AV}$ 
4: Calcular compuertas:
5: Proyectar entrada:  $\mathbf{G} \leftarrow \mathbf{XW}_g$  ▷ forma:  $[n, q, d_{\text{compuerta}}]$ 
6: Aplicar sigmoide:  $\mathbf{G} \leftarrow \sigma(\mathbf{G})$  ▷ Compuerta elemento a elemento
7: Aplicar compuerta:
8: if  $d_{\text{compuerta}} = 1$  then
9:   Transmitir y escalar:  $\mathbf{Y}' \leftarrow \mathbf{Y} \cdot \mathbf{G}$  ▷ Escalado por cabeza
10: else ▷  $d_{\text{compuerta}} = d_k$ 
11:   Modulación elemento a elemento:  $\mathbf{Y}' \leftarrow \mathbf{Y} \odot \mathbf{G}$  ▷ Compuerta por dimensión
12: end if return  $\mathbf{Y}'$ 

```

5.7.2. Hallazgos Clave

- **Supresión del sumidero de atención:** La compuerta rompe el ciclo de retroalimentación donde softmax se concentra en el primer token, permitiendo una atención más equilibrada.
- **Estabilidad de entrenamiento:** Los modelos con compuerta toleran tasas de aprendizaje más grandes (+30 % mayor η_{max} estable).
- **Sobrecarga mínima:** El impacto en latencia es solo del 1,6 % en GPUs H100; el rendimiento cae como máximo 2 – 3 %.
- **Mejora en la ley de escalado:** Mejora la pérdida en todas las escalas de modelo, desde 800M hasta 110B parámetros de forma consistente.

5.8. 7. Gated DeltaNet-2: Borrado y Escritura Canalizados Decouplados

5.8.1. Fundamento Matemático

Gated DeltaNet-2 [21] aborda una limitación compartida de Gated DeltaNet y Kimi Delta Attention (KDA): ambos usan una única *compuerta delta escalar* β_t para controlar simultáneamente (i) cuánto contenido antiguo se borra en la dirección de lectura del eje de claves y (ii) cuánto valor nuevo se escribe en el eje de valores. Gated DeltaNet-2 **desacopla** estos dos roles en una compuerta de borrado canalizada $\mathbf{b}_t$ (eje de claves) y una compuerta de escritura canalizada $\mathbf{w}_t$ (eje de valores), heredando además el decaimiento canalizado α_t de KDA. Con estado $\mathbf{S}_t \in \mathbb{R}^{d_k \times d_v}$, decaimiento $\mathbf{D}_t = \text{Diag}(\alpha_t)$, dirección de borrado $\mathbf{e}_t = \mathbf{b}_t \odot \mathbf{k}_t$ y objetivo de escritura $\mathbf{z}_t = \mathbf{w}_t \odot \mathbf{v}_t$, la actualización de estado es:

$$\mathbf{S}_t = \left(\mathbf{I} - \mathbf{k}_t \mathbf{e}_t^\top \right) \mathbf{D}_t \mathbf{S}_{t-1} + \mathbf{k}_t \mathbf{z}_t^\top, \quad \mathbf{o}_t = \mathbf{S}_t^\top \mathbf{q}_t$$

o, expandiendo los productos de las compuertas:

$$\mathbf{S}_t = \left(\mathbf{I} - \mathbf{k}_t(\mathbf{b}_t \odot \mathbf{k}_t)^\top \right) \mathbf{D}_t \mathbf{S}_{t-1} + \mathbf{k}_t(\mathbf{w}_t \odot \mathbf{v}_t)^\top$$

Las dos compuertas se producen mediante proyecciones lineales independientes seguidas de sigmoide:

$$\mathbf{b}_t = \sigma(\mathbf{W}_b \mathbf{x}_t), \quad \mathbf{w}_t = \sigma(\mathbf{W}_w \mathbf{x}_t)$$

y el decaimiento canalizado sigue una parametrización de log-decaimiento (calculada en fp32 para evitar pérdida de precisión en productos acumulativos largos):

$$\mathbf{g}_t = \mathbf{a} - \text{softplus}(\boldsymbol{\delta}), \quad \boldsymbol{\alpha}_t = \exp(\mathbf{g}_t)$$

Los roles estructurales de las dos compuertas son complementarios:

- $\mathbf{b}_t$ (eje de claves) hace que la **dirección de lectura** sea selectiva por canal: determina qué canales de clave se eliminan antes de escribir.
- $\mathbf{w}_t$ (eje de valores) hace que el **objetivo de escritura** sea selectivo por canal: determina qué canales de valor se depositan en memoria.
- $\boldsymbol{\alpha}_t$ (por canal de clave) proporciona decaimiento global, como en KDA.

Reducciones. Fijar $\mathbf{b}_t = \beta_t \mathbf{1}_{d_k}$ y $\mathbf{w}_t = \beta_t \mathbf{1}_{d_v}$ recupera KDA; colapsar además $\boldsymbol{\alpha}_t$ a un escalar recupera Gated DeltaNet. Desde la perspectiva de aprendizaje en línea/pesos rápidos, la actualización es el minimizador de:

$$\mathcal{L}_t(\mathbf{S}) = \frac{1}{2} \|\mathbf{S} - \mathbf{D}_t \mathbf{S}_{t-1}\|_F^2 - 2 \left\langle \mathbf{S}^\top \mathbf{k}_t, \mathbf{z}_t - (\mathbf{D}_t \mathbf{S}_{t-1})^\top \mathbf{e}_t \right\rangle$$

Algorithm 38 Avance Recurrente de Gated DeltaNet-2 (Forma de Referencia)

Require: Entrada $\mathbf{x}_t$; estado previo $\mathbf{S}_{t-1} \in \mathbb{R}^{d_k \times d_v}$

Require: Proyecciones: W_q, W_k, W_v, W_b, W_w ; parámetros de log-decaimiento $\mathbf{a}, \boldsymbol{\delta}$

Ensure: Salida $\mathbf{o}_t$ y estado $\mathbf{S}_t$

- 1: $\mathbf{q}_t \leftarrow \text{normalize}(W_q \mathbf{x}_t)$, $\mathbf{k}_t \leftarrow \text{normalize}(W_k \mathbf{x}_t)$, $\mathbf{v}_t \leftarrow \text{silu}(W_v \mathbf{x}_t)$
 - 2: $\mathbf{b}_t \leftarrow \sigma(W_b \mathbf{x}_t)$ ▷ compuerta de borrado eje-clave, $\mathbb{R}^{d_k}$
 - 3: $\mathbf{w}_t \leftarrow \sigma(W_w \mathbf{x}_t)$ ▷ compuerta de escritura eje-valor, $\mathbb{R}^{d_v}$
 - 4: $\boldsymbol{\alpha}_t \leftarrow \exp(\mathbf{a} - \text{softplus}(\boldsymbol{\delta}))$ ▷ decaimiento canalizado, fp32
 - 5: Dirección de borrado: $\mathbf{e}_t \leftarrow \mathbf{b}_t \odot \mathbf{k}_t$; objetivo de escritura: $\mathbf{z}_t \leftarrow \mathbf{w}_t \odot \mathbf{v}_t$
 - 6: Decaer estado: $\mathbf{S}_t \leftarrow \text{Diag}(\boldsymbol{\alpha}_t) \mathbf{S}_{t-1}$
 - 7: Lectura: $\mathbf{r}_t \leftarrow \mathbf{S}_t^\top \mathbf{e}_t$ ▷ suma sobre eje de claves
 - 8: Actualización delta: $\mathbf{S}_t \leftarrow \mathbf{S}_t - \mathbf{k}_t \mathbf{r}_t^\top + \mathbf{k}_t \mathbf{z}_t^\top$
 - 9: Recuperar: $\mathbf{o}_t \leftarrow \mathbf{S}_t^\top \mathbf{q}_t$
 - 10: Compuerta de salida: $\mathbf{o}_t \leftarrow \text{GroupNorm}(\mathbf{o}_t) \odot \text{silu}(W_g \mathbf{x}_t)$
 - 11: **return** $\mathbf{o}_t, \mathbf{S}_t$
-

El algoritmo de entrenamiento por fragmentos extiende la representación WY de KDA absorbiendo el decaimiento canalizado en una recurrencia delta asimétrica sobre un estado normalizado por decaimiento, permitiendo paralelismo rico en matmults en tensor cores. Como $\mathbf{b}_t$ y $\mathbf{w}_t$ son operadores diagonales por canal que varían por fila, el atajo de post-escalado escalar de KDA se rompe: el paso hacia atrás requiere una **inversión WY consciente de las compuertas** que incorpora las compuertas en cada producto escalar que acumula el gradiente.

5.8.2. Rendimiento Empírico y Compromisos

A 1.3B parámetros entrenados con 100B tokens FineWeb-Edu, Gated DeltaNet-2 supera a Mamba-2, Mamba-3, Gated DeltaNet y KDA en modelado de lenguaje, razonamiento de sentido común y—más notablemente—recuperación de contexto largo del mundo real y RULER (especialmente multi-key needle-in-a-haystack). Su rendimiento en H100 se mantiene casi plano ($38,0 \rightarrow 36,1$ Kt/s) al crecer la longitud de secuencia de 2K a 16K, donde el de un Transformer cae bruscamente. Los estudios de ablación muestran que la compuerta de borrado $\mathbf{b}_t$ explica la mayor parte de la ganancia; la compuerta de escritura $\mathbf{w}_t$ aporta una mejora adicional menor. La variante híbrida (Gated DeltaNet-2 recurrente + atención de ventana deslizante) reduce aún más la brecha con los modelos de atención pura en tareas que requieren agregación de evidencia local.

Aspecto	Fortaleza	Limitación
Recuperación	La mejor entre modelos delta recurrentes en multi-key NIAH.	La variante recurrente aún supera al Transformer en NQ/DROP.
Rendimiento	Escalado casi plano 2K→16K; pequeña sobrecarga vs KDA.	Requiere un nuevo kernel Triton WY consciente de las compuertas.
Memoria	Estado de inferencia constante $\mathcal{O}(d^2)$.	Rango de borrado de autovalores negativos $[0, 2]$ no da ganancia consistente a 1.3B.

Cuadro 10: Compromisos de Gated DeltaNet-2 a escala 1.3B / 100B tokens [21].

5.9. 8. Kimi Delta Attention (KDA): Compuerta de Decaimiento Canalizada para la Regla Delta

5.9.1. Fundamento Matemático

Kimi Delta Attention [23] es el módulo central de Kimi Linear y extiende Gated DeltaNet con una **compuerta de decaimiento canalizada** de grano más fino. Mientras que Gated DeltaNet usa un único decaimiento escalar por cabeza α_t junto con la compuerta escalar de escritura delta β_t , KDA reemplaza el decaimiento escalar con un decaimiento **por canal de clave** $\alpha_t \in \mathbb{R}^{d_k}$, parametrizado mediante una formulación de log-decaimiento y aplicado como una matriz diagonal $\mathbf{D}_t = \text{diag}(\alpha_t)$ antes de la actualización de la regla delta. La regla delta en sí retiene una compuerta de escritura escalar por cabeza β_t :

$$\mathbf{S}_t = (\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top) \mathbf{D}_t \mathbf{S}_{t-1} + \beta_t \mathbf{v}_t \mathbf{k}_t^\top, \quad \mathbf{o}_t = \mathbf{S}_t^\top \mathbf{q}_t$$

El decaimiento canalizado sigue una parametrización de log-decaimiento (calculada en fp32 por estabilidad numérica en productos acumulativos largos):

$$\mathbf{g} = \mathbf{a} - \text{softplus}(\boldsymbol{\delta}), \quad \alpha_t = \exp(\mathbf{g})$$

donde $\mathbf{a}$ y $\boldsymbol{\delta}$ son parámetros aprendidos por canal. Esto da a cada canal de clave una escala temporal de olvido independiente, permitiendo al modelo retener algunos canales más tiempo que otros dentro de la misma cabeza.

Reducción a Gated DeltaNet : Cuando α_t colapsa a un escalar (es decir, todos los canales comparten el mismo decaimiento), KDA se reduce exactamente a Gated DeltaNet. El decaimiento canalizado es por tanto una generalización estricta que añade selectividad por canal sobre el decaimiento/escritura escalares de Gated DeltaNet.

Entrenamiento por fragmentos : Un algoritmo por fragmentos ad hoc usa una variante **Diagonal-Plus-Low-Rank (DPLR)** especializada de la matriz de transición para mantener la eficiencia en hardware. El producto de las transiciones por paso $(\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top) \mathbf{D}_t$ admite una descomposición DPLR

Algorithm 39 Avance Recurrente de Kimi Delta Attention (Forma de Referencia)

Require: Entrada $\mathbf{x}_t$; estado previo $\mathbf{S}_{t-1} \in \mathbb{R}^{d_k \times d_v}$

Require: Proyecciones: W_q, W_k, W_v, W_β ; parámetros de log-decaimiento $\mathbf{a}, \delta$

Ensure: Salida $\mathbf{o}_t$ y estado $\mathbf{S}_t$

- 1: $\mathbf{q}_t \leftarrow W_q \mathbf{x}_t, \mathbf{k}_t \leftarrow W_k \mathbf{x}_t, \mathbf{v}_t \leftarrow W_v \mathbf{x}_t$
 - 2: $\beta_t \leftarrow \text{sigmoid}(W_\beta \mathbf{x}_t)$ ▷ compuerta de escritura escalar por cabeza
 - 3: $\boldsymbol{\alpha}_t \leftarrow \exp(\mathbf{a} - \text{softplus}(\delta))$ ▷ decaimiento canalizado, fp32
 - 4: $\mathbf{D}_t \leftarrow \text{diag}(\boldsymbol{\alpha}_t)$ ▷ matriz diagonal de decaimiento
 - 5: Decaer estado: $\mathbf{S}_t \leftarrow \mathbf{D}_t \mathbf{S}_{t-1}$
 - 6: Lectura: $\mathbf{r}_t \leftarrow \mathbf{S}_t^\top \mathbf{k}_t$ ▷ predicción actual a lo largo de la clave
 - 7: Actualización delta: $\mathbf{S}_t \leftarrow \mathbf{S}_t - \beta_t \mathbf{k}_t \mathbf{r}_t^\top + \beta_t \mathbf{v}_t \mathbf{k}_t^\top$
 - 8: Recuperar: $\mathbf{o}_t \leftarrow \mathbf{S}_t^\top \mathbf{q}_t$
 - 9: Compuerta de salida: $\mathbf{o}_t \leftarrow \text{GroupNorm}(\mathbf{o}_t) \odot \text{silu}(W_g \mathbf{x}_t)$
 - 10: **return** $\mathbf{o}_t, \mathbf{S}_t$
-

5.9.2. Rendimiento Empírico y Compromisos

KDA está desplegado en Kimi Linear, un modelo híbrido (3B activados / 48B parámetros totales) que combina capas KDA con Multi-head Latent Attention (MLA). Kimi Linear supera a un baseline de MLA completa en todas las tareas evaluadas, reduce el caché KV hasta un 75 %, y logra hasta 6× de throughput de decodificación para contexto de 1M de tokens. El decaimiento canalizado es el principal impulsor de la ganancia: permite control fino sobre qué canales de clave persisten, mejorando el recuerdo de contexto largo frente a Gated DeltaNet con decaimiento escalar.

Aspecto	Fortaleza	Limitación
Recuperación	El decaimiento canalizado mejora el recuerdo de contexto largo frente a baselines de decaimiento escalar; desplegado en Kimi Linear (híbrido 3B/48B).	El estado recurrente matricial $\mathcal{O}(d^2)$ es más pesado que la atención lineal con compuerta diagonal.
Rendimiento	Hasta 6× de throughput de decodificación a 1M de contexto; 75 % de reducción de caché KV frente a MLA completa.	Requiere kernels por fragmentos conscientes de DPLR para eficiencia en hardware.
Memoria	Estado de inferencia constante $\mathcal{O}(d^2)$ (recurrente matricial).	El cálculo log-decaimiento en fp32 añade sobrecarga; la reducción a Gated DeltaNet pierde selectividad por canal.

Cuadro 11: Compromisos de KDA en Kimi Linear (3B activados / 48B total) [23].

5.10. Análisis Comparativo: Taxonomía de Mecanismos con Compuerta

Arquitectura	Año de Publicación	Representación de Estado	Tipo de Compuerta	Complejidad de Entrenamiento	Complejidad de Inferencia	Ref
GLA	2023	Matriz (recurrente)	Decaimiento diagonal	$\mathcal{O}(Ld^2)$	$\mathcal{O}(d^2)$ memoria	[56]
DeltaNet	2024	Matriz (recurrente)	Escritura escalar (β)	$\mathcal{O}(Ld^2)$	$\mathcal{O}(d^2)$ memoria	[57]

Arquitectura	Año de Publicación	Representación de Estado	Tipo de Compuerta	Complejidad de Entrenamiento	Complejidad de Inferencia	Ref
Gated DeltaNet	2024	Matriz (recurrente)	Decaimiento + escritura	$\mathcal{O}(Ld^2)$	$\mathcal{O}(d^2)$ memoria	[58]
RetNet	2023	Matriz (recurrente)	Exponencial fija	$\mathcal{O}(Ld^2)$	$\mathcal{O}(d)$ por paso	[49]
HGRN2	2024	Matriz (recurrente)	Diagonal (acotada)	$\mathcal{O}(Ld^2)$	$\mathcal{O}(d^2)$ memoria	[40]
FoX	2025	Atención completa	Sesgo en espacio de logits	$\mathcal{O}(L^2d)$	$\mathcal{O}(L)$ caché KV	[28]
Gated max	2025	Atención completa	Sigmoide post-SDPA	$\mathcal{O}(L^2d)$	$\mathcal{O}(L)$ caché KV	[41]
Gated DeltaNet-2	2026	Matriz (recurrente)	Borrado + escritura decouplados (canalizado)	$\mathcal{O}(Ld^2)$	$\mathcal{O}(d^2)$ memoria	[21]
KDA	2025	Matriz (recurrente)	Decaimiento canalizado + escritura escalar	$\mathcal{O}(Ld^2)$	$\mathcal{O}(d^2)$ memoria	[23]

Arquitectura	Entrenable	Recuperación en Contexto	Recuperación Asociativa	Extrapolación de Longitud	Ventaja Clave
GLA	Sí	Moderada	Débil	Buena (20K)	Eficiencia en hardware; paralelismo por fragmentos
DeltaNet	Sí	Fuerte	Perfecta (MQAR)	Buena	Semántica de corrección de errores; recuperación sólida
Gated DeltaNet	Sí	Muy fuerte	Perfecta	Excelente	Equilibrada: olvido + actualizaciones dirigidas
RetNet	Sí	Débil	Débil	Buena	Simplicidad; no necesita caché KV
HGRN2	Sí	Moderada	Débil	Moderada	Modelado temporal multi-escala
FoX	Sí	Muy fuerte	Muy fuerte (casi perfecta)	Excelente	Preserva softmax; sobrecarga mínima
Gated max	Sí	Fuerte	Buena	Buena	Simplicidad; no reemplaza SDPA

Arquitectura Entrenable		Recuperación en Contexto	Recuperación Asociativa	Extrapolación de Longitud	Ventaja Clave
Gated DeltaNet-2	Sí	Muy fuerte	Muy fuerte	Excelente	Borrado/escritura canalizados decouplados; mejor RULER multi-key
KDA	Sí	Muy fuerte	Muy fuerte	Excelente	Decaimiento canalizado; desplegado en Kimi Linear (75 % reducción KV, 6× decodificación)

Arquitectura	Rendimiento Típico	Memoria por Inferencia	Caso de Uso Principal
GLA	Alto (CUDA por fragmentos)	$O(d^2)$ constante	Contexto largo con restricciones de memoria
DeltaNet	Medio-Alto	$O(d^2)$ constante	Recuperación y razonamiento asociativo
Gated DeltaNet	Medio	$O(d^2)$ constante	Producción: Qwen3-Next, rendimiento equilibrado
RetNet	Alto	$O(d)$ por paso	Inferencia extremadamente rápida; modelos simples
HGRN2	Medio-Alto	$O(d^2)$ constante	Patrones temporales multi-escala
FoX	Moderado (cuadrático)	$O(Ld)$ caché KV	Extrapolación de longitud; generación de contexto largo
Gated Softmax	Alto (sobrecarga mínima)	$O(Ld)$ caché KV	Mejora directa para modelos existentes
Gated DeltaNet-2	Alto (escalado casi plano)	$O(d^2)$ constante	Recuperación de contexto largo; multi-key NIAH
KDA	Alto (hasta 6× decodificación a 1M)	$O(d^2)$ constante	Producción híbrida (Kimi Linear); decodificación de contexto largo

5.11. Principio Unificado de Compuerta

Las ocho arquitecturas comparten un principio común: **la compuerta es un mecanismo para controlar el flujo de información y la retención selectiva de memoria**. Las decisiones de diseño específicas divergen a lo largo de varias dimensiones:

1. **Ubicación de la compuerta:** Actualizaciones de estado recurrente (GLA, DeltaNet, Gated DeltaNet, Gated DeltaNet-2, HGRN2) versus espacio de logits/salida softmax (FoX, Gated Softmax, RetNet).
2. **Granularidad de control:** Por dimensión (GLA, HGRN2 diagonal), por clave (DeltaNet), por token (FoX), por cabeza (Gated Softmax) o borrado/escritura canalizados decouplados (Gated DeltaNet-2).

3. **Dependencia de datos:** Completamente dependiente de datos (GLA, DeltaNet, Gated DeltaNet, Gated DeltaNet-2, FoX, Gated Softmax) versus esquemas fijos (RetNet con decaimiento exponencial).
4. **Compensación de expresividad:** Eficiencia recurrente lineal (GLA, DeltaNet, Gated DeltaNet, Gated DeltaNet-2, HGRN2, RetNet) versus expresividad completa de softmax (FoX, Gated Softmax).

5.12. Implementación y Orientación Práctica

5.12.1. Cuándo Usar Cada Arquitectura

1. **GLA:** Inferencia rápida con contextos moderados a largos (2K–20K tokens), cuando los kernels CUDA personalizados son aceptables y el rendimiento de recuperación no es crítico.
2. **DeltaNet:** Fuerte recuperación asociativa y tareas de aprendizaje en contexto; bueno para tareas sintéticas (MQAR) y pruebas de asociación clave-valor.
3. **Gated DeltaNet:** Modelos de producción que requieren el mejor equilibrio de recuperación, olvido y rendimiento (probado en Qwen3-Next; ICLR 2025).
4. **Gated DeltaNet-2:** Cargas de recuperación de contexto largo (multi-key NIAH) donde el borrado/escritura canalizado decouplado da el recuerdo más fuerte; cuando se dispone de un kernel WY consciente de las compuertas.
5. **RetNet:** Inferencia extremadamente eficiente sin almacenamiento en caché KV; adecuado para despliegues en dispositivos móviles/de borde o modelos de juguete.
6. **HGRN2:** Jerarquías temporales multi-escala; cuando los límites de olvido específicos por capa son deseables.
7. **FoX:** Extrapolación de longitud y comprensión de contexto largo; cuando el entrenamiento cuadrático es aceptable y no se desea reemplazar softmax.
8. **Gated Softmax:** Mejora rápida para modelos softmax existentes con cambios mínimos de código; mejor para profesionales que desean ganancias inmediatas sin una reestructuración importante.

5.12.2. Consideraciones de Hardware

Objetivo de Hardware	Arquitecturas Recomendadas	Notas
GPU (H100/A100)	Gated Softmax, FoX, Gated DeltaNet, Gated DeltaNet-2	Implementaciones maduras de softmax/lineal. GLA/DeltaNet requieren kernels personalizados.
TPU (alta memoria)	GLA, DeltaNet, Gated DeltaNet, Gated DeltaNet-2, HGRN2	Hardware soporta matmuls eficientes; los métodos recurrentes lineales brillan.
Móvil/Borde	RetNet, Gated Softmax ligero	Inferencia con memoria constante es crítica.

6. Introducción Conceptual—Transformers y Atención para Principiantes

Este apéndice proporciona una introducción accesible a los conceptos fundamentales de los transformers y el mecanismo de atención, dirigido a lectores sin conocimientos profundos previos en aprendizaje profundo. Los transformers son el motor detrás de los sistemas modernos de inteligencia artificial como ChatGPT, pero su funcionamiento puede entenderse mediante analogías del mundo real.

6.1. ¿Qué es un Transformer?

Un transformer es una arquitectura neuronal que procesa texto o secuencias de datos interpretando el significado de cada palabra mientras considera todas las demás palabras en contexto. Imagina que lees una oración:

“El banco estaba lleno de gente esperando. María fue al banco.”

¿Qué significa “banco” en cada caso? En la primera oración, se refiere a una institución financiera (o un asiento). En la segunda, no está claro sin contexto. Un transformer resuelve esto automáticamente al mirar todas las palabras circundantes. En la segunda oración, al ver “María fue”, el modelo comprende mejor que probablemente se trata de una ribera (donde va a nadar) o una institución financiera (donde va a realizar una transacción).

Esta capacidad de entender el significado completo de una palabra considerando toda la oración se llama *contextualización*, y es el corazón de cómo funcionan los transformers modernos.

6.2. El Mecanismo de Atención: Una Analogía

El mecanismo de atención es el “truco mágico” que permite a los transformers entender el contexto. Piensa en ello como participar en una conversación importante en una habitación ruidosa:

1. **Mucho ruido:** Alguien está hablando y es muy importante para ti.
2. **Ignoras el resto:** Tu cerebro automáticamente enfoca la atención en esa persona, ignorando otros sonidos.
3. **Entiendes mejor:** Al concentrarte, captas cada palabra claramente.

El mecanismo de atención neuronal funciona de la misma manera: cada palabra “pregunta” qué tan importante es cada otra palabra para entender su significado, luego “enfoca” su atención en las más relevantes.

6.3. Ejemplo Práctico Paso a Paso

Veamos cómo un transformer entiende la palabra “come” en dos contextos diferentes:

6.3.1. Contexto 1: “El gato come pescado”

El transformer, al procesar “come”, pregunta internamente:

- ¿Qué tan relevante es “El”? Poco (es solo un artículo). **Atención baja.**
- ¿Qué tan relevante es “gato”? Muy relevante (es el sujeto, quien come). **Atención alta.**
- ¿Qué tan relevante es “pescado”? Muy relevante (es lo que se come). **Atención alta.**

Por lo tanto, la representación mental de “come” se enfoca principalmente en “gato” y “pescado”.

6.3.2. Contexto 2: “El restaurante come los márgenes de beneficio”

Aquí el transformer pregunta en un contexto diferente:

- ¿Qué tan relevante es “restaurante”? Muy relevante (es el sujeto). **Atención alta.**
- ¿Qué tan relevante es “beneficio”? Muy relevante (se ve afectado). **Atención alta.**
- ¿Qué tan relevante es “márgenes”? Muy relevante (es lo que se consume). **Atención alta.**

La palabra “come” obtiene una representación completamente diferente porque “atiende” a palabras distintas en contextos distintos.

6.4. Cómo Funciona la Atención Matemáticamente (Versión Simple)

Detrás del cambio de atención hay matemáticas. Aquí está la versión simplificada sin demasiado detalle técnico:

6.4.1. Paso 1: Consultas, Claves y Valores

Cada palabra en la oración se transforma en tres versiones:

- **Consulta:** “¿Qué información necesito saber?”
- **Clave:** “¿Qué información tengo?”
- **Valor:** “Aquí está mi información importante.”

Imagina en una biblioteca, cada visitante es una “consulta”, cada libro es una “clave” y el contenido es el “valor”. El bibliotecario (atención) empareja las consultas con las claves más relevantes para acceder al valor correcto.

6.4.2. Paso 2: Compatibilidad

El sistema examina qué tan *compatible* es cada consulta con cada clave. Las consultas similares a las claves obtienen un puntaje de compatibilidad *alto*. Esto se calcula multiplicando la consulta por la clave (matemáticamente, el producto punto).

6.4.3. Paso 3: Enfoque

Los puntajes de compatibilidad se convierten en “pesos de enfoque”. Las compatibilidades altas significan “enfocar atención aquí”, y las bajas significan “ignorar esto”. Matemáticamente, una función llamada **softmax** convierte los puntajes en porcentajes (como: 40 % atención aquí, 35 % aquí, 25 % allá).

6.4.4. Paso 4: Combinar

Finalmente, el sistema combina todos los valores, ponderados por los pesos de enfoque. Si una palabra tiene un 80 % de atención en el sujeto, el 80 % de la información del sujeto se mezcla en la representación de esa palabra.

6.5. Múltiples Cabezas de Atención: Múltiples Perspectivas

Una característica clave es que los transformers no usan una única atención sino *múltiples cabezas de atención* en paralelo. Es como si tuvieras 8 personas analizando la oración *simultáneamente*, cada una prestando atención a diferentes aspectos:

- **Persona 1:** “Me enfoco en las relaciones sujeto-verbo.”
- **Persona 2:** “Busco el objeto directo.”
- **Persona 3:** “Sigo la información sobre el tiempo verbal.”
- **Persona 4:** “Busco modificadores y adjetivos.”

Cada “cabeza” aprende a enfocarse en diferentes patrones del lenguaje. Juntas, capturan una comprensión mucho más rica que una sola cabeza.

6.6. Apilar Capas

Los transformers procesan información en *múltiples capas* (generalmente de 12 a 48 para modelos prácticos). Imagina editar un ensayo:

1. **Primera pasada:** Corrige ortografía y gramática básica.
2. **Segunda pasada:** Mejora la claridad y la estructura de las oraciones.
3. **Tercera pasada:** Asegura consistencia y flujo narrativo.

Los transformers funcionan de la misma manera: cada capa refina la comprensión del texto. La primera capa captura características básicas (palabras simples, géneros), mientras que las capas posteriores entienden conceptos complejos (relaciones entre palabras, significados abstractos).

6.7. ¿Por Qué es Importante?

El mecanismo de atención fue revolucionario porque:

1. **Paralelismo:** Encuentra contexto para *cualquier palabra con cualquier otra palabra*, sin procesarlas secuencialmente (a diferencia de sistemas anteriores). Esto lo hace muy rápido.
2. **Flexibilidad:** Aprende qué patrones buscar automáticamente a partir de los datos, sin necesidad de programar reglas manualmente.
3. **Escalabilidad:** Funciona desde textos pequeños hasta contextos con millones de palabras.
4. **Capacidades generales:** El mismo mecanismo funciona para traducción, resumen, preguntas-respuestas, generación de texto, visión por computadora y más.

6.8. Desafíos Prácticos

A pesar del extraordinario éxito de los transformers, presentan desafíos:

6.8.1. Complejidad Computacional

Si una oración tiene 1,000 palabras, el mecanismo de atención estándar debe comparar cada palabra con las otras 1,000 palabras. Eso es $1,000 \times 1,000 = 1,000,000$ de comparaciones. Para documentos largos con millones de palabras, esto se vuelve prohibitivamente costoso en tiempo y memoria.

6.8.2. Requisitos de Memoria

Especialmente durante la inferencia (cuando se usan modelos entrenados), almacenar la matriz de atención completa puede consumir enormes cantidades de RAM o memoria de GPU, limitando las longitudes de secuencia que se pueden procesar.

6.8.3. Eficiencia del Dispositivo

Los transformers fueron diseñados para TPUs y GPUs potentes. Ejecutarlos en teléfonos o dispositivos integrados es un desafío.

6.9. Soluciones Modernas

Para resolver estos desafíos, la investigación ha propuesto muchas variantes:

- **Atención Dispersa:** En lugar de comparar cada palabra con TODAS las demás, compara solo con un subconjunto estratégico (vecinos cercanos, patrones periódicos). Reduce la complejidad de $\mathcal{O}(n^2)$ a $\mathcal{O}(n \log n)$ o $\mathcal{O}(n)$.
- **Modelos Recurrentes:** Como Mamba, incorporan aspectos de las antiguas redes neuronales recurrentes pero con mejor eficiencia moderna.
- **Atención con Compuerta:** Mecanismos que aprenden selectivamente qué información pasar hacia adelante, reduciendo las necesidades de almacenamiento.
- **Cuantización:** Usar números más pequeños (enteros en lugar de decimales) para reducir la memoria sin perder demasiada precisión.

Estas innovaciones son lo que este kit de herramientas (Frankenstein) te permite experimentar fácilmente.

6.10. Conclusiones Clave

- Un **transformer** es una red neuronal que entiende el contexto del lenguaje muy bien.
- El **mecanismo de atención** permite que cada palabra “se enfoque” en qué otras palabras son relevantes para entender su significado.
- La atención funciona calculando la **compatibilidad** entre cada palabra (consulta) y todas las demás (claves), luego combinando información basada en esa compatibilidad (valores).
- **Múltiples cabezas** de atención exploran diferentes patrones en paralelo.
- **Múltiples capas** refinan progresivamente la comprensión.
- El principal desafío es que la atención estándar tiene complejidad cuadrática, lo que es costoso para textos largos.
- Muchas **soluciones modernas** (atención dispersa, modelos recurrentes, cuantización) abordan estos desafíos, y este kit de herramientas te permite explorarlas todas.

7. Normalización, Cuantización, Enrutamiento de Profundidad y Algoritmos de Ejecución

Este anexo recopila el detalle a nivel de implementación para las variantes de normalización, cuantización ternaria estilo BitNet, enrutamiento adaptativo de profundidad (Mixture-of-Depths), memoria condicional (Engram), embeddings factorizados y los algoritmos de ejecución (controles de estabilidad del paso de entrenamiento, enrutador de inferencia SBERT) que soportan la tubería impulsada por configuración. Las secciones principales difieren a este anexo para las formulaciones matemáticas y el pseudocódigo; las claves bibliográficas citadas aquí se resuelven contra `docs/bibliography/`.

7.1. Variantes de Normalización: LayerNorm, RMSNorm, Tanh Dinámico y Erf Dinámico

La normalización determina cómo se controla la escala de activación a través de la profundidad. En este repositorio, la normalización no es solo una elección de modelado sino también una cuestión de compatibilidad con el esquema, porque solo ciertos valores son actualmente aceptados por `norm_type`. El contrato del esquema es:

$$\text{norm_type} \in \{\text{layer_norm}, \text{dynamic_tanh}, \text{derf}, \text{rms_norm}, \text{prms_norm}\}$$

Las formulaciones más relevantes para este código base son:

7.1.1. LayerNorm (Línea Base)

LayerNorm [3] es el normalizador por defecto; centra y reescala cada token por su media y varianza por token:

$$\mu = \frac{1}{d} \sum_{i=1}^d x_i, \quad \sigma^2 = \frac{1}{d} \sum_{i=1}^d (x_i - \mu)^2, \quad y_i = \gamma_i \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta_i.$$

7.1.2. RMSNorm

RMSNorm elimina la centralización de la media y solo reescala por la magnitud de la raíz cuadrada media [63]:

$$\text{RMS}(x) = \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \epsilon}, \quad y_i = \gamma_i \frac{x_i}{\text{RMS}(x)}.$$

En comparación con LayerNorm, RMSNorm es computacionalmente más simple (sin resta de la media de características) y se usa a menudo cuando es importante reducir la sobrecarga de normalización (p. ej., Llama, GPT-NeoX). Tiene una escala γ aprendible por dimensión pero sin término de bias.

7.1.3. RMSNorm Parcial (pRMSNorm)

pRMSNorm [63] reduce aún más la sobrecarga estimando el RMS de solo el primer $p\%$ de las dimensiones ocultas:

$$\text{RMS}(x) = \sqrt{\frac{1}{k} \sum_{i=1}^k x_i^2 + \epsilon}, \quad k = \lceil d \cdot p \rceil,$$

explotando el supuesto de que las neuronas dentro de una capa son aproximadamente i.i.d. La misma escala γ se aplica a todas las dimensiones. La razón parcial p se controla mediante el campo de configuración `prms_partial_ratio` (por defecto 6,25 %). El paper reporta que se pueden lograr resultados competitivos con tan solo el 6,25 % de las entradas, aunque puede surgir inestabilidad de gradiente con razones muy pequeñas.

7.1.4. Tanh Dinámico (DyT)

El Tanh Dinámico propone reemplazar la normalización explícita con un mapeo elemento a elemento acotado [68]:

$$\text{DyT}(x) = \tanh(\alpha x)$$

donde α se aprende. La idea central es que la contracción no lineal acotada puede proporcionar un escalado de señal estable sin calcular explícitamente estadísticas de normalización por token.

7.1.5. Erf Dinámico (Derf)

Derf extiende la misma dirección libre de normalización usando un mapeo basado en la función error [8]:

$$\text{Derf}(x) = \text{erf}(\alpha x + s)$$

con escala/desplazamiento aprendibles. Los resultados reportados en el trabajo citado indican un rendimiento superior al de DyT y las líneas base de normalización comunes en múltiples dominios.

7.1.6. Comparación

Método	Fórmula	Estadísticas Necesarias	Notas
LayerNorm	$\gamma_i(x_i - \mu)/\sqrt{\sigma^2 + \epsilon} + \beta_i$	media + varianza	Por defecto; centrado y reescalado completo [3].
RMSNorm	$\gamma_i x_i / \sqrt{\frac{1}{d} \sum_j x_j^2 + \epsilon}$	Solo RMS	Menor sobrecarga; línea base ampliamente usada [63].
pRMSNorm	$\gamma_i x_i / \sqrt{\frac{1}{k} \sum_{j \leq k} x_j^2 + \epsilon}$	RMS parcial	RMS del primer p % de dims; <code>prms_partial_ratio</code> por defecto 6,25 % [63].
Tanh Dinámico	$\tanh(\alpha x)$	ninguna	Transformación acotada libre de normalización; reemplazo directo [68].
Erf Dinámico	$\text{erf}(\alpha x + s)$	ninguna	Alternativa libre de normalización; mejora sobre DyT [8].

7.2. Ruta de Cuantización Ternaria BitNet

Cuando `use_bitnet=true`, las capas BitLinear reemplazan a `nn.Linear` estándar con cuantización ternaria de pesos $(-1, 0, +1)$ siguiendo BitNet [53]. Dado el tensor de pesos W , un escalado práctico es:

$$s = \text{mean}(|W|), \quad \tilde{W} = \text{clip} \left(\text{round} \left(\frac{W}{s} \right), -1, 1 \right).$$

El mapeo empaquetado utiliza dos bits por símbolo de peso para eficiencia de almacenamiento. Esto permite una compresión significativa ($\sim 32\times$ frente a FP32) pero es experimental y puede afectar la calidad del modelo; es más adecuado para el despliegue de inferencia.

La cuantización de activaciones para la ruta INT8 es:

$$q = \text{round} \left(x \cdot \frac{127}{\text{máx}(|x|) + \epsilon} \right), \quad q \in [-128, 127]$$

con des-cuantización $x \approx q/\alpha$. Para N parámetros, las estimaciones de tamaño antes de metadatos y sobrecarga de empaquetado son:

$$\text{Tamaño FP32} \approx 4N, \quad \text{Tamaño FP16} \approx 2N, \quad \text{Tamaño 1.58-bit} \approx \frac{1,58}{8}N$$

lo que se alinea con objetivos de despliegue ligero [44, 7].

7.3. Enrutamiento de Tokens Mixture-of-Depths (MoD)

Cuando `use_mixture_of_depths=true`, cada bloque del transformer puntúa los tokens con un enrutador ligero. Solo el subconjunto superior de `mixture_of_depths_capacity_ratio` se actualiza; los tokens omitidos pasan sin cambios [69]. Esto asigna más profundidad a los tokens destacados, reduciendo el cómputo por capa.

- 1: **Entrada:** estados ocultos $H \in \mathbb{R}^{n \times d}$, enrutador R , ratio de capacidad ρ
- 2: $\text{puntajes} \leftarrow R(H)$ ▷ puntaje del enrutador por token
- 3: $\mathcal{S} \leftarrow \text{top-k}(\text{puntajes}, k = \lceil \rho \cdot n \rceil)$ ▷ seleccionar subconjunto destacado
- 4: $H_{\text{actualizar}} \leftarrow \text{bloque}(H[\mathcal{S}])$ ▷ aplicar bloque transformer a tokens seleccionados
- 5: $H[\mathcal{S}] \leftarrow H_{\text{actualizar}}$ ▷ escribir de vuelta; los tokens no seleccionados bypassan el bloque
- 6: **Salida:** H ▷ pérdida auxiliar de balance de carga añadida desde `mixture_of_depths_router_aux_loss_weight`

7.3.1. Configuración

- `use_mixture_of_depths`: bool — habilitar enrutamiento MoD de tokens.
- `mixture_of_depths_capacity_ratio`: float en $(0, 1]$ — fracción de tokens actualizados por capa.
- `mixture_of_depths_router_aux_loss_weight`: float ≥ 0 — peso de pérdida auxiliar para regularización del enrutador.

7.4. Memoria Condicional Engram

Engram (`engram_attn`) implementa memoria condicional mediante búsqueda N-gram escalable [10]. Construye tablas de búsqueda basadas en hash para contextos N-gram desde tamaño 2 hasta `engram_max_ngram_size`, con cabezas hash independientes por orden N-gram. Una convolución causal depthwise (`engram_kernel_size`) proporciona contexto de corto alcance antes de la búsqueda N-gram.

- 1: **Entrada:** flujo de tokens $x_{1:n}$, órdenes N-gram $\{2, \dots, N\}$, cabezas hash por orden H
- 2: $h_{\text{conv}} \leftarrow \text{DepthwiseConv1d}(x_{1:n}, k = \text{engram_kernel_size})$ ▷ contexto de corto alcance
- 3: **for** cada orden N-gram $k \in \{2, \dots, N\}$ **do**
- 4: $\text{ctx}_t^{(k)} \leftarrow x_{t-k+1:t}$ ▷ ventana de contexto N-gram
- 5: $\text{lookup}_t^{(k)} \leftarrow \text{HashTable}^{(k)}[\text{hash}(\text{ctx}_t^{(k)})]$ ▷ recuperación por cabeza, H cabezas por orden
- 6: **end for**
- 7: $y_t \leftarrow \sum_k \text{lookup}_t^{(k)} + h_{\text{conv},t}$ ▷ combinar memorias condicionales
- 8: **Salida:** $y_{1:n}$

7.4.1. Configuración

- `engram_max_ngram_size`: int ≥ 2 — orden N-gram más alto (por defecto 3).
- `engram_n_heads_per_ngram`: int ≥ 1 — cabezas hash por orden N-gram (por defecto 4).

- `engram_embed_dim_per_head`: $\text{int} \geq 1$ — dimensión de embedding por cabeza hash (por defecto 32).
- `engram_kernel_size`: $\text{int} \geq 1$ — ancho del kernel de conv causal depthwise (por defecto 4).
- `engram_seed`: int — semilla hash para reproducibilidad (por defecto 42).

Tamaño oculto total de Engram = $(\text{max_ngram_size}-1) \times \text{n_heads_per_ngram} \times \text{embed_dim_per_head}$.

7.5. Embeddings Factorizados y Convolución de Embedding

7.5.1. Embeddings Factorizados

Cuando `use_factorized_embedding=true`, la matriz de embedding se factoriza en dos matrices más pequeñas, reduciendo parámetros de $O(V \times H)$ a $O(V \times R + R \times H)$ donde $R = \text{factorized_embedding_dim}$.

- `use_factorized_embedding`: bool — habilitar factorización.
- `factorized_embedding_dim`: $\text{int} \geq 1$ — dimensión intermedia (típico: 64–256).

7.5.2. Convolución de Embedding

Cuando `use_embedding_conv=true`, se aplica una Conv1d depthwise sobre el flujo de embeddings antes del primer bloque transformer, con tamaño de kernel `embedding_conv_kernel`. Esto proporciona un filtro ligero de contexto local sobre embeddings de tokens.

7.6. Controles de Estabilidad del Paso de Entrenamiento

El paso de entrenamiento guiado por esquema aplica acumulación de gradientes, recorte de norma global, guardas de explosión post-recorte y reintentos acotados de NaN/Inf. El gradiente acumulado es:

$$g_{\text{acc}} = \frac{1}{K} \sum_{i=1}^K g_i, \quad K = \text{gradient_accumulation_steps}$$

$$g_{\text{clip}} = g_{\text{acc}} \cdot \min\left(1, \frac{\tau}{\|g_{\text{acc}}\|_2 + \epsilon}\right), \quad \tau = \text{grad_clip_max_norm}$$

luego los guardas de desbordamiento usan `inf_post_clip_threshold` y lógica de reintentos limitada por `max_nan_retries`.

7.7. Enrutador de Modo de Inferencia SBERT

El despachador de inferencia SBERT selecciona entre puntuación de similitud, búsqueda sobre corpus, agrupamiento y codificación persistente sobre un único codificador compartido [43]. La similitud coseno y la pérdida de regresión son:

$$\cos(e_1, e_2) = \frac{e_1^\top e_2}{\|e_1\| \|e_2\|}, \quad \mathcal{L}_{\text{cos}} = (\cos(e_1, e_2) - y)^2$$

con $y \in [-1, 1]$ en este pipeline.

Algorithm 40 Paso de Entrenamiento Guiado por Esquema con Controles de Estabilidad

Require: Flujo de lotes, configuración C

```
1: Inicializar contador de reintentos  $r \leftarrow 0$ 
2: for cada paso del optimizador do
3:   Acumular gradientes para  $K = C.\text{gradient\_accumulation\_steps}$  micro-lotes
4:   Aplicar recorte de norma global con  $\tau = C.\text{grad\_clip\_max\_norm}$ 
5:   if el gradiente post-recorte excede  $C.\text{inf\_post\_clip\_threshold}$  o se detecta NaN/Inf
   then
6:     if  $r < C.\text{max\_nan\_retries}$  then
7:       restaurar estado seguro / saltar paso;  $r \leftarrow r + 1$ 
8:       continue
9:     else
10:      detener entrenamiento con estado de fallo
11:    end if
12:  end if
13:  ejecutar paso del optimizador seleccionado por  $\text{optimizer\_class}$ 
14:  actualizar scheduler (cosine, constant, o  $\text{linear\_warmup\_then\_constant}$ )
15:  if  $\text{paso mod checkpoint\_every\_n\_steps} = 0$  then
16:    guardar checkpoint rodante y podar a  $\text{max\_rolling\_checkpoints}$ 
17:  end if
18:  actualizar mejores checkpoints hasta  $\text{num\_best\_checkpoints}$ 
19:  emitir CSV + telemetría según  $\text{gradient\_log\_interval}$  y  $\text{telemetry\_log\_interval}$ 
20: end for
```

Algorithm 41 Enrutador de Modo de Inferencia SBERT

Require: modo m , modelo E , entradas X

```
1: if  $m = \text{similarity}$  then
2:   retornar  $\cos(E(x_1), E(x_2))$ 
3: else if  $m = \text{search}$  then
4:   retornar top- $k$  por producto-punto/coseno contra embeddings del corpus
5: else if  $m = \text{cluster}$  then
6:   retornar etiquetas de clustering sobre  $E(X)$ 
7: else
8:   retornar embeddings serializados  $E(X)$ 
9: end if
```
